

Introduction to matrix-product states and tensor networks

Grégoire Misguich¹

¹Université Paris-Saclay, CEA, CNRS, Institut de Physique Théorique
91191 Gif-sur-Yvette, France
gregoire.misguich@ipht.fr

June 15, 2026

Abstract

These notes provide an introduction to tensor-network (TN) methods, with an emphasis on matrix-product states (MPS) and their applications to lattice quantum many-body physics, including the simulation of open quantum systems. Some of the central concepts and algorithms used to represent and simulate quantum many-body states with TNs are presented. The topics covered include graphical notations for TN and MPS, entanglement properties, gauge freedom and canonical forms of MPS, QR and singular value decompositions and their role in MPS algorithms, an overview on the density-matrix renormalization group (DMRG) algorithm and time-evolution methods, selected examples of projected entangled-pair states (PEPS) and the extension of TN methods to mixed states for the simulation of thermal states and open quantum systems. These notes are intended to be practical and they are accompanied by several code examples based on the Julia libraries `ITensor`, `ITensorMPS`, and `TensorMixedStates`.

This document was written at the occasion of a course given at the 9th *Les Houches Summer School on Computational Physics: Open Quantum Systems*, held in June 2026. This text is a work in progress, and feedback is very welcome.

Contents

1	Introduction	3
1.1	What are TNs useful for?	3
1.1.1	Compressed representation of quantum many-body states	4
1.1.1.1	Ground-state search	4
1.1.1.2	Time evolution	6
1.1.1.3	Thermal states, mixed states and open quantum systems	6
1.1.1.4	Simulating quantum circuits	6
1.1.2	Classifying quantum phases of matter	6
1.1.3	Classical statistical mechanics	7
1.1.4	Tensor cross interpolation	7
1.2	Reviews and lecture notes	7
1.3	Software and libraries	7
2	MPS and associated algorithms	8
2.1	Definition	8
2.2	Gauge redundancy	10
2.3	A few examples	10
2.3.1	GHZ state	10
2.3.2	W state	11
2.3.3	Valence-bond crystal	11
2.3.4	AKLT state	11
2.4	MPS construction by successive QR and canonical forms	13
2.4.1	Construction of a left-canonical MPS by successive QR decompositions	13
2.4.2	Making an MPS left or right canonical	14
2.4.3	Mixed-canonical form and orthogonality center	15

2.4.4	Entanglement entropy and MPS compression from mixed-canonical form	16
2.4.5	Expectation value of a local observable	17
2.5	Elementary manipulations	17
2.5.1	Addition of two MPS	17
2.5.2	Scalar product	18
2.5.3	Correlation function	18
2.6	Matrix product operators (MPO) and local Hamiltonians	20
2.6.1	Example 1: transverse-field Ising chain	21
2.6.2	Example 2: Heisenberg model	21
2.6.3	Example 3: 2-qubit gate	22
2.6.4	Finite-state automaton picture	22
2.6.5	Multiplication of an MPO by an MPS and zip-up algorithm	23
2.7	DMRG algorithm	25
2.8	Time evolution with MPS	28
2.8.1	Time-evolving block decimation (TEBD) algorithm	28
2.8.2	Time-dependent variational principle (TDVP) algorithm	30
2.8.3	MPO methods (W^I and W^{II})	30
3	Projected entangled pair states (PEPS)	32
3.1	Area law and difficulty of treating 2d systems with MPS	33
3.2	Example 1: PEPS derived from a classical thermal state	35
3.3	Example 2: Toric code	36
3.4	Other examples	38
3.5	Contraction of PEPS	38
4	Mixed states and open systems	38
4.1	Vectorization and TN for a mixed state	39
4.2	Operator-space entanglement entropy	41
4.3	Thermal equilibrium states by purification	41
4.4	Superoperators as MPOs	42
4.5	Lindblad dynamics in MPS form	45
	Acknowledgements	47
	Appendixes	47
A	Singular-value decomposition (SVD) and QR factorization	48
A.1	SVD and Schmidt decompositions	48
A.2	QR	49
B	Injective MPS	50
B.1	Map from the virtual space to the physical space	50
B.2	Space of matrices spanned by products of MPS matrices	50
B.3	Parent Hamiltonian of an injective MPS	51
C	MPS representation of the AKLT state	52
	References	53

List of code examples

1	Simple Néel-state MPS	12
2	Random state to MPS	14
3	MPS orthogonality center	15
4	MPS transfer matrix	20
5	Finite-range transverse-field Ising Hamiltonian as an MPO	23
6	DMRG for the spin- $\frac{1}{2}$ Heisenberg chain – energy and entanglement entropy	27
7	DMRG for the spin-1 AKLT chain	27
8	DMRG for 1d free fermions	28

9	TDVP for domain-wall melting in XX spin chain	30
10	Exact time evolution with W^H MPO for onsite Hamiltonian	32
11	DMRG for 2d free fermions	34
12	Thermal equilibrium state of an XX chain by purification and TDVP	42
13	Noisy random-circuit simulation	44
14	MPO bond dimensions for Lindbladians	45
15	Lindblad dynamics of a dissipative XX chain	46
16	Lindblad dynamics and OSEE in a dissipative XX chain	46

1 Introduction

These notes provide a beginner-friendly introduction to tensor-network (TN) methods in quantum many-body physics, with an emphasis on matrix-product states (MPS) and their applications to lattice systems, including the simulation of open quantum systems.

A substantial part of the notes is devoted to MPS. The reason is that, among TNs, MPS are the best understood, the easiest to manipulate numerically, and the most widely used, with a history spanning roughly three decades. They also provide the simplest setting in which to introduce central notions of the TN language: virtual indices and bond dimensions, tensor contractions, reshaping and factorization, gauge freedom, truncations, matrix-product operators, and variational algorithms such as the density-matrix renormalization group (DMRG) and time-evolution methods. Many of these ideas have natural generalizations to more complex TN, such as projected entangled-pair states (PEPS), which are briefly discussed.

The presentation is intended to be concrete and practical, and is accompanied by several code examples [1] illustrating the main ideas. These examples use the Julia libraries `ITensor` and `ITensorMPS` [2]. Some examples involving dissipative or mixed-state systems use the `TensorMixedStates` library [3], which is built on top of `ITensor`.

To make full use of the code examples, the reader is expected to have some familiarity with models such as spin chains or simple free fermion Hamiltonians. Very basic notions from quantum information theory—in particular bipartite entanglement, reduced density matrices, Schmidt decomposition, and entanglement entropy—will be used throughout the notes, with only brief reminders.

The TN literature is vast, with applications extending well beyond lattice quantum many-body problems. These include quantum computing and the simulation of quantum circuits, classical statistical mechanics, and the use of tensor cross interpolation in a variety of non-quantum settings. Although these notes focus on lattice quantum many-body physics, we have included a few references that may serve as entry points into other areas of application of TN.

1.1 What are TNs useful for?

TNs have profoundly changed the way we think about many-body quantum physics, from condensed matter physics to quantum computing. Instead of viewing a many-body state as a vector in an exponentially large Hilbert space, they offer structured objects whose essential content is the structure of the entanglement between spatial regions. They provide a language to describe and manipulate such states, and a framework for designing algorithms.

While an (exponentially large) vector state representation can by definition encode any state, TN representations are adapted to encode states with particular patterns of entanglement and some underlying low-rank structure. They brought a major computational breakthrough: by encoding only the relevant entanglement, they make it possible to represent and manipulate several types of states that are relevant to physics (in particular low-energy states of local Hamiltonians and those obeying area-law entanglement) with resources that grow in a controlled way, rather than exponentially with system size.

At the same time, tensor-network methods are not a universal remedy for the exponential complexity of quantum many-body physics. Highly entangled states (those obeying volume-law entanglement in particular), long-time dynamics, critical systems, and generic contractions in dimensions higher than one are examples of situations in which the computational cost can still grow prohibitively large.

As will be briefly mentioned below, TN applications extend beyond quantum many-body physics and quantum information, and are growing rapidly across physics and related fields. Some of these applications will be mentioned in the following, although these notes focus on TN applications to (lattice) quantum many-body physics.

1.1.1 Compressed representation of quantum many-body states

A central application of TNs is to provide “compressed” representations of quantum many-body states. Here, compression means that TNs can be used to represent (exactly or approximately) certain states using a number of parameters that is much smaller than the dimension of the full Hilbert space. This turns out to be particularly useful in the context of numerical simulations of quantum many-body systems. A crucial quantity in this context is the bipartite entanglement entropy of the state, which quantifies the amount of quantum correlations between two subsystems.¹ Generally, the more entangled a state is, the more parameters (larger tensors and more memory) are needed to represent it as a TN. The number of parameters in a TN can be varied by changing the so-called bond dimensions of the TN. The associated indices are called virtual or auxiliary indices. TNs can be viewed as wave-function ansatzes with tunable entanglement properties. By increasing the bond dimension of the TN, one can increase the amount of entanglement that can be represented and thus improve the accuracy of the approximation of a given state. There exist many approaches to tackle quantum many-body problems, such as exact diagonalization, quantum Monte Carlo, series expansions, mean-field decoupling, variational wave functions, etc. They all have their own advantages and limitations, and they are often complementary to each other. From this perspective, TNs are quite different from the approaches mentioned above since they are adapted to represent low-entangled states, and in particular states with an area law of entanglement.

TNs can be used to encode states of lattice spin models, but fermionic systems can also be treated. In 1d, the Jordan-Wigner transformation allows one to treat fermionic models essentially as spin models. In higher dimensions it is also possible to study interacting fermions using TNs, but a different technique (not discussed in these notes) is needed to maintain the locality of the Hamiltonian [4].

The system often has a finite size, but it is also possible to work directly in the thermodynamic limit (infinite system size) using infinite TNs [5–7]. Many algorithms have been developed to work directly on an infinite system, by considering an infinite TN with a repeated unit cell. For simplicity, however, these notes will focus on finite TNs and on algorithms formulated for finite systems.

MPS can also be extended to continuous models and quantum field theories [8]. The fractional quantum Hall effect is an example of a domain where problems are defined in the continuum and where MPS have been used successfully [9–11]. There, Landau level orbitals provide a natural way to formulate the Hamiltonians in a discrete basis adapted to an MPS formulation, in spherical or cylindrical geometries.

In practice, such approaches are efficient for ground states or low-energy states of local Hamiltonians in low dimensions (typically 1d and 2d). States obeying the area law of entanglement [12] can be efficiently represented as TNs, while states with volume-law entanglement (such as generic highly excited states) can generally not be well represented by TNs unless the bond dimension is allowed to grow exponentially with the system size (in which case the TN representation is not useful).

MPS [13]² are a particular class of TNs that are well suited to represent low-entangled states of 1d quantum systems. They can sometimes also be used in 2d [15]. Among all TNs, MPS are the best understood, the easiest to manipulate numerically, and the most developed both from the theoretical and numerical points of view. MPS are now the standard framework for performing numerical simulations of 1d quantum systems.

Another class of TNs are the PEPS [8, 16], which are a natural generalization of MPS to higher dimensions. The associated states can support area-law entanglement in $d > 1$ (unlike MPS, which can only support area-law entanglement in 1d), and they have mostly been used in $d = 2$.

1.1.1.1 Ground-state search Several algorithms exist to find the ground state (or low-energy excited states) of Hamiltonians as TNs. This is, for instance, the idea of the Density Matrix Renormalization Group (DMRG) algorithm [20–22], which constructs (approximate) ground states in the space of MPS. This algorithm will be described in more detail in Sec. 2.7.

Once a state is expressed as an MPS, it is possible (and often easy) to *efficiently* compute expectation values of observables, correlation functions, entanglement properties, and more. Here, *efficiently* means that the computational cost scales polynomially with the system size and with the TN bond dimensions, avoiding operations that would be *exponential* in N .³

¹A common way to quantify such bipartite entanglement is to compute the von Neumann entropy of one of the subsystems. The von Neumann entropy of a subsystem A is defined as $S_{\text{vN}} = -\text{Tr}(\rho_A \log \rho_A)$ where ρ_A is the reduced density matrix of the subsystem A . $\rho_A = \text{Tr}_B |\psi\rangle\langle\psi|$ is obtained by tracing out the degrees of freedom of the complementary subsystem B from the pure state $|\psi\rangle$ of the full system. The von Neumann entropy is a measure of the entanglement between the subsystem A and its complement. It is zero for product states (no entanglement) and can be as large as $\log d_A$ for maximally entangled states, where d_A is the dimension of the Hilbert space of subsystem A .

²In the mathematical literature MPS are also known as tensor-train decompositions [14]

³Obtaining expectation values of observables in a state represented by a TN such as a PEPS is in general more complex

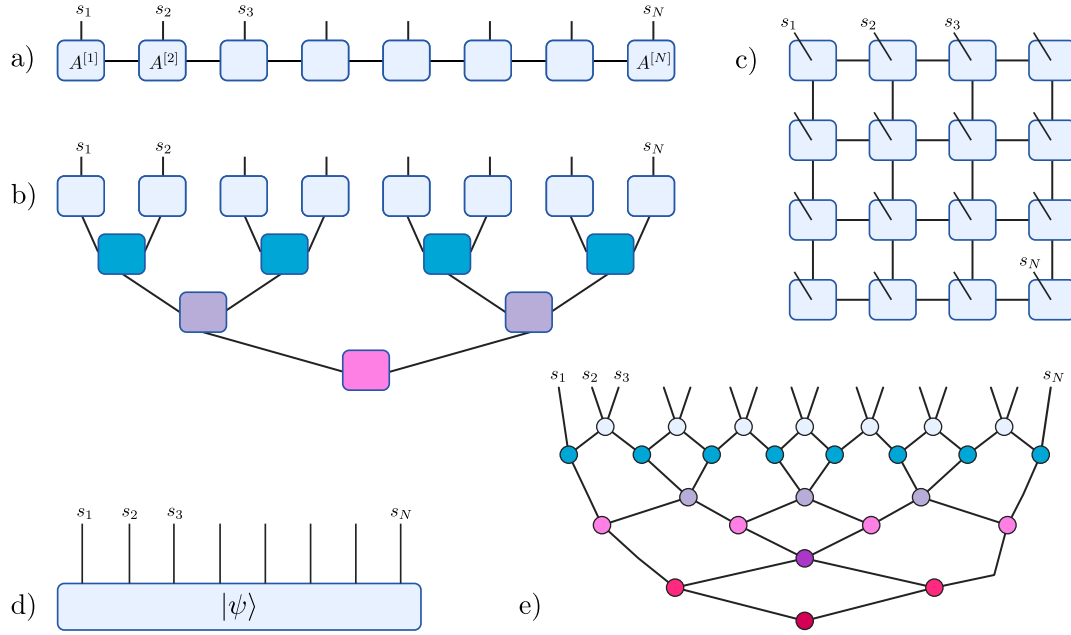


Figure 1: Examples of TNs: a) MPS for a chain of length 8, b) binary tree TN for a system of 8 sites, c) projected entangled pair state for a system with 4×4 sites in a square lattice geometry, d) many-body wave function represented as a single N -leg tensor and e) multi-scale entanglement renormalization ansatz (MERA) TN [17–19]. Each box (or circle) represents a tensor, and each leg represents an index of the tensor. In this representation the open legs (labeled $s_1, \dots, s_N$) that are not connected to any other tensor represent the physical indices. They correspond to the degrees of freedom of the physical system (e.g., spin states, where for a spin- S system such indices can take $2S + 1$ values). The legs that connect two tensors represent the virtual indices, which encode the entanglement between different parts of the system. The dimensions of the virtual indices, called the bond dimensions, determine the amount of entanglement that can be represented. Larger bond dimensions allow for a more accurate representation of entangled states.

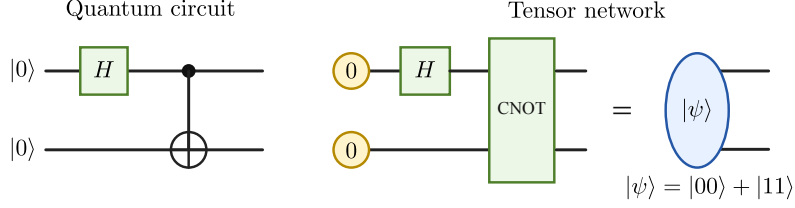


Figure 2: The Bell state $|\psi\rangle = |00\rangle + |11\rangle$ viewed as a two-index tensor, as the output of a quantum circuit, and as the corresponding TN. H is the Hadamard gate (matrix) $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ and $CNOT$ is $CNOT = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X$, where X is the Pauli X matrix.

1.1.1.2 Time evolution Computing time evolution is also an important domain where TNs are very useful. A typical application is to compute the time evolution $|\psi(t)\rangle = e^{-iHt} |\psi(0)\rangle$ of an initial state $|\psi(0)\rangle$ under the action of a Hamiltonian H . It is also possible to study problems where the Hamiltonian is time-dependent. We can in particular mention the tDMRG algorithm [23–25], TEBD [26, 27], the time-dependent variational principle (TDVP) for MPS [28, 29], or the W^I and W^II algorithms [30]. Note that such dynamics often leads to some growth of the entanglement entropy. The bond dimension of the TN representation of the state will then need to grow as well (to maintain the precision of the approximation). This can lead to a difficulty known as the *entropy barrier*, which limits the time up to which the time evolution can be accurately computed with TN methods. A very similar difficulty arises when simulating quantum circuits with TNs, where the circuit depth plays a role similar to time in the Hamiltonian evolution case. Finding methods that are able to overcome this entropy barrier is currently an active area of research in the field of TNs and quantum many-body dynamics [31, 32].

1.1.1.3 Thermal states, mixed states and open quantum systems TNs can also be used to represent thermal states of quantum many-body systems, such as the density matrix $\rho = e^{-\beta H}/Z$, where H is the Hamiltonian of the system and $Z = \text{Tr}(e^{-\beta H})$ is the partition function. It is also possible to simulate the dynamics of open quantum systems. In both cases, this can be done by extending the TN descriptions to *mixed states* [33, 34]. In the case of MPS, this will be discussed in more detail in Sec. 4.1.

1.1.1.4 Simulating quantum circuits The simulation of quantum circuits is another important field of application of TNs (and of MPS in particular).

First, a quantum circuit is nothing but a special type of TN. In such a representation, each qubit wire carries an index. The initial state is often a product state, so that the state of each qubit can be represented by a one-index tensor. For instance, if all qubits are initialized in the state $|0\rangle$, one attaches a copy of the vector $|0\rangle$ to each input leg. The unitary gates appearing in the circuit are then represented as tensors: a one-qubit gate is a rank-2 tensor, a two-qubit gate is a rank-4 tensor, and so on. After all internal indices have been contracted, the TN associated with the circuit evaluates a transition amplitude. More precisely, for an output bitstring $x_1 x_2 \dots x_N$, with $x_i \in \{0, 1\}$, one attaches the corresponding dual vectors $\langle x_i|$ to the output legs. The full contraction then gives $\langle x_1 x_2 \dots x_N | U_{\text{circuit}} | 0 \dots 0 \rangle$. An example is shown in Fig. 2. This basic observation also makes clear why TN contraction is closely related to the classical simulation of quantum circuits and quantum computers.

The case of noisy quantum circuits is particularly interesting. The effect of noise and decoherence can be modeled by non-unitary quantum channels, and the output of the circuit is no longer a pure state, but a mixed state described by a density matrix ρ . The simulation of such noisy quantum circuits can be done using TNs and MPS in particular. Since the noise generally limits the growth of entanglement, it is possible to simulate larger systems than in the ideal (unitary) case. In fact, in the field of quantum computing, calculations based on MPS have raised the bar concerning the performance that quantum processors must exceed in order to offer a quantum advantage [35, 36].

1.1.2 Classifying quantum phases of matter

Beyond the computational aspects mentioned above, TNs and MPS in particular have also been used as tools to classify phases of matter. MPS have in particular been used to prove the absence of intrinsic

and can usually not be carried out exactly in polynomial time. Approximate methods are then used to compute such expectation values. See Sec. 3.5 for more details.

topological order in 1d and to classify gapped symmetry-protected phases in 1d [37–39]. It has been proven that, in the absence of symmetry, all injective MPS (appendix B) are smoothly connected⁴ to a product state, and they therefore belong to the topologically trivial phase. However, when global symmetries are imposed, gapped states without spontaneous symmetry breaking form distinct phases. These are symmetry-protected topological phases, classified in the MPS formalism by the projective representation carried by the virtual indices of the MPS tensors.

1.1.3 Classical statistical mechanics

In the field of non-equilibrium classical statistical physics, a notable achievement is the formulation of the steady state of a totally asymmetric exclusion process (TASEP) as an MPS [40]. This seminal work opened the way to the use of MPS in the study of the dynamics of 1d classical stochastic models [41].

When computing the partition function of a 2d classical model, it can be useful to consider the associated transfer matrix, which is a linear operator that describes how the system evolves from one row to the next. The DMRG algorithm has been used as early as 1995 [42] to compute the dominant eigenvector of such transfer matrices, which gives access to the thermodynamic properties of the system. Along this line, another important development is the use of iTEBD to study the partition function of infinite classical models [43]. As for the 2d TN known as PEPS in the context of quantum many-body physics, they were originally introduced in the context of 3d classical models [16].

1.1.4 Tensor cross interpolation

Tensor Cross Interpolation (TCI) is a learning and compression algorithm that reconstructs a large tensor, vector, or function as a compact MPS, using only selected point evaluations rather than enumerating the full exponentially large object [44]. Once the function has been put in MPS or MPO form, the usual TN toolbox can be applied to it, for instance to contract sums, apply operators, or solve linear problems. TCI has a growing number of applications, and we can mention in particular the representation of multivariate functions in quantics⁵ form [45], fast numerical differentiation, integration, convolution, Fourier transforms, and solving PDEs. Ref. [46] provides a recent introduction to TCI and its applications.

1.2 Reviews and lecture notes

There are many reviews and lecture notes on TNs and MPS in particular. We mention here a few of them, but this list is certainly far from being exhaustive. General beginner-friendly reviews and lecture notes on TNs include for instance (chronological order): Orus [47], Bridgeman *et al.* [48], Silvi *et al.* [49], Cirac *et al.* [50], Bañuls [51], Vancraeynest-De Cuiper *et al.* [52]. For beginners, the reviews by Orus [47] and Bañuls [51] are particularly useful starting points. Concerning MPS in particular, the review from Schollwöck [22] is one of the main references on MPS and DMRG. We also wish to mention the review on MPS methods for time evolution by Paeckel *et al.* [53]. Finally, we recommend the very comprehensive lecture notes by J. von Delft [54].

1.3 Software and libraries

Many open-source libraries for doing simulations with TNs and MPS are available, providing low-level routines for basic tensor operations to high-level functions that implement algorithms such as DMRG, time evolution, and more. Here we just mention three of the most widely used ones:

- **ITensor** is in Julia and is one of the most widely used libraries for working with TNs and MPS in particular. See [2]. Most of the examples in these notes are based on this library and its MPS extension **ITensorMPS**.

⁴Two states are smoothly connected if there exists a continuous path of Hamiltonians such that i) the initial and final states are the ground states of the initial and final Hamiltonians, respectively and ii) the gap of the Hamiltonian does not close along the path. An equivalent definition is that two states are smoothly connected if one can obtain one from the other by applying a finite-depth local unitary circuit, i.e., a quantum circuit with a finite number of layers of local unitary gates.

⁵As an illustration, consider a function f of a single real variable x defined on the interval $[0, 1]$. The interval is discretized into 2^N points, so that each point $x_{\mathbf{s}}$ can be labeled by a binary string $\mathbf{s} = (s_1, s_2, \dots, s_N)$, where $s_i \in \{0, 1\}$ and $x_{\mathbf{s}} = \sum_{i=1}^N s_i 2^{-i}$ in the interval $[0, 1]$. On the discretized set, f becomes a function of N binary variables (like the wave function of a spin chain), which can be represented as an MPS with N physical indices. This is the so-called quantics representation of f . Increasing N amounts to increasing (exponentially) the number of discretization points, but the MPS representation of f can be kept compact if f has some underlying low-rank structure. From there, numerous operations such as differentiation, integration, convolution, and Fourier transforms can be performed using MPS–MPO tools.

- **Quimb** is a Python library for quantum information calculation that is focused on TNs [55].
- **TenPy** is in Python and is another popular library [56, 57].

We also indicate `tensornetwork.org` which is a website with many TN resources, applications, and a longer list of software libraries [58]. `tensors.net` is another website with resources on TNs, including tutorials [59].

Finally, `TensorMixedStates` is a recent Julia library built on top of `ITensor` for working with MPS representations of mixed states [3] and which is particularly adapted to simulate open quantum systems. A few code examples using it are included in these notes; see code examples #10, #13 and #15.

2 MPS and associated algorithms

We now turn to MPS, which are one of the simplest yet powerful classes of TNs. They are particularly well suited to represent low-entangled states of 1d quantum systems, and they have been widely used for three decades in the field of quantum many-body physics. They also provide the simplest setting in which to introduce several central notions of the TN language.

2.1 Definition

Consider N sites, each site having a local Hilbert space of dimension d . An MPS of bond dimension χ is a state of the form

$$|\psi\rangle = \sum_{s_1, s_2, \dots, s_N} A^{[1] s_1} A^{[2] s_2} \dots A^{[N] s_N} |s_1 s_2 \dots s_N\rangle \quad (1)$$

where $A^{[i] s_i}$ are $\chi \times \chi$ matrices, except for the first and last sites, where they are $1 \times \chi$ and $\chi \times 1$, respectively (this ensures that the product of the A_i matrices evaluates to a scalar). The parameter χ is called the bond dimension of the MPS. It is sometimes also useful to consider *periodic* MPS, where the first and last tensors are also $\chi \times \chi$ matrices, and the state is obtained by taking the trace of the product of the N matrices:

$$|\psi\rangle = \sum_{s_1, s_2, \dots, s_N} \text{Tr}(A^{[1] s_1} A^{[2] s_2} \dots A^{[N] s_N}) |s_1 s_2 \dots s_N\rangle. \quad (2)$$

Another notation consists in absorbing the physical index into the entries of a matrix. This defines a matrix $\mathcal{A}^{[i]}$ whose entries are single-site basis states of the type $|s_i\rangle$, rather than complex numbers. More precisely:

$$\mathcal{A}_{\alpha_{i-1}, \alpha_i}^{[i]} = \sum_{s_i} A_{\alpha_{i-1}, \alpha_i}^{[i] s_i} |s_i\rangle. \quad (3)$$

For open boundary conditions, the MPS is recovered as

$$|\psi\rangle = \sum_{\alpha_1, \dots, \alpha_{N-1}} \mathcal{A}_{1, \alpha_1}^{[1]} \otimes \mathcal{A}_{\alpha_1, \alpha_2}^{[2]} \otimes \dots \otimes \mathcal{A}_{\alpha_{N-1}, 1}^{[N]}. \quad (4)$$

If χ is large enough, any state can be represented as an MPS. In particular, an arbitrary state can be put in the form of an MPS with χ at most equal to $d^{N/2}$ (see Sec. 2.4). But in practice, for many physically relevant states, a good approximation can be obtained with a much smaller χ (even $\chi = \mathcal{O}(1)$ for some states).

The MPS representation is particularly well suited to represent low-entangled states of 1d quantum systems. For instance, the ground state of a gapped local Hamiltonian in 1d can be efficiently approximated by an MPS with a bond dimension that does not grow with the system size.

TNs and MPS in particular can be represented graphically, as shown in Fig. 1. For an MPS, each tensor $A^{[k]}$ is represented as a box, with three legs corresponding to its three indices. The vertical legs correspond to the physical indices $s_k = 1, \dots, d$ of the local Hilbert space, while the horizontal legs represent the virtual indices of dimension χ that are contracted between neighboring tensors. For an open MPS the first and last tensors have only one horizontal leg.

The graphical representation of MPS and TNs in general is a very useful tool to visualize the structure of the states, of the operators, and also to manipulate them. It allows us to represent operations such as contractions, reshaping, SVD, etc. in a simple and intuitive way. Graphical representations of a few elementary tensor operations are shown in Fig. 3. A more complete description of the graphical rules associated with the most common operations can be found in Sec. 2 of [48]. The most common ones will appear in the figures of these notes.

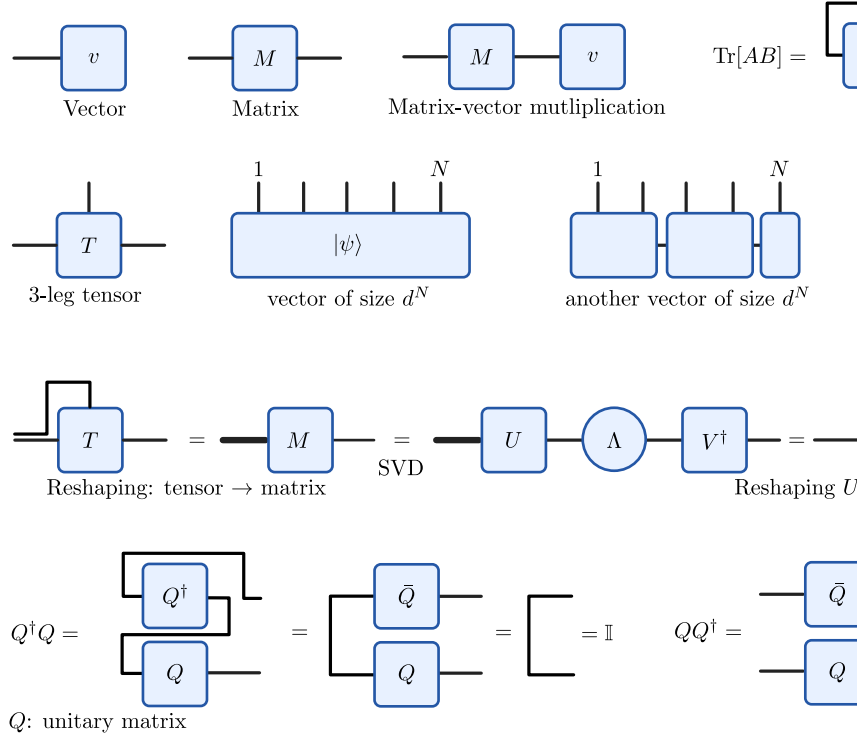


Figure 3: Graphical representation of a few tensor examples and associated operations.

2.2 Gauge redundancy

The number of parameters in an MPS described by Eq. (2) is $Nd\chi^2$ (for N sites, each with d local states, and each matrix A_i^s having χ^2 parameters). But there is a *gauge* redundancy in the MPS representation: one can multiply the matrices $A^{[i]s_i}$ by invertible matrices X_i without changing the state $|\psi\rangle$. More precisely, the MPS defined by the matrices $A^{[i]s_i} \rightarrow A^{[i]s_i} X_{i-1}^{-1} X_i$ describes the same state as the original MPS (each X_i will simplify with its inverse X_i^{-1} in the product of matrices in Eq. (2)). This gauge redundancy plays an important role in the theory and algorithms of MPS. For instance, it is often useful to choose a particular gauge (for instance the left or right canonical gauge, see Sec. 2.4), which simplifies calculations. This gauge property is also important for the classification of phases of matter using MPS, and for understanding symmetries and topological properties of MPS states.

2.3 A few examples

We start with a trivial case: product states. They correspond to MPS with bond dimension $\chi = 1$.

$$|\psi\rangle = |\phi_1\rangle \otimes |\phi_2\rangle \otimes \cdots \otimes |\phi_N\rangle$$

can be represented as an MPS with bond dimension $\chi = 1$. The matrix $A^{[i]s}$ is just a scalar that gives the amplitude of the local state $|\phi_i\rangle$ in the local basis: $A^{[i]s} = \langle s | \phi_i \rangle$. Or simply

$$\mathcal{A}^{[i]} = |\phi_i\rangle. \quad (5)$$

2.3.1 GHZ state

The GHZ state is defined as the equal superposition of all sites in state $|0\rangle$ and all sites in state $|1\rangle$:

$$|\text{GHZ}\rangle = \frac{1}{\sqrt{2}}(|00\dots 0\rangle + |11\dots 1\rangle). \quad (6)$$

It can be represented as an MPS with bond dimension $\chi = 2$ by choosing the following tensors (in Eq. (2)):

$$A^{[k]0} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad A^{[k]1} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \quad (7)$$

for $k = 1, \dots, N$. The product of two consecutive matrices $A_k^s A_{k+1}^{s'}$ gives zero if $s \neq s'$. The only non-zero contributions to the sum in Eq. (2) are then for $s_1 = s_2 = \dots = s_N = 0$ and for $s_1 = s_2 = \dots = s_N = 1$, which gives the GHZ state (up to a normalization factor $\sqrt{2}$). One can also represent the GHZ state as an open MPS. In this case, the first and last tensors are 1×2 and 2×1 matrices, respectively, while the tensors in the bulk are 2×2 matrices. For instance, one can choose

$$A^{[1]0} = (1/\sqrt{2} \quad 0), \quad A^{[1]1} = (0 \quad 1/\sqrt{2}) \quad (8)$$

for the first site, and

$$A^{[N]0} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad A^{[N]1} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (9)$$

for the last site. The matrices in the bulk (sites $k = 2, \dots, N-1$) are the same as for the periodic representation Eq. (7).

Using the state-valued matrix notation of Eq. (3), this open-boundary representation can equivalently be written as

$$\mathcal{A}^{[1]} = \frac{1}{\sqrt{2}} (|0\rangle \quad |1\rangle), \quad \mathcal{A}^{[N]} = \begin{pmatrix} |0\rangle \\ |1\rangle \end{pmatrix}, \quad (10)$$

and, for $k = 2, \dots, N-1$,

$$\mathcal{A}^{[k]} = \begin{pmatrix} |0\rangle & 0 \\ 0 & |1\rangle \end{pmatrix}. \quad (11)$$

Again, the zero entries denote the zero vector in the one-site Hilbert space.

2.3.2 W state

The W state is defined as the equal superposition of all states with exactly one site in state $|1\rangle$ and the remaining sites in state $|0\rangle$:

$$|W\rangle = \frac{1}{\sqrt{N}}(|100\dots 0\rangle + |010\dots 0\rangle + \dots + |000\dots 1\rangle). \quad (12)$$

It can also be represented as an MPS with bond dimension $\chi = 2$. On the first and last sites one can choose (state-valued matrix notation)

$$\mathcal{A}^{[1]} = \frac{1}{\sqrt{N}} \begin{pmatrix} |0\rangle & |1\rangle \end{pmatrix}, \quad \mathcal{A}^{[N]} = \begin{pmatrix} |1\rangle \\ |0\rangle \end{pmatrix}, \quad (13)$$

and, for $k = 2, \dots, N-1$,

$$\mathcal{A}^{[k]} = \begin{pmatrix} |0\rangle & |1\rangle \\ 0 & |0\rangle \end{pmatrix}. \quad (14)$$

Going from left to right in the product of Eq. (4), the auxiliary index $\alpha \in \{1, 2\}$ of the MPS tensors keeps track of whether we have already encountered a site in state $|1\rangle$ ($\alpha = 2$) or not ($\alpha = 1$). The matrix $\mathcal{A}^{[k]0}$, which corresponds to a $|0\rangle$, does not change this information. On the other hand, the matrix $\mathcal{A}^{[k]1}$ changes the auxiliary index from $\alpha = 1$ to $\alpha = 2$. It corresponds to the fact that one has encountered a site in state $|1\rangle$. Once we have encountered such a $|1\rangle$ state, the auxiliary index remains $\alpha = 2$ until the last site. This ensures that there is a single site in state $|1\rangle$ in the superposition.

2.3.3 Valence-bond crystal

A valence-bond crystal (VBC) is a state of a spin- $\frac{1}{2}$ system where some pairs of neighboring spins form singlets. In 1d and for an even number of sites N , the state reads

$$|\Psi_{\text{VBC}}\rangle = \bigotimes_{k=1}^{N/2} \frac{1}{\sqrt{2}} \left(|\uparrow_{2k-1}\downarrow_{2k}\rangle - |\downarrow_{2k-1}\uparrow_{2k}\rangle \right).$$

Using the notation of Eq. (3), this can be written as an open-boundary MPS with alternating tensors

$$\mathcal{A}^{[2k-1]} = \begin{pmatrix} |\uparrow\rangle & |\downarrow\rangle \end{pmatrix}, \quad \mathcal{A}^{[2k]} = \frac{1}{\sqrt{2}} \begin{pmatrix} |\downarrow\rangle \\ -|\uparrow\rangle \end{pmatrix}, \quad k = 1, \dots, N/2. \quad (15)$$

The contraction of the virtual index inside one pair gives $\mathcal{A}^{[2k-1]}\mathcal{A}^{[2k]} = \frac{1}{\sqrt{2}} (|\uparrow_{2k-1}\downarrow_{2k}\rangle - |\downarrow_{2k-1}\uparrow_{2k}\rangle)$, so the product of the alternating MPS tensors gives the VBC state above. The bond dimension is 1 (no entanglement) between sites $2k$ and $2k+1$, while the bond dimension is $\chi = 2$ between sites $2k-1$ and $2k$ (singlet state).

2.3.4 AKLT state

In 1987 Affleck, Kennedy, Lieb and Tasaki (AKLT) introduced a spin-1 chain Hamiltonian with a ground state that can be exactly represented as an MPS with bond dimension $\chi = 2$ [60]. The idea is to start from a state of $2N$ spin- $\frac{1}{2}$ degrees of freedom, where each pair of neighboring spin- $\frac{1}{2}$ degrees of freedom is in a singlet state. This state can be represented as an MPS with bond dimension $\chi = 2$. This state is a paradigmatic example of a state with short-range correlations and non-trivial (symmetry-protected) topological properties. Its construction (detailed in appendix C) from virtual spin- $\frac{1}{2}$ degrees of freedom is also at the heart of the construction of PEPS in higher dimensions.

By projecting the two spin- $\frac{1}{2}$ degrees of freedom at each site onto the $S = 1$ triplet subspace, one obtains a state of N spin-1 degrees of freedom that can also be represented as an MPS with bond dimension $\chi = 2$ (Fig. 4). Now consider the following spin-1 Hamiltonian:

$$H = \sum_{i=1}^{N-1} \Pi_{i,i+1}^{S=2} \quad (16)$$

where $\Pi_{i,i+1}^{S=2}$ is the projector onto the total spin $S_{\text{tot}} = 2$ subspace of the two neighboring spin-1 degrees of freedom at sites i and $i+1$. This projector can be constructed explicitly by noting that the square of

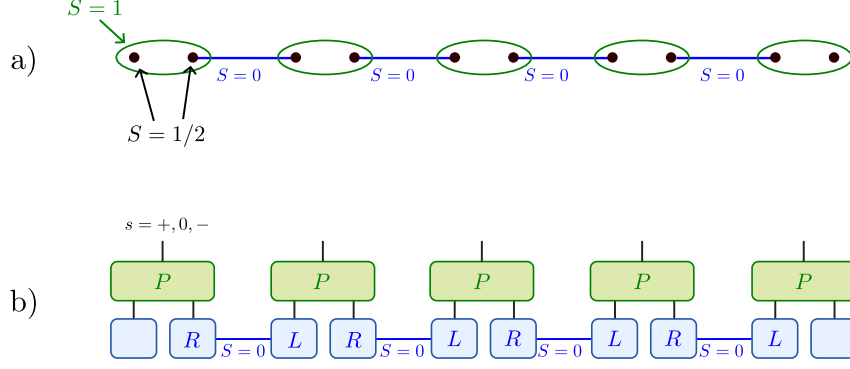


Figure 4: AKLT state for a spin-1 chain. a) The Hilbert space is enlarged by replacing each spin-1 (green ovals) by two fictitious spin- $\frac{1}{2}$ (black dots). The state of the $2N$ spin- $\frac{1}{2}$ degrees of freedom is a product of spin singlet states between neighboring spin- $\frac{1}{2}$ degrees of freedom (blue bonds). b) This product of singlets is an MPS with bond dimension $\chi = 2$ for the fictitious spins. By projecting the two spin- $\frac{1}{2}$ degrees of freedom at each site onto the $S = 1$ triplet subspace (operator P), one obtains a state for the original spin-1 chain that can also be represented as an MPS with bond dimension $\chi = 2$. Each tensor of this MPS is obtained by contracting three tensors: P , L and R .

the total spin $S_{\text{tot}}^2 = (\vec{S}_i + \vec{S}_j)^2 = 2\vec{S}_i \cdot \vec{S}_j + 4$ has three eigenvalues $S_{\text{tot}}(S_{\text{tot}} + 1)$ with $S_{\text{tot}} = 0, 1, 2$. The projector onto the $S_{\text{tot}} = 2$ subspace is thus $\Pi_{i,j}^{S=2} = \frac{1}{24} S_{\text{tot}}^2 (S_{\text{tot}}^2 - 2)$. After a few lines of elementary algebra one finds

$$H = \frac{1}{6} \sum_{i=1}^{N-1} ((\vec{S}_i \cdot \vec{S}_{i+1})^2 + 3\vec{S}_i \cdot \vec{S}_{i+1} + 2) \quad (17)$$

By construction, any state $|\psi\rangle$ that has no component in the $S_{\text{tot}} = 2$ subspace of any pair of neighboring sites is annihilated by the above Hamiltonian since it gives $\Pi_{i,i+1}^{S=2} |\psi\rangle = 0$. Since H is a sum of projectors, its eigenvalues are necessarily ≥ 0 . A state that is annihilated by H is thus a ground state. By construction, two neighboring spin-1 sites have a total spin at most equal to 1 since two of the four spin- $\frac{1}{2}$ degrees of freedom on these two spin-1 sites are in a singlet state. So, the state of Fig. 4 satisfies $\Pi_{i,i+1}^{S=2} |\psi\rangle = 0$. This state is thus a ground state of the above Hamiltonian. The construction shown schematically in Fig. 4 also shows that this state can be represented as an MPS with bond dimension $\chi = 2$. This state is usually called the AKLT state or valence bond solid (VBS) state. The derivation is detailed in appendix C, together with the explicit expression of the resulting MPS tensors.

Readers interested in additional simple MPS examples can for instance refer to [47], which discusses the Majumdar-Ghosh state and the 1d cluster state.

Code example 1: Creation of a simple (Néel) MPS state for a system of spin- $\frac{1}{2}$ using the `ITensorMPS` library in Julia. The code builds the site indices, chooses the alternating product-state configuration `Up,Dn,Up,Dn,...`, converts it directly into an MPS, and prints the maximal bond dimension. Since this is a product state, the bond dimension remains minimal. Before executing the code above, install the `ITensor` and `ITensorMPS` libraries by running the following commands in the Julia REPL: `using Pkg ; Pkg.add("ITensors") ; Pkg.add("ITensorMPS")`.

[mps.jl](#) on GitHub.

Code example 1: Simple Néel-state MPS

```
1 using ITensors
2 using ITensorMPS
3 N = 10
4 sites = siteinds("S=1/2", N)
5 states = [isodd(j) ? "Up" : "Dn" for j in 1:N]
6 psi_neel = MPS(sites, states)
7 println("max. bond dim. = ", maxlinkdim(psi_neel))
```

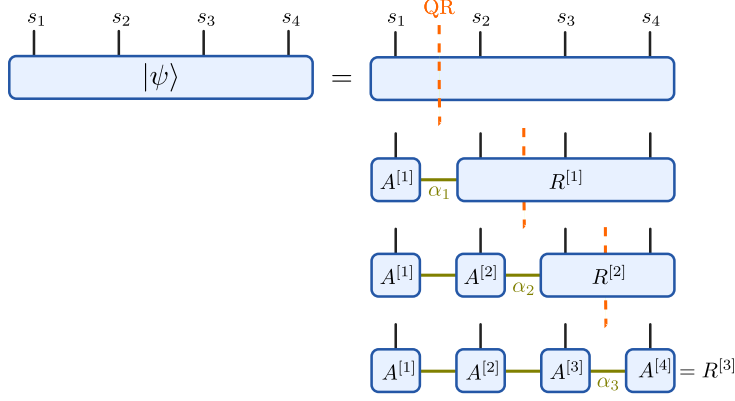


Figure 5: Construction of an MPS from a vector representation by successive QR decompositions. At each step, the Q matrix gives an MPS tensor $A^{[i]}$, while the R matrix is reshaped and factorized again at the next site. Such a construction leads to an MPS in a left-canonical gauge (see Fig. 8).

2.4 MPS construction by successive QR and canonical forms

2.4.1 Construction of a left-canonical MPS by successive QR decompositions

Starting from a generic state $|\psi\rangle$ expressed in the local basis $|s_1 s_2 \dots s_N\rangle$, one can construct an MPS representation of this state by performing successive QR decompositions (appendix A.2). Such a construction is illustrated in Fig. 5 for an $N = 4$ -site state with coefficients $\psi_{s_1 s_2 s_3 s_4}$. First, one *reshapes* this tensor into a matrix with row index s_1 and combined column index (s_2, s_3, s_4) . This matrix, of size $d \times d^{N-1}$, is then factorized using a QR decomposition. The Q matrix is interpreted as the first MPS tensor $A_{s_1, \alpha_1}^{[1]}$, while the R matrix is reshaped as a new tensor $R_{\alpha_1, s_2, s_3, s_4}^{[1]}$. One then groups (α_1, s_2) as the row index and (s_3, s_4) as the column index of $R^{[1]}$ and repeats the QR decomposition, producing $A_{\alpha_1 s_2 \alpha_2}^{[2]}$ and $R_{\alpha_2 s_3 s_4}^{[2]}$. Repeating this step gives $A_{\alpha_2 s_3 \alpha_3}^{[3]}$ and a final tensor $A_{\alpha_3 s_4}^{[4]}$, yielding an open-boundary MPS. An example of code, based on the `ITensor` library, which transforms a general state (here random) into an MPS by successive QR decompositions is given in code example #2.

Remark

The decomposition depends on the way the qubits have been ordered. This is an important point to keep in mind when working with MPS, as the choice of the ordering can have a significant impact on the efficiency of the representation. If the original physical system is 1d, there is usually a natural ordering of the qubits that follows the geometry of the system. However, for higher-dimensional systems or systems with more complex geometries, the choice of ordering can be more subtle. Figure 21 shows an example of a 2d system where the qubits are ordered in a snake-like pattern to create a 1d MPS representation. Other choices would however be possible, and may lead to more efficient representations depending on the structure of the state.

The MPS produced by this construction is said to be in the *left-canonical gauge*, which means that the tensors $A^{[i]}$ satisfy the left-orthonormalization condition $\sum_{s_i} (A^{[i] s_i})^\dagger A^{[i] s_i} = I$ for all i . This is a consequence of the fact that the Q matrices obtained from the QR decomposition are unitary (or isometric in the case of reduced QR). This property of the A tensors is illustrated in Fig. 8 and is often exploited to simplify certain calculations. In particular, consider a cut between site n and site $n + 1$ and fix the value of the associated bond index α_n . The partial contraction of the first n tensors defines a state in the Hilbert space of the first n sites:

$$|\alpha_n\rangle_L = \sum_{s_1, \dots, s_n} \left(A^{[1] s_1} A^{[2] s_2} \dots A^{[n] s_n} \right)_{\alpha_n} |s_1 \dots s_n\rangle. \quad (18)$$

If the MPS is left-canonical, these states form an orthonormal family in the Hilbert space of the first n sites

$${}_L \langle \alpha_n | \beta_n \rangle_L = \delta_{\alpha_n, \beta_n}, \quad (19)$$

as illustrated in Fig. 6.

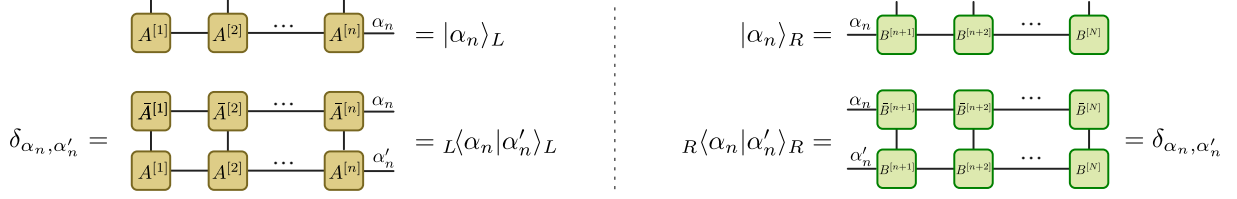


Figure 6: Partial contractions of a left-canonical MPS and a right-canonical MPS. For each value of the open virtual index α_n , the left contraction defines a state $|\alpha_n\rangle_L$ in the Hilbert space of the first n sites, while the right contraction defines a state $|\alpha_n\rangle_R$ in the Hilbert space of sites $n+1, \dots, N$. The corresponding canonical conditions imply that these states form orthonormal families. See also Fig. 8.

Code example 2: Julia code using the `ITensor` and `ITensorMPS` libraries to generate a random state of N spin- $\frac{1}{2}$ degrees of freedom as a dense vector of dimension 2^N , convert it to an MPS, and print the bond dimensions of the resulting MPS. As expected for a random state, the bond dimension between sites i and $i+1$ takes its maximum possible value $\chi_i = 2^{\min(i, N-i)}$. Output of the code: `bond dimensions = [2, 4, 8, 16, 32, 64, 128, 64, 32, 16, 8, 4, 2]`.

[random_state_to_MPS.jl](#) on GitHub.

Code example 2: Random state to MPS

```

1 using LinearAlgebra
2 using ITensors
3 using ITensorMPS
4
5 N = 14
6 psi = randn(2^N)
7 psi ./= norm(psi)
8
9 ITensors.disable_warn_order()
10 sites = siteinds(2, N)
11 psi_mps = MPS(psi, sites)
12
13 bond_dims = [dim(commonind(psi_mps[j], psi_mps[j + 1])) for j in 1:(N - 1)]
14 println("bond dimensions = ", bond_dims)

```

Remark

What are the bond dimensions obtained by rewriting a generic state $|\psi\rangle$ with local physical dimension d as an MPS? The first QR decomposition involves a matrix of size $d \times d^{N-1}$. The first bond index, α_1 , thus takes d values. At the next step, the tensor $R_{\alpha_1, s_2, s_3, \dots, s_N}^{[1]}$ is reshaped into a matrix with row index (α_1, s_2) and column index $(s_3, \dots, s_N)$, of size $d^2 \times d^{N-2}$, so that the second bond dimension is d^2 . More generally, at step $i \leq N/2$ the QR decomposition is applied to a “wide” matrix of size $d^i \times d^{N-i}$, leading to a unitary Q matrix of size $d^i \times d^i$ and a wide upper triangular R matrix of size $d^i \times d^{N-i}$. Beyond the middle of the chain, the matrices to be QR-decomposed are “tall” matrices of size $d^i \times d^{N-i}$ with $i > N/2$. In such a case one instead uses a *reduced* QR decomposition (appendix A.2) where the matrix Q is now rectangular of size $d^i \times d^{N-i}$, and the R matrix is square of size $d^{N-i} \times d^{N-i}$. The associated bond dimension is thus d^{N-i} for $i > N/2$. Thus, as expected, for a generic state the QR decompositions give bond dimensions

$$\chi_i = d^{\min(i, N-i)}. \quad (20)$$

The code in code example #2 confirms this for a random state of spin- $\frac{1}{2}$ ($d = 2$), where the bond dimensions are found to be $\chi_i = (2, 4, 8, 16, 32, 64, 128, 64, 32, 16, 8, 4, 2)$.

2.4.2 Making an MPS left or right canonical

The construction of an MPS from a vector representation is conceptually important, but in most cases MPS are used for large systems where the full vector representation (size d^N) is too large to be tractable. On the other hand, it is frequent to perform a similar series of QR decompositions starting from an MPS,

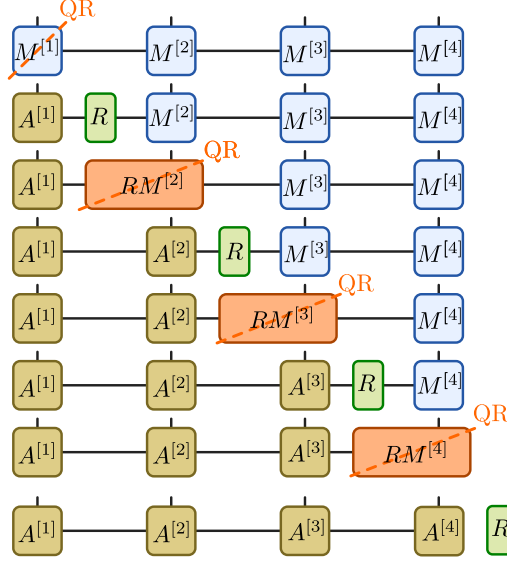


Figure 7: Transformation of an MPS into left-canonical form by successive QR decompositions. At each step, the active tensor is reshaped into a matrix, QR-decomposed, and the resulting R tensor is contracted into the next tensor on the right. The Q tensors produced in this way become the left-canonical tensors $A^{[i]}$.

in order to bring it to a canonical gauge. The construction of a left-canonical MPS from a generic MPS by successive QR decompositions is illustrated in Fig. 7. One starts from the leftmost tensor of the MPS, reshapes it into a matrix, and performs a QR decomposition. The Q matrix gives the first left-canonical tensor $A^{[1]}$, while the R matrix is contracted into the next tensor on the right. One then repeats this step for the second site, and so on until the end of the chain. The last QR decomposition involves a matrix with a single column, so that the Q matrix is a vector and the R matrix is a scalar. This scalar is the normalization factor of the MPS ($\langle \psi | \psi \rangle = |R|^2$), which can be set to 1 by rescaling the last tensor $A^{[N]}$. Performing a similar series of QR decompositions in the other direction, from right to left, gives an MPS in the right-canonical gauge. The resulting tensors $B^{[i]}$ satisfy the right-orthonormalization condition $\sum_{s_i} B^{[i] s_i} (B^{[i] s_i})^\dagger = I$ for all i (see Fig. 6 and Fig. 8).

The existence of such canonical forms is not a peculiarity of MPS, but rather a consequence of the fact that the underlying TN has no loops. For any loop-free TN, and in particular for tree TNs (TTN), one can choose a root tensor and successively QR-decompose the tensors along the branches so that all tensors away from the root become isometries, defining an orthogonality center analogous to the mixed-canonical form of an MPS [49, 61]. By contrast, the presence of loops, as in PEPS, prevents one from gauging all environments to identities at the same time; exact contraction is generically hard, and tensor truncation or variational optimization then requires approximate environments, making the corresponding algorithms substantially more involved [47, 62].

2.4.3 Mixed-canonical form and orthogonality center

It is also useful to mix the two sweeps above to obtain a *mixed-canonical form* of the MPS, where the tensors to the left of a given site k are left-canonical, the tensors to the right of site k are right-canonical, and the tensor at site k is unconstrained, as illustrated in Fig. 8. Starting from the left boundary, one performs QR decompositions only up to site $k - 1$, absorbing each remaining factor into the tensor on its right; the tensors at sites $1, \dots, k - 1$ are then left-canonical. Starting from the right boundary, one performs the analogous right-to-left decompositions down to site $k + 1$, absorbing the remaining factors into the tensor on their left; the tensors at sites $k + 1, \dots, N$ are then right-canonical. All factors left by these two sweeps are finally gathered into the tensor at site k . This tensor is generally not constrained to be left- or right-canonical: it is called the orthogonality center. In the `ITensorMPS` library, the function `orthogonalize!` can be used to bring an MPS into such a mixed-canonical form with an orthogonality center at a given site k . An example is given in code example #3.

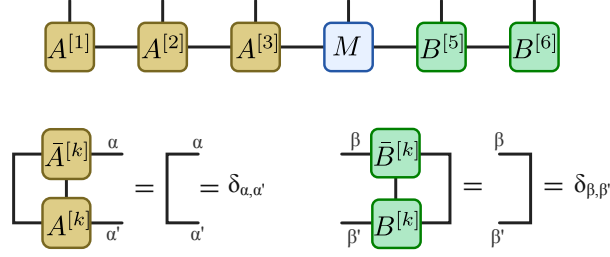


Figure 8: Top: mixed-canonical form of a six-site MPS with the orthogonality center at site $k = 4$. The tensors to its left are left-canonical, and those to its right are right-canonical. Bottom: left- and right-canonical conditions for the corresponding tensors. The isometric properties of $A^{[i]}$ (left-canonical) and $B^{[j]}$ (right-canonical) imply that $A^\dagger A = 1$ and $B B^\dagger = 1$.

Code example 3: Use of the `orthogonalize!` function of `ITensorMPS` to bring an MPS into mixed-canonical form with an orthogonality center at site k . Once this is done, the expectation value of the single-site operator σ_k^z can be computed from the center tensor alone (tensor M in Fig. 8). The first contraction illustrates that the same local calculation does not give the correct result before the orthogonality center has been moved to site k .

[MPS_orthogonality_center.jl](#) on GitHub.

Code example 3: MPS orthogonality center

```

1  using ITensors
2  using ITensorMPS
3
4  N = 6 # number of qubits
5  k = 3 # site where we want to compute <sigma^z_k>
6
7  sites = siteinds("Qubit", N)
8  psi0 = MPS(sites, fill("0", N)) # |000000>
9  psi1 = MPS(sites, fill("1", N)) # |111111>
10
11 x=0.25 # arbitrary mixing parameter
12 psi = x*psi0 + (1-x)*psi1 # MPS with bond dimension 2, but not in canonical form
13 normalize!(psi)
14
15 A = psi[k] # MPS tensor at site k
16 Z = op("Z", sites[k]) # tensor representing the Pauli sigma^z operator on site k
17 wrong_expect_Z_k = scalar(dag(prime(A, sites[k])) * Z * A)
18 println("wrong <sigma^z_{$k}> = ", real(wrong_expect_Z_k))
19
20 # Bring the MPS into mixed canonical with orthogonality center at site k
21 orthogonalize!(psi, k)
22 A = psi[k]
23 # Compute <sigma^z_k> using only A = psi[k] (thanks to the orthogonality center property)
24 expect_Z_k = scalar(dag(prime(A, sites[k])) * Z * A)
25 println("<sigma^z_{$k}> = ", real(expect_Z_k), "... should be ", (2*x-1)/(2*x*x-2*x+1))

```

2.4.4 Entanglement entropy and MPS compression from mixed-canonical form

The mixed canonical form is also useful for computing entanglement entropies associated with a bipartition of the system into two parts $(1, \dots, n)$ and $(n+1, \dots, N)$. In terms of the states $|\alpha_{n-1}\rangle_L$ and $|\alpha_n\rangle_R$ defined by the left and right canonical parts of the MPS, the full state can be written as

$$|\psi\rangle = \sum_{\alpha_{n-1}, s_n, \alpha_n} M_{\alpha_{n-1}, \alpha_n}^{[n] s_n} |\alpha_{n-1}\rangle_L |s_n\rangle \otimes |\alpha_n\rangle_R. \quad (21)$$

Since the states $|\alpha_{n-1}\rangle_L |s_n\rangle$ are orthonormal (in subsystem $(1, \dots, n)$), and since the states $|\alpha_n\rangle_R$ are also orthonormal (in subsystem $(n+1, \dots, N)$) the SVD of the center matrix $M^{[n]}$ (obtained by grouping the indices α_{n-1} and s_n as the row index, and β_n as the column index) gives the Schmidt decomposition (appendix A.1) of the state $|\psi\rangle$ across the bond $(n, n+1)$. The von Neumann entropy then follows directly from these singular values. It is interesting that, with a well-defined orthogonality center, the Schmidt decomposition across a given bond can be obtained from a single tensor.

An example of code (based on `ITensor/ITensorMPS`) which computes the von Neumann entanglement entropy across a given bond from the SVD of the center tensor is given in code example #6.

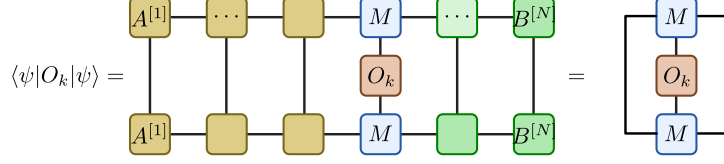


Figure 9: Expectation value of a single-site observable O at site k in an MPS written in mixed-canonical form with orthogonality center at site k . The contraction simplifies and only the tensor M at the orthogonality center remains.

When an MPS has a well-defined orthogonality center, the SVD of the center matrix can also be used to compress the MPS by truncating the small singular values. This is a key step in many algorithms. For this, as for the entanglement entropy calculation, one reshapes the center tensor $M^{[n]}$ into a matrix with combined row index (α_{n-1}, s_n) and column index α_n . One then performs an SVD of this matrix, and truncates the small singular values to obtain a new center matrix with smaller bond dimension on bond $(n, n+1)$.

2.4.5 Expectation value of a local observable

Consider a local observable O_k acting on site k . The expectation $\langle \psi | O_k | \psi \rangle$ can be computed by contracting the double-layer TN shown in the left of Fig. 9. If the MPS is written in mixed-canonical form with the orthogonality center at site k , one can use the rules illustrated in Fig. 8 to simplify the calculation. As a result, the contraction simplifies considerably, as shown in the right part of Fig. 9. The calculation finally reduces to a contraction of M , its conjugate and the observable O_k . The computational cost of this contraction is $\mathcal{O}(d^2 \chi^2)$, where d is the physical dimension of the local Hilbert space and χ is the bond dimension of the MPS. An elementary code example where such a calculation is performed is given in code example #3.

2.5 Elementary manipulations

2.5.1 Addition of two MPS

Consider two MPS $|\psi_A\rangle$ and $|\psi_B\rangle$ with the same local Hilbert space and the same number of sites. We can write them as

$$|\psi_A\rangle = \sum_{s_1, s_2, \dots, s_N} A_1^{s_1} A_2^{s_2} \cdots A_N^{s_N} |s_1 s_2 \dots s_N\rangle \quad (22)$$

and

$$|\psi_B\rangle = \sum_{s_1, s_2, \dots, s_N} B_1^{s_1} B_2^{s_2} \cdots B_N^{s_N} |s_1 s_2 \dots s_N\rangle. \quad (23)$$

The sum of the two MPS can be written as

$$|\psi_A\rangle + |\psi_B\rangle = \sum_{s_1, s_2, \dots, s_N} C_1^{s_1} C_2^{s_2} \cdots C_N^{s_N} |s_1 s_2 \dots s_N\rangle \quad (24)$$

where the new tensors C_k are defined as

$$C_k^{s_k} = \begin{pmatrix} A_k^{s_k} & 0 \\ 0 & B_k^{s_k} \end{pmatrix}. \quad (25)$$

The new MPS has a bond dimension that is the sum of the bond dimensions of the original MPS. This is a simple way to construct an MPS representation of a linear combination of two states, but it is in general not optimal. For instance, if one tries to write the W state as an MPS using this method, one would get an MPS with bond dimension $\chi = N$, whereas the W state admits an MPS representation with bond dimension $\chi = 2$.

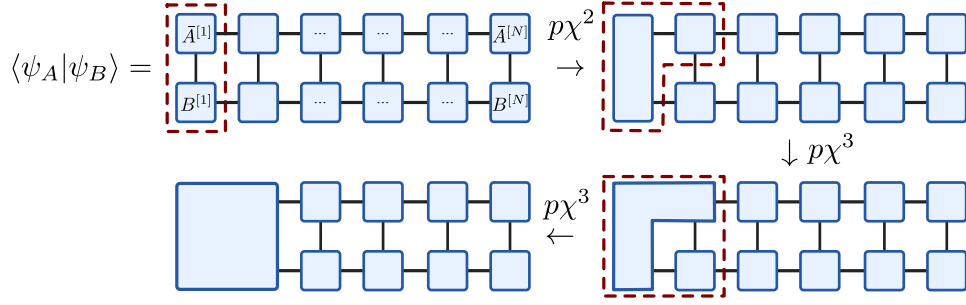


Figure 10: MPS representation of the scalar product $\langle \psi_A | \psi_B \rangle$, together with the left-to-right sequence of partial contractions.

$$\begin{aligned}
 \mathbb{E} &= \begin{array}{c} \chi \quad \bar{A} \quad \chi \\ | \\ \chi \quad A \quad \chi \end{array} = \chi^2 \begin{array}{c} \boxed{\mathbb{E}} \end{array} \chi^2 \quad \langle L| = \begin{array}{c} \bar{A}^{[1]} \\ | \\ A^{[1]} \end{array} \quad \begin{array}{c} \bar{A}^{[N]} \\ | \\ A^{[N]} \end{array} = |R\rangle \\
 \mathbb{E}_O &= \begin{array}{c} \chi \quad \bar{A} \quad \chi \\ | \\ \chi \quad O \quad \chi \\ | \\ \chi \quad A \quad \chi \end{array} = \chi^2 \begin{array}{c} \boxed{\mathbb{E}_O} \end{array} \chi^2
 \end{aligned}$$

Figure 11: Graphical definition of the transfer matrix for a translationally invariant MPS. The physical index of A^s and $\bar{A}^s$ is contracted, and the pairs of left and right virtual indices are grouped to form the matrix indices of $\mathbb{E}$. The second line shows the corresponding transfer matrix $\mathbb{E}_O$ with a single-site observable inserted on the physical leg. The bottom line represents the boundary vectors $\langle L|$ and $|R\rangle$ used in Eq. (28).

2.5.2 Scalar product

The scalar product $\langle \psi_A | \psi_B \rangle$ between two MPS $|\psi_A\rangle$ and $|\psi_B\rangle$ can be computed by contracting the double-layer TN displayed in Fig. 10. The contraction of this TN can be performed efficiently, from left to right (or from right to left), by contracting one tensor ($\bar{A}^{[i]}$ or $B^{[i]}$) at a time (Fig. 10). The computational cost of this contraction is $\mathcal{O}(Nd\chi^3)$, where N is the number of sites, d is the physical dimension of the local Hilbert space, and χ is the bond dimension of the MPS.

2.5.3 Correlation function

For a translationally invariant MPS $A^{[i]} = A$ for all i , the correlation function $\langle O_k O_{k+r} \rangle$ can be computed using the transfer matrix

$$\mathbb{E} = \sum_{s=1}^d A^s \otimes \bar{A}^s, \quad (26)$$

where A^s are the tensors of the MPS and $\bar{A}^s$ are their complex conjugates. This matrix, of dimension $\chi^2 \times \chi^2$, is represented graphically in Fig. 11. The physical indices of A^s and $\bar{A}^s$ are contracted together, and the pairs of left and right virtual indices are grouped to form the matrix indices of $\mathbb{E}$.⁶ It is also useful to define the transfer matrix associated with a single-site observable O :

$$\mathbb{E}_O = \sum_{s,s'=1}^d \langle s|O|s' \rangle A^s \otimes \bar{A}^{s'}, \quad (27)$$

as shown in the second line of Fig. 11.

⁶Note that $\mathbb{E}$ is unchanged if one performs a local basis change on the physical index of A^s (i.e., if one replaces A^s by $\sum_{s'} U_{ss'} A^{s'}$ with U a unitary matrix).

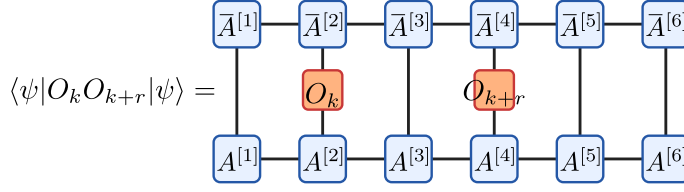


Figure 12: TN representation of the two-point correlation function of the type $\langle \psi | O_k O_{k+r} | \psi \rangle$. No canonical form is assumed here.

For a finite MPS, the two-point function can first be represented directly as a double-layer TN, with the operators inserted on the corresponding physical legs, as illustrated in Fig. 12.

The two-point correlation function can be expressed as

$$\langle O_k O'_{k+r} \rangle = \frac{\langle L | \mathbb{E}^{k-2} \mathbb{E}_O \mathbb{E}^{r-1} \mathbb{E}_{O'} \mathbb{E}^{N-k-r-1} | R \rangle}{\langle L | \mathbb{E}^{N-2} | R \rangle}. \quad (28)$$

Using the spectral decomposition

$$\mathbb{E} = \sum_{\alpha=1}^{\chi^2} \lambda_{\alpha} |r_{\alpha}\rangle \langle l_{\alpha}|, \quad \langle l_{\alpha} | r_{\beta} \rangle = \delta_{\alpha\beta},$$

and assuming that the eigenvalue λ_1 with largest modulus is non-degenerate, we have in the bulk limit (k and $k+r$ are very far from the boundaries)

$$\mathbb{E}^{k-2} \simeq \lambda_1^{k-2} |r_1\rangle \langle l_1|, \quad \mathbb{E}^{N-k-r-1} \simeq \lambda_1^{N-k-r-1} |r_1\rangle \langle l_1|.$$

As for the denominator, it becomes

$$\langle L | \mathbb{E}^{N-2} | R \rangle \simeq \lambda_1^{N-2} \langle L | r_1 \rangle \langle l_1 | R \rangle.$$

Therefore,

$$\langle O_k O'_{k+r} \rangle = \sum_{\alpha=1}^{\chi^2} \left(\frac{\lambda_{\alpha}}{\lambda_1} \right)^{r-1} \frac{\langle l_1 | \mathbb{E}_O | r_{\alpha} \rangle \langle l_{\alpha} | \mathbb{E}_{O'} | r_1 \rangle}{\lambda_1^2}.$$

Similarly,

$$\langle O_k \rangle = \frac{\langle l_1 | \mathbb{E}_O | r_1 \rangle}{\lambda_1}, \quad \langle O'_{k+r} \rangle = \frac{\langle l_1 | \mathbb{E}_{O'} | r_1 \rangle}{\lambda_1}.$$

The $\alpha = 1$ contribution to the two-point function is

$$\frac{\langle l_1 | \mathbb{E}_O | r_1 \rangle \langle l_1 | \mathbb{E}_{O'} | r_1 \rangle}{\lambda_1^2} = \langle O_k \rangle \langle O'_{k+r} \rangle.$$

Hence, the connected correlation function is

$$\langle O_k O'_{k+r} \rangle_c = \langle O_k O'_{k+r} \rangle - \langle O_k \rangle \langle O'_{k+r} \rangle = \sum_{\alpha=2}^{\chi^2} \left(\frac{\lambda_{\alpha}}{\lambda_1} \right)^{r-1} \frac{\langle l_1 | \mathbb{E}_O | r_{\alpha} \rangle \langle l_{\alpha} | \mathbb{E}_{O'} | r_1 \rangle}{\lambda_1^2}.$$

If the second eigenvalue has degeneracy n , $\lambda_2 = \lambda_3 = \dots = \lambda_{n+1}$ and at large distance r we find

$$\langle O_k O'_{k+r} \rangle_c \sim \left(\frac{\lambda_2}{\lambda_1} \right)^{r-1} \sum_{\alpha=2}^{n+1} \frac{\langle l_1 | \mathbb{E}_O | r_{\alpha} \rangle \langle l_{\alpha} | \mathbb{E}_{O'} | r_1 \rangle}{\lambda_1^2}$$

with corrections of order $\mathcal{O} \left[\left(\frac{\lambda_{n+2}}{\lambda_1} \right)^{r-1} \right]$. The connected correlation decays exponentially with the distance r , with a correlation length given by

$$\xi^{-1} = -\log \left| \frac{\lambda_2}{\lambda_1} \right|.$$

This shows that, generically, an MPS (with finite bond dimension) has exponentially decaying correlations. As will be discussed in Sec. 3, this is not the case for higher-dimensional TNs such as PEPS, which can have algebraically decaying correlations even with finite bond dimension.

Code example 4: Code using the `ITensor` library which constructs the local tensor A of the AKLT MPS (with bond dimension $\chi = 2$, see Sec. 2.3.4 and appendix C), contracts it with its complex conjugate to form the transfer matrix $\mathbb{E}$, reshapes $\mathbb{E}$ as a $\chi^2 \times \chi^2$ matrix, and computes the two largest eigenvalues. The modulus of their ratio gives the correlation length of the MPS (Sec. 2.5.3).

[AKLT_transfer_matrix.jl](#) on GitHub.

Code example 4: MPS transfer matrix

```

1 using LinearAlgebra
2 using ITensors
3
4 chi = 2 # bond dimension of the AKLT MPS
5
6 l = Index(chi, "left") # left virtual index
7 r = Index(chi, "right") # right virtual index
8 s = Index(3, "spin-1") # physical index
9
10 # Physical basis: s = 1, 2, 3 corresponds to |+, |0>, |->.
11 A = ITensor(l, s, r) # three-leg MPS tensor with indices (l, s, r)
12
13 # Tensor entries for the AKLT state
14 A[l => 1, s => 1, r => 2] = 1/sqrt(2)
15 A[l => 1, s => 2, r => 1] = -1/2
16 A[l => 2, s => 2, r => 2] = 1/2
17 A[l => 2, s => 3, r => 1] = -1/sqrt(2)
18
19 # Contract only the physical index, leaving the two doubled virtual indices open.
20 E = A * dag(prime(A, 1, r))
21
22 row = combiner(l, prime(l); tags = "row") # combines the 2 left indices (l and l') into one row index
23 col = combiner(r, prime(r); tags = "col") # combines the 2 right indices (r and r') into one column index
24 E_reshaped = row * E * col # reshape E into a two-leg tensor
25 E_matrix = Array(E_reshaped, combinedind(row), combinedind(col)) # Convert reshaped E into a matrix
26
27 lambda = sort(eigvals(E_matrix); by = abs, rev = true)
28 println("largest two eigenvalues of E = ", lambda[1:2], " ratio |\lambda2/\lambda1| = ", abs(lambda[2] / lambda[1]))
29 println("correlation length \xi = -1/\log(|\lambda2/\lambda1|) = ", -1 / log(abs(lambda[2] / lambda[1])))

```

2.6 Matrix product operators (MPO) and local Hamiltonians

In the same way a pure state can be represented as an MPS, an operator can be represented as a matrix product operator (MPO) [6, 22, 63]. An MPO is a TN of the same structure as an MPS, but with two physical indices per site, one for the input state and one for the output state. This is illustrated in Fig. 13. The tensor on the left is the generic representation for a matrix of size $d^N \times d^N$, while the tensor on the right is the factorized MPO representation. In the same way any vector of dimension d^N (state $|\psi\rangle$) can be exactly represented by an MPS provided the bond dimension is large enough, any operator acting in the Hilbert space can be represented as an MPO with a large enough bond dimension. The MPO representation can be used for many types of operators: Hamiltonians, observables, unitary evolution operators, quantum gates, density matrices, quantum channels, Lindbladians, etc. In this section we focus on the representation of Hamiltonians as MPOs, which is particularly useful for performing DMRG calculations (Sec. 2.7) or to simulate time evolution.

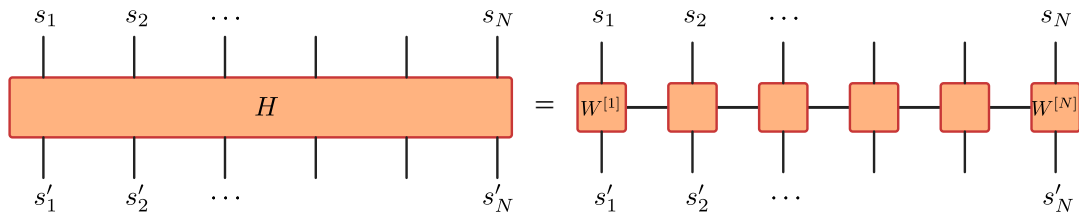


Figure 13: A Hamiltonian (or some generic operator) viewed either as a single tensor or factorized as a matrix product operator (MPO).

For a state $|\psi\rangle$, one generally needs to perform some truncation to get an MPS representation with a reasonable bond dimension (say polynomial in N). In contrast, for a Hamiltonian H containing only local interactions, one can find an *exact* MPO representation with a bond dimension that is independent of the system size N .

2.6.1 Example 1: transverse-field Ising chain

As a simple example, consider the transverse-field Ising chain with open boundary conditions,

$$H = J \sum_{i=1}^{N-1} \sigma_i^z \sigma_{i+1}^z + h \sum_i \sigma_i^x.$$

An MPO for this Hamiltonian can be constructed by letting the virtual index keep track of whether a nearest-neighbor term has already started. In the bulk, one possible local tensor is the following operator-valued matrix,

$$W^{[2 \leq i \leq N-1]} = \begin{pmatrix} \mathbb{1} & J\sigma^z & h\sigma^x \\ 0 & 0 & \sigma^z \\ 0 & 0 & \mathbb{1} \end{pmatrix} = \mathbb{W}.$$

In the expression above, each entry is an operator acting on the local Hilbert space at site i ($\sigma^z = \sigma_i^z$ etc.) For an open chain, the first and last tensors can be chosen as

$$L^t = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}, \quad W^{[1]} = L^t \mathbb{W} = \begin{pmatrix} \mathbb{1}_1 & J\sigma_1^z & h\sigma_1^x \end{pmatrix} \quad (29)$$

$$R = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad W^{[N]} = \mathbb{W} R = \begin{pmatrix} h\sigma_N^x \\ \sigma_N^z \\ \mathbb{1}_N \end{pmatrix}. \quad (30)$$

To understand how the product of the tensors reconstructs H , one should think of the virtual index $\alpha \in \{1, 2, 3\}$ as a virtual state that keeps track of the terms in the Hamiltonian. This is the so-called finite-state automaton picture [63]. Expanding the product $L^t \mathbb{W} \cdots \mathbb{W} R$ amounts to summing over all possible paths $(\alpha_0, \alpha_1, \dots, \alpha_N)$ in the virtual space, where each transition $\alpha_{i-1} \rightarrow \alpha_i$ adds the factor $\mathbb{W}_{\alpha_{i-1}, \alpha_i}^{[i]}$ to the operator product.

The choice of the left vector L in Sec. 2.6.1 means that the path starts from the first virtual state ($\alpha_0 = 1$) at site 1 and Sec. 2.6.1 means that the path ends in the third virtual state ($\alpha_N = 3$) at site N . The matrix $\mathbb{W}$ is designed so that only the two following types of paths contribute to the sum:

- path $\alpha = (1, \dots, 1_{i-1}, 3_i, 3, \dots, 3) \rightarrow$ operator $\mathbb{1}_1 \otimes \cdots \otimes \mathbb{1} \otimes h\sigma_i^x \otimes \mathbb{1} \otimes \cdots \otimes \mathbb{1}_N$
- path $\alpha = (1, \dots, 1_{i-1}, 2_i, 3_{i+1}, 3, \dots, 3) \rightarrow$ operator $\mathbb{1}_1 \otimes \cdots \otimes \mathbb{1} \otimes J\sigma_i^z \otimes \sigma_{i+1}^z \otimes \mathbb{1} \otimes \cdots \otimes \mathbb{1}_N$.

2.6.2 Example 2: Heisenberg model

A straightforward generalization of the MPO construction above can be used to find an exact MPO representation of the Heisenberg model,

$$H = J \sum_{i=1}^{N-1} (\sigma_i^x \sigma_{i+1}^x + \sigma_i^y \sigma_{i+1}^y + \sigma_i^z \sigma_{i+1}^z).$$

The local tensor in the bulk can be chosen as

$$W^{[2 \leq i \leq N-1]} = \begin{pmatrix} \mathbb{1} & J\sigma^x & J\sigma^y & J\sigma^z & 0 \\ 0 & 0 & 0 & 0 & \sigma^x \\ 0 & 0 & 0 & 0 & \sigma^y \\ 0 & 0 & 0 & 0 & \sigma^z \\ 0 & 0 & 0 & 0 & \mathbb{1} \end{pmatrix} = \mathbb{W}.$$

The first and last tensors can be chosen as

$$L^t = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad W^{[1]} = L^t \mathbb{W} = \begin{pmatrix} \mathbb{1} & J\sigma^x & J\sigma^y & J\sigma^z & 0 \end{pmatrix} \quad (31)$$

$$R = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \quad W^{[N]} = \mathbb{W} R = \begin{pmatrix} 0 \\ \sigma^x \\ \sigma^y \\ \sigma^z \\ \mathbb{1} \end{pmatrix}. \quad (32)$$

One can check that the above tensors reconstruct the Heisenberg Hamiltonian by realizing that the possible α_i -paths are of the following types:

- path $\alpha = (1, \dots, 1_{i-1}, 2_i, 5_{i+1}, 5, \dots, 5) \longrightarrow$ operator $\mathbb{1}_1 \otimes \dots \otimes \mathbb{1} \otimes J\sigma_i^x \otimes \sigma_{i+1}^x \otimes \mathbb{1} \otimes \dots \otimes \mathbb{1}_N$
- path $\alpha = (1, \dots, 1_{i-1}, 3_i, 5_{i+1}, 5, \dots, 5) \longrightarrow$ operator $\mathbb{1}_1 \otimes \dots \otimes \mathbb{1} \otimes J\sigma_i^y \otimes \sigma_{i+1}^y \otimes \mathbb{1} \otimes \dots \otimes \mathbb{1}_N$
- path $\alpha = (1, \dots, 1_{i-1}, 4_i, 5_{i+1}, 5, \dots, 5) \longrightarrow$ operator $\mathbb{1}_1 \otimes \dots \otimes \mathbb{1} \otimes J\sigma_i^z \otimes \sigma_{i+1}^z \otimes \mathbb{1} \otimes \dots \otimes \mathbb{1}_N$.

2.6.3 Example 3: 2-qubit gate

Consider a two-qubit gate U acting on sites i and j . A two-qubit gate U acting on sites i and j is a $d^2 \times d^2$ matrix, and it can be factorized using a QR or SVD decomposition. If U acts on neighboring sites, such factorization means that U can be represented as an MPO with bond dimension (at most) d^2 . This is represented in the upper part of Fig. 14. If U acts on non-neighboring sites, one can still find an MPO representation by inserting diagonal tensors on the sites between i and j to propagate the virtual index from site i to site j . These tensors act as the identity in the physical space and in the virtual space. This is represented in the lower part of Fig. 14.

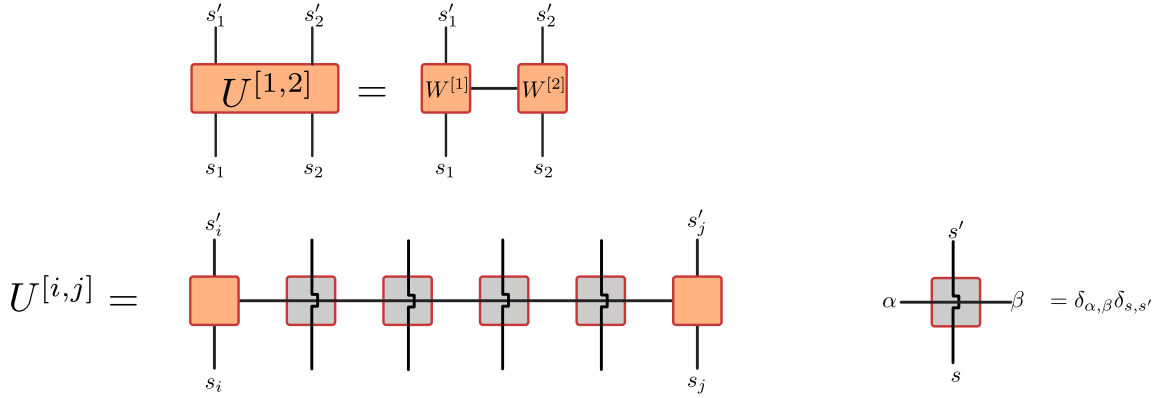


Figure 14: MPO representations of a two-qubit gate U acting on neighboring sites (top) or on non-neighboring sites (bottom). The diagonal tensors (gray) simply propagate the virtual index from site i to site j while acting as the identity in the physical space.

2.6.4 Finite-state automaton picture

We describe here a general method to construct an MPO representation of a Hamiltonian, which is based on the finite-state automaton picture [63]. We follow here the presentation given in Ref. [30].

Consider a bond $(i, i+1)$. The terms in the Hamiltonian can be classified into three types:

- terms that act on sites $1, 2, \dots, i$ only (type L)
- terms that act on sites $i+1, i+2, \dots, N$ only (type R)
- the terms of the form $h_{L,\alpha} \otimes h_{R,\alpha}$ that act on both sides of the bond (type C)

This corresponds to the following expression for the Hamiltonian:

$$H = H_L(i) \otimes \mathbb{1}_{[i+1,N]} + \mathbb{1}_{[1,i]} \otimes H_R(i) + \sum_{\alpha=1}^k h_{L,\alpha}(i) \otimes h_{R,\alpha}(i). \quad (33)$$

$H_L(i)$ is the sum of all terms of type L, $H_R(i)$ is the sum of all terms of type R, and $h_{L,\alpha}(i) \otimes h_{R,\alpha}(i)$ are the k terms of type C. For instance, a purely onsite Hamiltonian corresponds to $k = 0$. For a nearest-neighbor Ising Hamiltonian, $k = 1$ (see Sec. 2.6.1). An Ising model in transverse field with both nearest-neighbor and next-nearest neighbor interactions, $k = 2$. For a Heisenberg Hamiltonian, $k = 3$ (see Sec. 2.6.2). In principle k can be different for each site (at least close to the chain boundaries); here, to simplify the notation, we will ignore this dependence and simply write k instead of $k(i)$.

The next step is to realize that the decomposition across the bond $(i-1, i)$ is related to that across the next bond $(i, i+1)$ through a recursion relation. This recursion relation takes the form

$$\begin{pmatrix} H_R(i-1) \\ h_R(i-1) \\ \mathbb{1}_{[i,N]} \end{pmatrix} = \begin{pmatrix} \mathbb{1} & C^{[i]} & D^{[i]} \\ 0 & A^{[i]} & B^{[i]} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} H_R(i) \\ h_R(i) \\ \mathbb{1}_{[i+1,N]} \end{pmatrix}. \quad (34)$$

The matrix $A^{[i]}$ is a $k \times k$ matrix, $B^{[i]}$ is a $k \times 1$ matrix, $C^{[i]}$ has dimension $1 \times k$, $D^{[i]}$ is 1×1 . In total, the block matrix therefore has dimension $\chi = k + 2$. $D^{[i]}$ encodes the terms in the Hamiltonian that act on site i only, $C^{[i]}$ encodes the terms that start at site i , $B^{[i]}$ encodes the terms that end at site i , and $A^{[i]}$ encodes the terms that are propagating through site i . It turns out that this matrix is precisely the local tensor of the MPO representation of H at site i :

$$W^{[i]} = \begin{pmatrix} \mathbb{1} & C^{[i]} & D^{[i]} \\ 0 & A^{[i]} & B^{[i]} \\ 0 & 0 & \mathbb{1} \end{pmatrix}. \quad (35)$$

As in the examples of Sec. 2.6, the first and last values of the virtual index α have a special meaning in the finite-state automaton picture: $\alpha = 1$ means that no term of the Hamiltonian has started yet, while $\alpha = \chi$ means that all terms have already ended. The k intermediate values of α keep track of the terms that are currently being added to the operator product.

A short example is given in code example #5. The Hamiltonian is specified as an `OpSum`, and the call `MPO(os, sites)` constructs the corresponding MPO automatically. In the context of mixed states, the same finite-state automaton picture can be used to construct MPO representations of Lindbladians, as discussed in Sec. 4.5.

Code example 5: Transverse-field Ising Hamiltonian on an open chain using `ITensors`. The Ising couplings $J_1, J_2, \dots, J_k$ act at distances $1, 2, \dots, k$, and a local transverse field is included. The Hamiltonian is first defined as an `OpSum`, then converted to an MPO. The code prints the MPO bond dimensions on all bonds. In the example shown, the couplings are chosen as a truncated power-law profile J_r up to range k . The printed bond dimensions illustrate how the finite interaction range is encoded in the MPO auxiliary space.

[MPO_transverse_field_ising_finite_range.jl](#) on GitHub.

Code example 5: Finite-range transverse-field Ising Hamiltonian as an MPO

```

1  using ITensors, ITensorMPS
2
3  N = 12
4  k = 4 # Ising couplings up to distance k
5  # 1/r^6 van der Waals interactions up to distance k
6  J = [1.0 / r^6 for r in 1:k]
7
8  h = 0.7 # transverse field
9
10 sites = siteinds("S=1/2", N)
11
12 os = OpSum()
13 for r in 1:k
14     for j in 1:(N - r)
15         global os += J[r], "Sz", j, "Sz", j + r
16     end
17 end
18 for j in 1:N
19     global os += h, "Sx", j
20 end
21
22 H = MPO(os, sites)
23 bond_dims = [dim(linkind(H, j)) for j in 1:(N - 1)]
24 println("MPO bond dimensions = ", bond_dims)
```

2.6.5 Multiplication of an MPO by an MPS and zip-up algorithm

Given an operator H (not necessarily a Hamiltonian) represented as an MPO and a pure state $|\psi\rangle$ represented as an MPS, one can compute the state $|\psi'\rangle = H|\psi\rangle$ by contracting the corresponding TN, as shown in Fig. 15. If such multiplication is performed exactly, the resulting MPS for $|\psi'\rangle$ has a bond dimension that is the product of the bond dimensions of the MPO and the MPS. Since one often

has many such MPS-MPO multiplications to perform, and the bond dimension of the resulting MPS grows exponentially with the number of multiplications, one usually wants to find an approximate MPS representation of $|\psi'\rangle$ with a smaller bond dimension $\chi' < k\chi$. One possible way to do this is to perform the multiplication and the truncation at the same time, thus computing directly the new truncated tensors of $|\psi'\rangle$. An algorithm to do this is the zip-up algorithm [64] and is illustrated in Fig. 16.

It is also possible to perform some variational optimization to find the best MPS approximation of $|\psi'\rangle$ to $H|\psi\rangle$ with a given bond dimension χ' . We are thus looking for the MPS $|\psi'\rangle$ with given bond dimension that minimizes the distance $\|H|\psi\rangle - |\psi'\rangle\|^2 = \langle\psi|H^\dagger H|\psi\rangle - \langle\psi'|H|\psi\rangle - \langle\psi|H^\dagger|\psi'\rangle + \langle\psi'|\psi'\rangle$. This can be done by optimizing one tensor at a time while keeping the others fixed, and sweeping through the lattice until convergence. Both methods can also be combined: one can first perform a zip-up to get an initial guess for the new MPS tensors, and then perform a variational optimization to further improve the approximation.

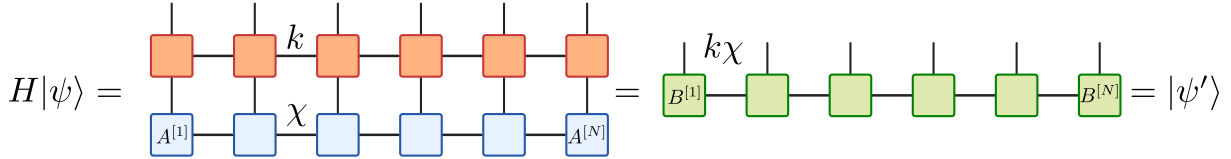


Figure 15: Exact multiplication of an MPO (bond dimension k) by an MPS (bond dimension χ). The resulting MPS for $|\psi'\rangle = H|\psi\rangle$ has bond dimension $\chi' = k\chi$.

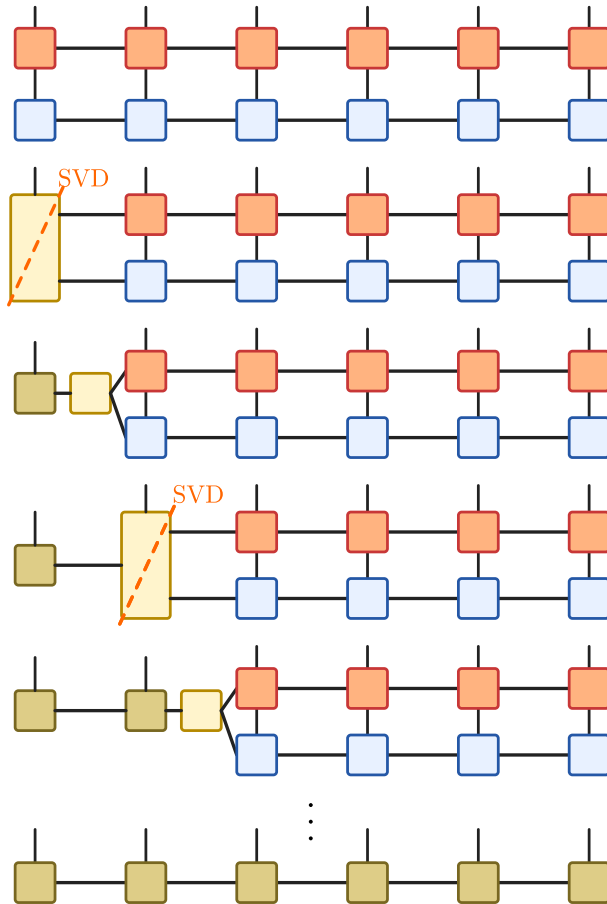


Figure 16: Schematic representation of the zip-up algorithm. The MPO and MPS tensors are contracted and factorized (SVD plus truncation) successively from left to right.

2.7 DMRG algorithm

The Density Matrix Renormalization Group (DMRG) algorithm was introduced by S. White in 1992 [20, 21] and is one of the most powerful numerical methods for computing the ground state of 1d (and more generally low-dimensional) quantum systems. Although it was not formulated in this way in the original paper, the DMRG algorithm is essentially a variational method in the space of MPS, where the tensors A_k are optimized to minimize the energy. The optimization is performed iteratively by sweeping through the lattice and updating one or two tensors at a time while keeping the others fixed. The DMRG algorithm can also be used to find low-energy excited states by targeting states that are orthogonal to the ground state. It can also be used to perform certain contractions, such as the multiplication of an MPO. Some extensions of the DMRG algorithm can also be used on loop-free TNs such as TTN [49, 61].

We give here a brief sketch of the two-site DMRG algorithm in the case where the Hamiltonian takes the form of an MPO. The two-site algorithm is slightly more complicated than the single-site DMRG algorithm, but it is often used in practice as it allows one to increase the bond dimension of the MPS during the optimization.

The starting point is the energy expectation value $E = \langle \psi | H | \psi \rangle / \langle \psi | \psi \rangle$, represented as a TN in Fig. 17. In the two-site version, one focuses on two neighboring MPS tensors, say $A^{[i]}$ and $A^{[i+1]}$, while all the other tensors are kept fixed. The two tensors are contracted together and reshaped into a single vector v collecting all variational parameters of this local update, as shown in Fig. 18. This vector v has dimension $d^2\chi^2$, where d is the physical dimension of the local Hilbert space and χ is the bond dimension of the MPS.

Once the rest of the TN has been contracted, both the numerator and the denominator of the Rayleigh quotient become quadratic forms in v :

$$\langle \psi | H | \psi \rangle = v^\dagger \mathcal{H} v, \quad \langle \psi | \psi \rangle = v^\dagger \mathcal{N} v, \quad (36)$$

where $\mathcal{H}$ is the effective Hamiltonian matrix and $\mathcal{N}$ is the effective norm matrix for the two optimized sites.

The minimization can therefore be written as a constrained variational problem. Introducing a Lagrange multiplier λ to impose $\langle \psi | \psi \rangle = 1$, one has to extremize

$$v^\dagger \mathcal{H} v - \lambda (v^\dagger \mathcal{N} v - 1). \quad (37)$$

Taking the derivative with respect to $v^\dagger$ gives the generalized eigenvalue problem

$$\mathcal{H} v = \lambda \mathcal{N} v. \quad (38)$$

The optimal local tensor is obtained from the eigenvector associated with the lowest eigenvalue.

If the MPS is brought into the appropriate mixed-canonical form, with the orthogonality center on the optimized bond, the effective norm matrix is simply the identity, $\mathcal{N} = 1$. The generalized eigenvalue problem then reduces to the standard one,

$$\mathcal{H} v = \lambda v. \quad (39)$$

The dimension of the matrix $\mathcal{H}$ is $(d^2\chi^2) \times (d^2\chi^2)$, which can be quite large. For large bond dimensions $\mathcal{H}$ becomes too large to be stored explicitly and fully diagonalized.⁷ For this reason, and also because one is only interested in the lowest eigenvalue, the eigenvalue problem is solved with an iterative eigensolver such as the Lanczos algorithm. Such an algorithm does not require the explicit construction and storage of the full matrices but only requires the action of $\mathcal{H}$ and $\mathcal{N}$ on trial vectors, i.e., the ability to compute $v' = \mathcal{H} v$ and $v'' = \mathcal{N} v$ for given v . This can be done by contracting the corresponding effective TN.

Once the optimal v is found, it is reshaped back into the two-site tensor. Then, an SVD is performed (with truncation of the smallest singular values if needed) to split the two-site tensor into two tensors $A^{[i]}$ and $A^{[i+1]}$ with the desired new bond dimension. After the update of the two tensors, one moves to the next pair of sites and repeats the process until convergence.

Two minimal DMRG code examples are given in code example #6 and code example #7 for spin chain models, the spin- $\frac{1}{2}$ Heisenberg model and the spin-1 AKLT model [60]. In the second example the exact ground state is known and can be represented as an MPS with bond dimension $\chi = 2$ (See Sec. 2.3). Another example is given in code example #8 for a noninteracting spinless fermion tight-binding model. There, the exact ground state energy can be computed analytically and compared to the DMRG result.

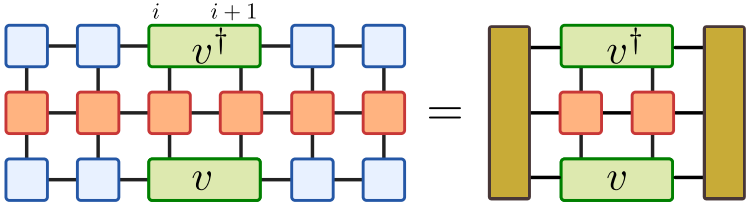
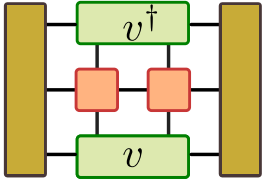
⁷Simulations often involve bond dimension χ of the order of a few hundreds, or even a few thousands. State-of-the-art DMRG calculations can reach χ of the order of 10^4 or more.

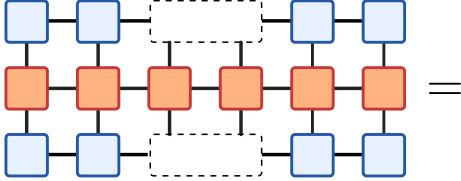
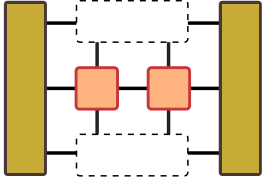
$$E = \frac{\langle \psi | H | \psi \rangle}{\langle \psi | \psi \rangle} = \frac{\text{Diagram 1}}{\text{Diagram 2}}$$

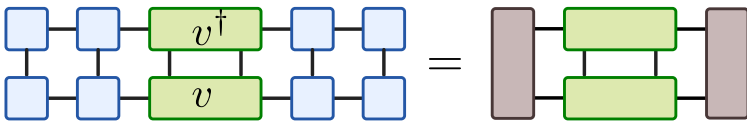
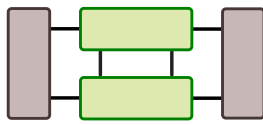
Diagram 1: A 3x6 grid of tensors. The top and bottom rows consist of blue tensors labeled $\bar{A}^{[1]}, \dots, \bar{A}^{[N]}$ and $A^{[1]}, \dots, A^{[N]}$ respectively. The middle row consists of orange tensors representing the Hamiltonian H . Vertical lines connect the blue tensors to the orange tensors.

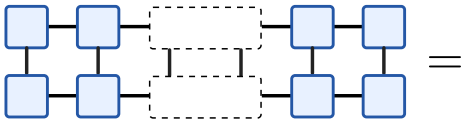
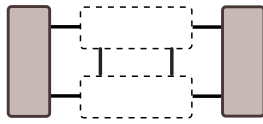
Diagram 2: A 2x6 grid of tensors. The top row consists of blue tensors labeled $\bar{A}^{[1]}, \dots, \bar{A}^{[N]}$. The bottom row consists of blue tensors labeled $A^{[1]}, \dots, A^{[N]}$. Vertical lines connect the top and bottom rows.

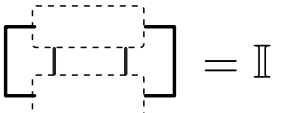
Figure 17: Energy expectation value $E = \langle \psi | H | \psi \rangle / \langle \psi | \psi \rangle$ as a TN. The state $|\psi\rangle$ is in an MPS form (tensors $A^{[i]}$) and the Hamiltonian is expressed as an MPO (orange tensors). E is iteratively minimized in the DMRG algorithm by optimizing one or two tensors of the MPS at a time while keeping the others fixed (see Fig. 18).

a) $v^\dagger \mathcal{H} v =$  $=$ 

b) $\mathcal{H} =$  $=$ 

c) $v^\dagger \mathcal{N} v =$  $=$ 

d) $\mathcal{N} =$  $=$ 

e) canonical form $\rightarrow \mathcal{N} =$  $= \mathbb{I}$

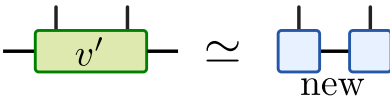
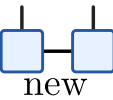
f)  $\simeq$ 

Figure 18: a) Expectation value $\langle \psi | H | \psi \rangle$ as a quadratic function of the tensors $A^{[i]}$ and $A^{[i+1]}$. The vector v contains the coefficients of the product $A^{[i]} A^{[i+1]}$. The vector v has dimension $d^2 \chi^2$, where d is the physical dimension of the local Hilbert space and χ is the bond dimension of the MPS. b) Matrix $\mathcal{H}$ of the effective Hamiltonian for the optimization of the tensors at sites i and $i+1$. $\mathcal{H}$ has dimension $d^2 \chi^2 \times d^2 \chi^2$. c) Norm $\langle \psi | \psi \rangle$ as a quadratic function of $v = A^{[i]} A^{[i+1]}$. d) Matrix $\mathcal{N}$ of the effective norm for the optimization of the tensors at sites i and $i+1$. e) If the MPS is in a mixed-canonical form with the orthogonality center at site i or $i+1$, the matrix $\mathcal{N}$ reduces to the identity matrix, and the optimization reduces to a standard eigenvalue problem $\mathcal{H}v = Ev$. In this case, the optimal v' is given by the eigenvector associated with the lowest eigenvalue E of $\mathcal{H}$. This lowest eigenvector is usually computed numerically using an iterative eigensolver such as the Lanczos algorithm. f) Once the optimal v' is found, it is reshaped back into the two-site tensor and then factorized into two tensors $A^{[i]}$ and $A^{[i+1]}$ using an SVD (and truncating the smallest singular values if needed to get the desired bond dimension).

The energy variance of the DMRG state is also computed, which gives an estimate of how close the DMRG state is to an eigenstate of the Hamiltonian (by construction this energy variance vanishes for an exact Hamiltonian eigenstate).

Code example 6: Short DMRG code to find the ground state of a spin- $\frac{1}{2}$ Heisenberg chain. The Hamiltonian is defined as a sum of two-site terms, and the initial state is a random MPS with a small bond dimension. The DMRG algorithm is then used to optimize the MPS representation of the ground state, with increasing bond dimensions and decreasing cutoff for the singular values. `H = MP0(os,sites)` converts the user-friendly `OpSum` representation of the Hamiltonian into an MPO, which is the format required by the DMRG algorithm. `energy,psi = dmrg(H,psi0;nsweeps,maxdim,cutoff)` performs the DMRG sweeps. The maximum bond dimension for each sweep is specified by `maxdim = [10,20,100,100,200]`. From sweep 5 onward, the maximum bond dimension is $\chi = 200$. The last part of the code computes the entanglement entropy for a bipartition in the center of the chain (between sites $i = N/2$ and site $i + 1$), which grows logarithmically with the system size for critical systems [65]. One first moves the orthogonality center to the site $i = N/2$ (`orthogonalize!(psi, i)`) and then (`U, S, V = svd(psi[i], (linkind(psi, i - 1), siteind(psi, i)))`) reshapes the MPS tensor at site i into a matrix by grouping the left index (`linkind(psi, i - 1)`) and physical index (`siteind(psi, i)`) together and keeping the right index as the second (column) index of the matrix. The SVD of M gives the singular values (S), which are needed to compute the von Neumann entanglement entropy.

[dmrg_spin_half_heisenberg.jl](#) on GitHub.

Code example 6: DMRG for the spin- $\frac{1}{2}$ Heisenberg chain – energy and entanglement entropy

```

1  using ITensors, ITensorMPS
2  let
3  N = 50
4  sites = siteinds("S=1/2",N)
5  os = OpSum()
6  for j=1:N-1
7  os += "Sz",j,"Sz",j+1
8  os += 1/2,"S+",j,"S-",j+1
9  os += 1/2,"S-",j,"S+",j+1
10 end
11 H = MP0(os,sites)
12 psi0 = random_mps(sites;linkdims=10)
13 nsweeps = 5
14 maxdim = [10,20,100,100,200]
15 cutoff = [1E-10]
16 energy,psi = dmrg(H,psi0;nsweeps,maxdim,cutoff)
17
18 # Compare with the exact ground state energy per bond in the thermodynamic limit (Bethe ansatz) e0 = -ln(2) + 1/4
19 e0 = - log(2) + 1/4
20 print("DMRG energy per bond: ", energy / (N-1), "\n")
21 print("Difference with 1/4-ln(2):", energy / (N-1) - e0, "\n")
22
23 # Compute the von Neumann entanglement entropy across the central bond.
24 i = N ÷ 2
25 orthogonalize!(psi, i)
26 # the virtual bond on the left of site i and the physical index (on site i) are grouped to define the matrix to be
27 # decomposed by SVD:
28 U, S, V = svd(psi[i], (linkind(psi, i - 1), siteind(psi, i)))
29 SvN = 0.0
30 for n in 1:dim(commonind(U, S))
31     p = S[n, n]^2
32     SvN -= p > 1e-14 ? p * log(p) : 0.0
33 end
34 print("Entanglement entropy of subsystem [1..",i,"] : ", SvN, "\n")
35 return
36 end

```

Code example 7: DMRG code for the spin-1 AKLT Hamiltonian [60], whose exact ground state is an MPS with bond dimension $\chi = 2$ (see Sec. 2.3.4 and appendix C). The Hamiltonian is first defined as a `OpSum` (lines with `os +=`) and then converted into an MPO (`H = MP0(os, sites)`). The DMRG algorithm is then used to minimize the energy with a given number of sweeps (`nsweeps`) and maximum bond dimension (`maxdim`). For the present model the DMRG algorithm gives the exact ground state energy, which is $-2J/3$ per bond.

[dmrg_AKLT.jl](#) on GitHub.

Code example 7: DMRG for the spin-1 AKLT chain

```

1  using ITensors, ITensorMPS
2  let
3  N = 10
4  J = 1.0
5  J2 = J / 3 # AKLT point
6  sites = siteinds("S=1", N)
7  os = OpSum()
8  spin_ops = ("Sx", "Sy", "Sz")
9
10 for j in 1:(N - 1)
11     for Sa in spin_ops
12         os += J, Sa, j, Sa, j + 1 #Heisenberg term
13         for Sb in spin_ops
14             os += J2, Sa, j, Sa, j + 1, Sb, j, Sb, j + 1 # biquadratic term
15         end
16     end
17 end
18
19 H = MPO(os, sites)
20 psi0 = random_mps(sites; linkdims = 3)
21 nsweeps = 6
22
23 # Below we allow for a maximal bond dimension = 10, but the actual bond
24 # dimension at the end of the DMRG sweeps is only 2 for the zero-field AKLT
25 # ground state, since it is exactly an MPS with bond dimension 2.
26 maxdim = [10]
27 cutoff = [1E-10]
28 energy, psi = dmrg(H, psi0; nsweeps, maxdim, cutoff)
29 energy_exact = -2 / 3 * (N - 1) * J
30 println("DMRG energy: ", energy, " Exact energy: ", energy_exact)
31 return
32 end

```

Code example 8: DMRG for 1d free fermions

([source online](#)) DMRG for a noninteracting spinless fermion 1d tight-binding model (open boundary conditions). The code compares the DMRG estimate of the ground state energy with the exact analytical result, which can be computed by filling the lowest single-particle energy levels. The code also computes the energy variance of the DMRG state, which gives an estimate of how close the DMRG state is to an eigenstate of the Hamiltonian. The code uses fermionic sites with conserved quantum numbers, starts from a half-filled product state, and performs DMRG sweeps with increasing values of maxdim.

[dmrg_FF.jl](#) on GitHub.

2.8 Time evolution with MPS

The simulation of the time evolution of quantum many-body systems is a central domain of application of MPS and TN methods in general. The first MPS-based time-evolution algorithm is usually traced back to Vidal's TEBD approach, introduced in 2003 [26] and formulated explicitly for 1d quantum many-body systems in 2004 [27]. Closely related adaptive tDMRG formulations appeared shortly afterwards [23, 24]. For a review on time-evolution algorithms for MPS, see Paeckel *et al.* [53].

In the following we give a brief presentation of the simplest time-evolution algorithm, the TEBD algorithm.

2.8.1 Time-evolving block decimation (TEBD) algorithm

Let us assume for simplicity that the (1d) Hamiltonian of the system is a sum of nearest-neighbor terms:

$$H = \sum_{i=1}^{N-1} h_i, \quad (40)$$

where h_i acts on sites i and $i + 1$ only. The time evolution operator $U(t) = e^{-iHt}$ can be approximated using a Trotter-Suzuki decomposition. For instance, at lowest order, one has

$$U(\tau) \approx e^{-ih_1\tau} e^{-ih_2\tau} \dots e^{-ih_{N-1}\tau} + \mathcal{O}(\tau^2). \quad (41)$$

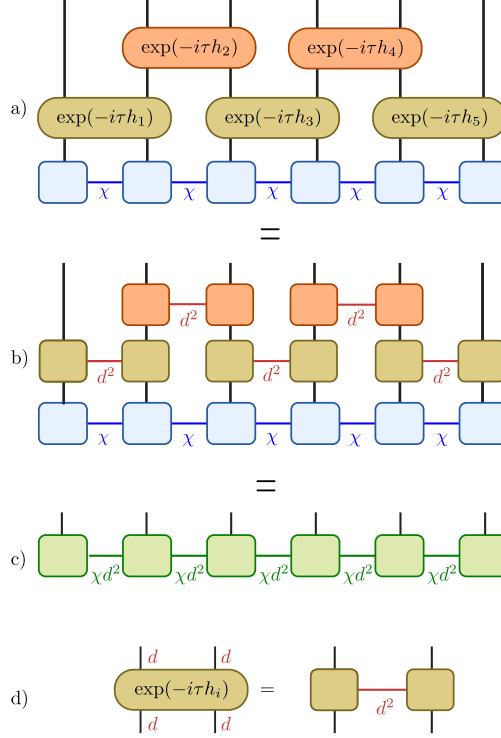


Figure 19: Time-evolution of an MPS using the TEBD algorithm. a) The time-evolution operator is decomposed into a product of two-site gates $e^{-ih_i\tau}$, which can be applied in parallel on the even and odd bonds. b) Using the fact that each $e^{-ih_i\tau}$ is a two-site operator, the SVD of the associated $d^2 \times d^2$ matrix can be used to write it as a product of two tensors with bond dimension at most d^2 [as shown in d)]. c) After one step the MPS bond dimension has increased to (at most) $\chi' = \chi d^2$. After a time step one generally needs to truncate the bond dimension of the MPS back to χ by keeping only the χ largest singular values.

At this level of approximation the order of the factors does not matter, and we can group the even and the odd terms:

$$U(\tau) \approx \left(\prod_{i=1}^{N/2} e^{-ih_{2i-1}\tau} \right) \left(\prod_{i=1}^{N/2-1} e^{-ih_{2i}\tau} \right) + \mathcal{O}(\tau^2). \quad (42)$$

The h_{2i} (even bonds) commute with each other, and the h_{2i+1} (odd bonds) commute with each other (but $[h_{2i}, h_{2i+1}] \neq 0$ in general). Therefore, the $e^{-ih_{2i}\tau}$ can be applied in parallel, and the $e^{-ih_{2i+1}\tau}$ can also be applied in parallel. In other words:

$$U(\tau) \approx e^{-i\tau H_{\text{odd}}} e^{-i\tau H_{\text{even}}} + \mathcal{O}(\tau^2), \quad (43)$$

where $H_{\text{odd}} = \sum_{i=1}^{N/2} h_{2i-1}$ and $H_{\text{even}} = \sum_{i=1}^{N/2-1} h_{2i}$. This is illustrated in Fig. 19.

It is also possible to decompose each time step into three sub-steps,

$$U(\tau) \approx e^{-i\tau H_{\text{odd}}/2} e^{-i\tau H_{\text{even}}} e^{-i\tau H_{\text{odd}}/2} + \mathcal{O}(\tau^3), \quad (44)$$

which leads to a smaller finite-step error at no extra computational cost (since the operator layer $e^{-i\tau H_{\text{odd}}/2}$ of one step is merged with the operator layer $e^{-i\tau H_{\text{odd}}/2}$ of the next step). The same logic can be used to construct higher-order decompositions. It is also possible to generalize the method to more complicated Hamiltonians, where the decomposition of H as a sum of Hamiltonians H_i in which all terms commute requires more than two terms [22, 53].

The algorithm above also works for imaginary time evolution, which can be used to find the ground state of a Hamiltonian by applying $e^{-\beta H}$ to an initial state and taking the limit $\beta \rightarrow +\infty$. However, this is not the most efficient way to find the ground state, and the DMRG algorithm is usually preferred for this purpose.

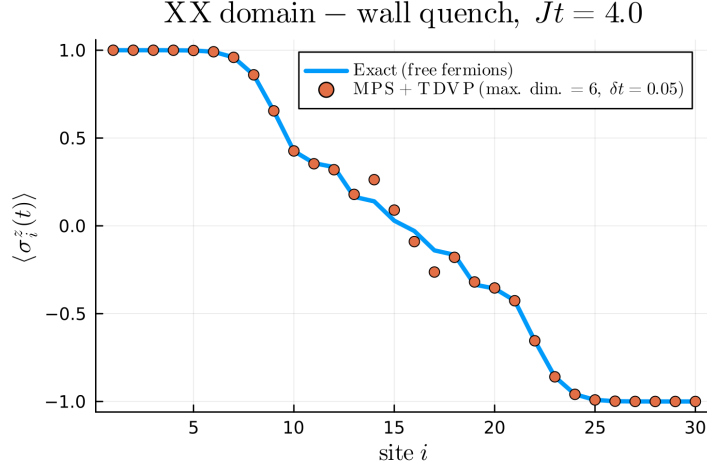


Figure 20: Magnetization profile for the domain wall quench in the XX spin- $\frac{1}{2}$ chain: exact versus TDVP results. Output of the code given in code example #9.

2.8.2 Time-dependent variational principle (TDVP) algorithm

Another important class of time-evolution algorithms for MPS is based on the time-dependent variational principle (TDVP). TDVP is a general strategy for approximating dynamics inside a given variational manifold. Instead of evolving the state in the full Hilbert space and projecting afterwards, one projects the Schrödinger equation itself onto the *tangent space of the variational manifold*, obtaining equations of motion for the variational parameters. Let $P_{T[|\psi\rangle]}$ be the projector onto the tangent space of the variational manifold at the point $|\psi\rangle$. The TDVP equation reads

$$i \frac{d}{dt} |\psi\rangle = P_{T[|\psi\rangle]} H |\psi\rangle. \quad (45)$$

The solution of this equation stays by construction in the variational manifold, and this approach is not specific to TNs. In the context of MPS, however, TDVP provides a powerful time-evolution algorithm [28, 29].

The simplest version is the one-site TDVP algorithm, which evolves within the manifold of MPS with a fixed bond dimension. The method is applicable for arbitrary Hamiltonians expressed as MPOs, and it guarantees energy conservation during the evolution. A two-site extension can also be used; it allows the bond dimension to grow dynamically during the evolution, which is important when entanglement increases. In that case, one uses a projector onto the linear space of *two-site* variations.

These algorithms are more complicated than TEBD and will not be presented in detail here. An example of code based on the TDVP implementation in ITensor is however given in code example #9.

Code example 9: TDVP for domain-wall melting in XX spin chain

([source online](#)) Time evolution of a domain-wall initial state in the XX spin- $\frac{1}{2}$ chain [66] using the two-site TDVP algorithm implemented in the ITensor library. The code compares the TDVP result with the exact solution obtained by mapping the XX chain to free fermions. The exact reference curve is obtained by diagonalizing the single-particle free-fermion Hamiltonian. The script evolves the domain-wall state up to the time $\mathfrak{t}$, plots $\langle \sigma_i^z(t) \rangle$, and prints the maximum pointwise deviation between TDVP and the exact result. The figure generated by the code is displayed in Fig. 20.

[XX_spin_chain_domain_wall.jl](#) on GitHub.

2.8.3 MPO methods (W^I and W^{II})

We mention here another family of time-evolution algorithms for MPS. The idea is to construct an MPO W which approximates the time-evolution operator $U = e^{tH}$ where t is a small (possibly complex) evolution parameter and where H is given as an MPO. We assume that H is extensive and write it, as

in [30], as a sum $H = \sum_x H_x$ where each term is labeled by a site x . Each term H_x acts only on a finite region around x , whose range we denote by k .

A simple possibility would be to write $U \simeq \mathbb{1} + t \sum_x H_x$, since this has a simple MPO representation as a sum of two MPOs. In terms of the time step t , the error would scale as $\mathcal{O}(t^2)$, but the number of second-order terms discarded in this approximation is quadratic in the system size N :

$$U - \left(\mathbb{1} + t \sum_x H_x \right) = \frac{t^2}{2} \sum_{x,y} H_x H_y + \mathcal{O}(t^3).$$

For this reason, this approximation would lead to an error per site that grows with N in large systems.

A second possibility would be to use $U \simeq \prod_x (\mathbb{1} + t H_x)$, which now keeps products of spatially separated terms. Its leading error (order t^2 as before) is now proportional to the system size:

$$U - \prod_x (\mathbb{1} + t H_x) = \frac{t^2}{2} \sum_x (H_x)^2 + \mathcal{O}(t^3).$$

So, the total error is $\mathcal{O}(Nt^2)$, instead of $\mathcal{O}(N^2t^2)$ for the previous approximation. The problem, however, is that $\prod_x (\mathbb{1} + t H_x)$ does not generally have a simple MPO form, even if H itself does.

In Ref. [30], Zaletel *et al.* proposed two approximations for U , called W^I and W^{II} , which i) have a compact MPO representation and ii) an error of order $\mathcal{O}(Nt^2)$. These algorithms can be applied to any Hamiltonian that can be expressed as an MPO. This is an important difference with the simple TEBD algorithm presented in Sec. 2.8.1, which is limited to Hamiltonians with nearest-neighbor interactions.

We briefly describe below the W^I method. As in [30], we use the notation $x < y$ if all sites in the support of H_x are strictly to the left of all sites in the support of H_y . If $x < y$, then the supports of H_x and H_y do not overlap (and the two terms commute). Since we assumed that each term H_x has a finite range around x , there is only a finite number of y such that neither $x < y$ nor $y < x$ holds. In the W^I approximation the evolution operator is approximated by

$$U_I(t) = \mathbb{1} + t \sum_x H_x + t^2 \sum_{x < y} H_x H_y + t^3 \sum_{x < y < z} H_x H_y H_z + \dots \quad (46)$$

The difference between U and U_I consists of products such as $H_x H_{x'}$, $H_x H_{x'} H_{x''}$, etc., in which at least two terms have overlapping support. The leading error is therefore of order t^2 and involves only pairs of overlapping terms. The error thus scales as $\mathcal{O}(Nt^2)$. The main advantage over $U \simeq \prod_x (\mathbb{1} + t H_x)$ is that U_I has a simple MPO form, with local tensors

$$W_I^{[i]} = \begin{pmatrix} \mathbb{1} + t D^{[i]} & \sqrt{t} C^{[i]} \\ \sqrt{t} B^{[i]} & A^{[i]} \end{pmatrix} \quad (47)$$

where the blocks A , B , C and D have been defined in Eq. (35) and describe the Hamiltonian MPO. If χ is the bond dimension of the MPO representing H , the above MPO for U_I has a bond dimension $\chi - 1$. We refer the reader to [30] for the proof that the MPO tensor Eq. (47) indeed gives the expansion Eq. (46) for the evolution operator.

Using Eq. (46) one can compose two time steps:

$$U_I(t_1)U_I(t_2) = \mathbb{1} + (t_1 + t_2) \sum_x H_x + (t_1^2 + t_2^2) \sum_{x < y} H_x H_y + t_1 t_2 \left(\sum_x H_x \right)^2 + \mathcal{O}(t_1^3) + \mathcal{O}(t_2^3). \quad (48)$$

It is possible to impose $U_I(t_1)U_I(t_2) = U(t) + \mathcal{O}(Nt^3)$ by choosing t_1 and t_2 such that $t_1 + t_2 = t$, $t_1^2 + t_2^2 = 0$ and $t_1 t_2 = t^2/2$. The solution is $t_1 = t(1+i)/2$ and $t_2 = t(1-i)/2$. Composing two imaginary time steps allows one to reduce the error from $\mathcal{O}(Nt^2)$ to $\mathcal{O}(Nt^3)$. In a similar way, composing four different time steps can reduce the error to $\mathcal{O}(Nt^4)$ and with seven steps the error is $\mathcal{O}(Nt^5)$ [67]. These schemes are implemented in the `TensorMixedStates` library.

The W^I method is simple but not yet optimal. For example, for a purely onsite Hamiltonian,

$$H = \sum_x h_x,$$

the exact evolution factorizes as

$$e^{tH} = \prod_x e^{t h_x},$$

which has a (trivial) MPO representation with bond dimension 1. By contrast, in this case W^I only gives

$$U_I = \prod_x (\mathbb{1} + t h_x).$$

This motivates the improved W^{II} construction, which also keeps products $H_x H_{x'} \dots$ such that no two terms cross the same bond. In particular, onsite terms are exponentiated exactly. The W^{II} MPO (not given here) is more complicated to construct than W^I , but it has the same bond dimension. Exactly as before, different time steps can be combined to reduce the error.

A short code illustrating how to call the W^{II} function implemented in `TensorMixedStates` is given in code example #10. This example illustrates that the method exactly solves the dynamics associated with a purely onsite Hamiltonian. Code examples #10, #15 and #16 are other code examples using the W^{II} method.

Code example 10: Short `TensorMixedStates` code showing that the W^{II} method *exactly* evolves a spin- $\frac{1}{2}$ chain in a uniform transverse field $H = B \sum_i S_i^x$, even with a large single time step. The initial state is the product state $|\uparrow\uparrow \dots \uparrow\rangle$, for which $\langle Z_i(t) \rangle = \cos(Bt)$. Note that the exactness of the time evolution would persist for an arbitrary initial state with bond dimension larger than 1.

[WII_onsite_field_exact.jl](#) on GitHub.

Code example 10: Exact time evolution with W^{II} MPO for onsite Hamiltonian

```

1  using TensorMixedStates, .Qubits
2  using Printf
3
4  N = 6
5  B = 1.0
6  t = 10.
7
8  system = System(N, Qubit())
9
10 # Initial state: normalized  $x$ /up up ... up> + (1 -  $x$ )/down down ... down>
11 psi0 = State{Pure}(system, "Up")
12
13 # Uniform magnetic field in the  $x$  direction:  $H = B * \sum_i S_{x,i}$ 
14 H = sum(B * Sx(i) for i in 1:N)
15
16 # WII evolution in a single finite time step.
17 psi_t = approx_W(
18     -im * H, t, psi0;
19     nsweeps = 1,
20     order = 1,
21     w = 2,
22     limits = Limits(cutoff = 1e-14, maxdim = 4),
23 )
24
25 mz = real(expect1(psi_t, Z))
26 mz_exact = fill(cos(B * t), N)
27 print("<Z_i> = ", mz, "\n")
28 @printf("max_i |<Z_i> - exact| = %.3e\n", maximum(abs.(mz .- mz_exact)))

```

3 Projected entangled pair states (PEPS)

Projected entangled pair states (PEPS) were introduced in 2004 by F. Verstraete and J. I. Cirac in [8]. They are a class of TN states that can efficiently represent quantum states obeying an area law for entanglement entropy in two or more spatial dimensions, and are the natural generalization of MPS and AKLT states. Each tensor is associated with a site of the lattice and has one physical index and several virtual indices that connect it to the neighboring tensors (Fig. 1). As for MPS, the bond dimension χ of the virtual indices controls the amount of entanglement that can be captured by the PEPS. Similarly, any quantum state can be represented as a PEPS if the associated bond dimension is large enough. However, such a representation is useful in practice only if the state of interest can be well approximated by a PEPS with a bond dimension that is reasonably small. One important difference with MPS is that there is no known efficient algorithm (i.e., polynomial in the system size) to contract exactly a generic PEPS. This implies that one has to rely on approximate methods to optimize PEPS and to compute expectation values of observables.

Before discussing PEPS it is useful to understand the difficulties of treating 2d systems with a 1d MPS, which motivates the introduction of PEPS.

3.1 Area law and difficulty of treating 2d systems with MPS

One can treat a 2d Hamiltonian with a 1d MPS by choosing an ordering of the lattice sites, for instance the snake ordering shown in Fig. 21. This strategy has been successfully used on several 2d models [15]. The first cost, however, is that the locality of the Hamiltonian is partly lost in the 1d representation, since some short-range interactions in the 2d problem translate into a long-range interaction along the 1d snake. This is illustrated in Fig. 21, where the vertical bonds of the original square lattice become longer-range couplings (orange arcs) in the 1d representation. We do not choose a specific Hamiltonian here, but we assume that it contains nearest-neighbor interactions on the square lattice. When expressing this Hamiltonian as an MPO, the bond dimension of the MPO will then grow with the range of the interactions along the 1d path. The bond dimension of the Hamiltonian MPO will thus grow with the dimension L_x of the 2d lattice.

A more serious difficulty is the fact that encoding a state with an area law for the entanglement entropy in 2d requires an MPS bond dimension that grows *exponentially* with the width of the system (L_x in Fig. 21). This can easily be understood by looking at the bipartition marked with the purple dashed line in Fig. 21. Such a bipartition cuts a single MPS bond (between sites 15 and 16). The entanglement entropy across this cut thus satisfies $S_{\text{vN}} \leq \log \chi$, where χ is the MPS bond dimension on bond (15, 16). However, this bipartition cuts L_x bonds of the 2d lattice. So, for a state with an area law for the entanglement entropy, the entanglement entropy across this cut scales as $S_{\text{vN}} \simeq aL_x$. We thus have $\chi \geq e^{aL_x}$. This means that the bond dimension of the MPS must grow exponentially with the width of the system to capture an area law state in 2d. This strongly limits the possible values of L_x (typically $L_x \lesssim 12$) but the length L_y can a priori be much larger. For an aspect ratio L_y/L_x that is of order one, the bond dimension of the MPS must grow exponentially with the system size in order to capture area law states in 2d.

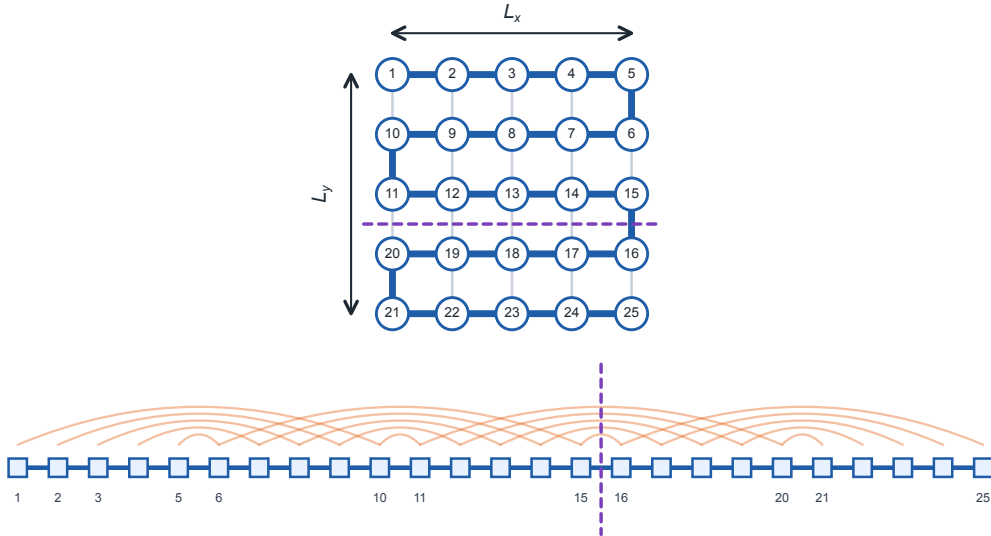


Figure 21: Mapping a Hamiltonian with nearest-neighbor interactions on a 5×5 square lattice to a 1d MPS by choosing a “snake” ordering of the sites. The original square lattice is shown in gray, while the blue path gives the ordering of the physical degrees of freedom in the MPS. In the unfolded 1d chain, the nearest-neighbor bonds along the snake path remain local, while vertical bonds of the original square lattice become longer-range couplings (orange arcs). A bipartition after site 15 (dashed purple line) corresponds to a single MPS bond cut, but cuts L_x bonds of the original 2d lattice.

The code example [#11](#) studies noninteracting spinless fermions on a 2d $L_x \times L_y$ square lattice with open boundary conditions. It first builds the single-particle tight-binding Hamiltonian and computes (in polynomial time) the exact half-filled ground-state energy and a connected density-density correlation. It then solves the same problem with DMRG (`ITensors`) and an MPO formulation of the Hamiltonian. To do so, a snake (or zigzag) site ordering is used to map the 2d lattice to a 1d chain with long-range

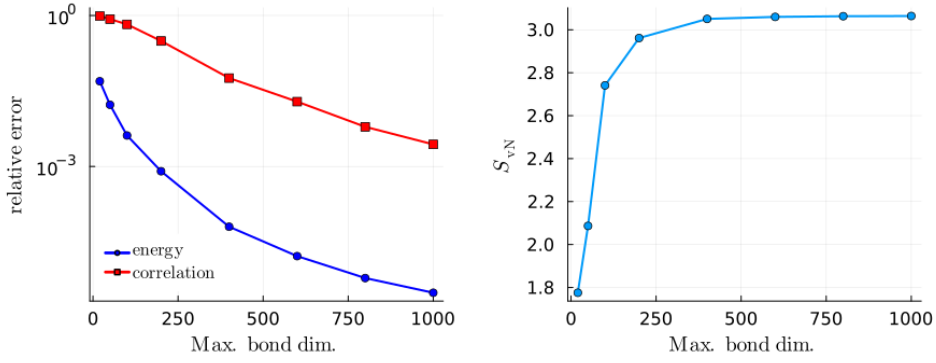


Figure 22: Data from code example #11 for a 7×6 lattice. The correlation considered is the connected density-density correlation between the two sites that are the farthest apart on the lattice, at opposite corners (see code). The left panel shows the relative error on the ground-state energy and on the connected density-density correlation as a function of the MPS bond dimension. The right panel shows the entanglement entropy across the middle bond of the snake ordering as a function of the MPS bond dimension.

hoppings. The code reports the DMRG ground state energy and the connected density-density correlation, and compares them to the exact results.

If one analyzes the *relative* error on the ground-state energy and the connected density-density correlation, one finds that the error on the energy is significantly smaller than the error on the correlation. This is shown in Fig. 22 for a 7×6 lattice, where the relative error on the energy is of order 10^{-5} while the relative error on the correlation is of order 10^{-2} for a bond dimension $\chi = 100$.

Code example 11: DMRG for 2d free fermions

([source online](#)) Noninteracting spinless fermions on a 2d $L_x \times L_y$ square lattice: exact free fermion calculation versus DMRG. The 2d lattice is mapped to a 1d chain using a snake ordering. The code computes the exact half-filled free-fermion ground-state energy by diagonalizing the single-particle Hamiltonian, then compares it with DMRG results for increasing values of `maxdim`. It also compares a connected density-density correlation between opposite corners of the lattice and reports the entanglement entropy across the middle bond of the snake-ordered MPS.

[dmrg_2D_fermions.jl](#) on GitHub.

Contrary to the snake-MPS discussed above, a PEPS can capture an area law for the entanglement entropy with a bond dimension that does not grow with the system size. This is easily understood by noticing that a spatial bipartition of the system cuts a number of virtual bonds that is by construction proportional to the length of the boundary between the two subsystems, as illustrated in Fig. 23. In this way, a PEPS with a fixed bond dimension χ can capture an area law state in 2d, with an entanglement entropy that scales as $S_{vN} \sim \mathcal{O}(l)$, where l is the length of the boundary between the two subsystems.

Remark

Consider a quantum state $|\psi\rangle$ defined on some lattice and represented by a TN, such as the PEPS shown in Fig. 23. The reduced density matrix ρ_A of a subsystem A is obtained by tracing out the degrees of freedom of the complementary subsystem B : $\rho_A = \text{Tr}_B |\psi\rangle\langle\psi|$. A certain number of virtual bonds $\alpha_1, \alpha_2, \dots, \alpha_l$ will cross the boundary between A and B . Assume for simplicity that they all have the same bond dimension χ . If we group them into a single index α which can take χ^l values, the state $|\psi\rangle$ reads

$$|\psi\rangle = \sum_{\alpha} |\alpha, A\rangle \otimes |\alpha, B\rangle, \quad (49)$$

where $|\alpha, A\rangle$ and $|\alpha, B\rangle$ are states of the subsystems A and B when the virtual indices crossing the boundary are fixed to $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_l)$. The reduced density matrix of the subsystem A can then

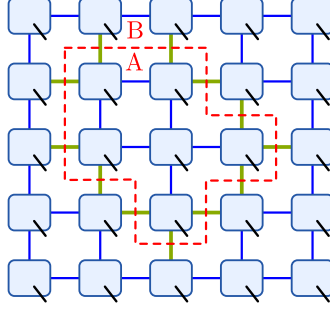


Figure 23: A PEPS representation of a quantum many-body state on a 2d lattice, with a subsystem A separated from its complement B by a boundary (red dotted line). The virtual indices which cross this boundary are colored in green. The physical indices are the (black) open legs.

be written as

$$\rho_A = \sum_{\alpha, \alpha'} \text{Tr}_B [|\alpha, B\rangle \langle \alpha, A| \langle \alpha', A| \langle \alpha', B|] \quad (50)$$

$$= \sum_{\alpha, \alpha'} M_{\alpha, \alpha'} |\alpha, A\rangle \langle \alpha', A| \quad (51)$$

with the matrix M given by $M_{\alpha, \alpha'} = \text{Tr}_B [|\alpha, B\rangle \langle \alpha', B|] = \langle \alpha', B | \alpha, B \rangle$. We see in Sec. 3.1 that the image of ρ_A is included in $\text{span}\{|\alpha, A\rangle\}$, whose dimension is upper bounded by $\text{rank}[\rho] \leq \chi^l$. The (von Neumann) entropy S_{vN} of ρ_A is therefore upper bounded by $\ln(\chi^l)$ and $S_{\text{vN}} \leq l \log \chi$.

3.2 Example 1: PEPS derived from a classical thermal state

Using the transfer matrix formalism we saw in Sec. 2.5.3 that MPS (with finite bond dimension) generically have exponentially decaying correlations. The situation is different for PEPS in $d > 1$. In particular, PEPS (with a finite bond dimension) can have power-law decaying correlations, and thus can represent critical states. This can be shown by constructing an explicit example of critical PEPS. To do so, one starts from a classical model of statistical mechanics defined by local Boltzmann weights. From this model one can construct a PEPS (and thus a quantum state) with the same correlation functions as the classical model.⁸ At the critical point, this gives a PEPS with power-law decaying correlations [70].

Following the presentation given in [47], we illustrate this construction in the case of the 2d Ising model with classical spins $\sigma_i = \pm 1$ and energy $E(\{\sigma\}) = -J \sum_{\langle i, j \rangle} \sigma_i \sigma_j$. The Boltzmann weight of a configuration $\{\sigma\}$ is given by $e^{-\beta E(\{\sigma\})}$, where β is the inverse temperature. The partition function is given by $Z = \sum_{\{\sigma\}} e^{-\beta E(\{\sigma\})}$. From this we define a quantum state $|\psi\rangle$ (parametrized by βJ) as follows:

$$|\psi\rangle = \frac{1}{\sqrt{Z}} \sum_{\{\sigma\}} e^{-\frac{\beta}{2} E(\{\sigma\})} |\sigma_1 \sigma_2 \dots \sigma_N\rangle \quad (52)$$

$$= e^{\frac{\beta J}{2} \sum_{\langle i, j \rangle} \sigma_i^z \sigma_j^z} |++\dots+\rangle, \quad (53)$$

with $|+\rangle = |\uparrow\rangle + |\downarrow\rangle$. By construction, a correlation function of the type $\langle \psi | \sigma_i^z \sigma_j^z | \psi \rangle$ is given by

$$\langle \psi | \sigma_i^z \sigma_j^z | \psi \rangle = \frac{1}{Z} \sum_{\{\sigma\}} e^{-\beta E(\{\sigma\})} \sigma_i \sigma_j, \quad (54)$$

which is nothing but the correlation function of the classical model. As we will see, the quantum state $|\psi\rangle$ can be represented as a PEPS with bond dimension $\chi = 2$.

The first step is to consider two neighboring sites i and j and the operator $e^{\frac{\beta J}{2} \sigma_i^z \sigma_j^z}$. This is a two-site operator that can be written as an MPO. It turns out that the bond dimension of this MPO is (only)

⁸Note that such quantum states constructed from a classical model are closely related to so-called Rokhsar-Kivelson states [68, 69].

$\chi = 2$. This can be seen by writing $e^{\frac{\beta J}{2} \sigma_i^z \sigma_j^z} = \cosh(\frac{\beta J}{2}) \mathbb{1}_i \otimes \mathbb{1}_j + \sinh(\frac{\beta J}{2}) \sigma_i^z \otimes \sigma_j^z$. This can be represented as an MPO with bond dimension $\chi = 2$ by introducing a virtual index α that takes two values, $\alpha = 0$ and $\alpha = 1$, and writing

$$e^{\frac{\beta J}{2} \sigma_i^z \sigma_j^z} = \sum_{\alpha=0,1} W_\alpha^{[i]} W_\alpha^{[j]}, \quad (55)$$

where $W_\alpha^{[i]}$ and $W_\alpha^{[j]}$ are local operators acting on sites i and j respectively, and are given by

$$W_0^{[i]} = \sqrt{c} \mathbb{1}_i, \quad W_1^{[i]} = \sqrt{s} \sigma_i^z, \quad (56)$$

$$W_0^{[j]} = \sqrt{c} \mathbb{1}_j, \quad W_1^{[j]} = \sqrt{s} \sigma_j^z, \quad (57)$$

$$\text{with } c = \cosh(\frac{\beta J}{2}) \text{ and } s = \sinh(\frac{\beta J}{2}). \quad (58)$$

The second step is to insert the above MPO representation for $e^{\frac{\beta J}{2} \sigma_i^z \sigma_j^z}$ into Sec. 3.2 for each pair of neighboring sites $\langle i, j \rangle$ of the lattice. One then obtains an expression for $|\psi\rangle$ as a PEPS

$$|\psi\rangle = \sum_{\{\alpha\}} \prod_{\langle i, j \rangle} W_{\alpha_{ij}}^{[i]} W_{\alpha_{ij}}^{[j]} |++\dots+\rangle, \quad (59)$$

where $\alpha_{ij} = 0, 1$ is the virtual index associated with the bond between sites i and j . By regrouping the operators acting on each site, one can write $|\psi\rangle$ as a PEPS where the local tensors $A^{[i]}$ are:

$$A_{\alpha_1, \alpha_2, \alpha_3, \alpha_4}^{[i] s_i} = \langle s_i | W_{\alpha_1}^{[i]} W_{\alpha_2}^{[i]} W_{\alpha_3}^{[i]} W_{\alpha_4}^{[i]} | + \rangle \quad (60)$$

$$= \langle s_i | W_{\alpha_1}^{[i]} W_{\alpha_2}^{[i]} W_{\alpha_3}^{[i]} W_{\alpha_4}^{[i]} | s_i \rangle, \quad (61)$$

where $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ are the virtual indices corresponding to the four nearest neighbors of site i , and $s_i = \uparrow, \downarrow$ is the physical index corresponding to the spin state at site i . In the second line we have used the fact that $W_\alpha^{[i]}$ is diagonal in the σ^z basis. This gives an explicit expression for the 5-index tensor $A^{[i]}$ of the PEPS, which has a bond dimension $\chi = 2$ and a physical dimension $d = 2$. As explained above, the correlation functions of $|\psi\rangle$ are the same as those of the classical 2d Ising model, and thus decay algebraically at the critical point ($\beta J = \frac{1}{2} \ln(1 + \sqrt{2})$).

3.3 Example 2: Toric code

Kitaev's toric code [71] provides a simple but paradigmatic example of a topologically ordered state. It exhibits long-range entanglement, anyonic excitations and topological degeneracy. As described below, it can be represented as a PEPS with bond dimension $\chi = 2$, which shows that a PEPS with finite bond dimension can capture topological order.

The Hamiltonian of the toric code is defined on a square lattice with spin- $\frac{1}{2}$ degrees of freedom living on the bonds of the lattice. It is given by

$$H = - \sum_s A_s - \sum_p B_p, \quad (62)$$

where $A_s = \prod_{i \in s} \sigma_i^x$ is a product of σ^x operators around a vertex s and $B_p = \prod_{i \in p} \sigma_i^z$ is a product of σ^z operators around a plaquette p . One can easily check that the A_s and B_p have eigenvalues ± 1 and they all commute with each other (even when they share some sites). The (possibly degenerate) ground states of H are the common eigenstates of all A_s and B_p with eigenvalue $+1$.⁹

To construct a PEPS representation of this state, one can start from the ferromagnetic product state $|\uparrow \dots \uparrow\rangle$, which is the eigenstate of all the σ_i^z with eigenvalue $+1$. This state trivially minimizes the energy of the B_p terms, but not of the A_s terms. To minimize the energy of the A_s terms, one can apply the projector $\prod_s \frac{1+A_s}{2}$ to the state $|\uparrow \dots \uparrow\rangle$:

$$|\psi\rangle = \prod_s \frac{1+A_s}{2} |\uparrow \dots \uparrow\rangle. \quad (63)$$

Since the A_s terms commute with the B_p terms, $|\psi\rangle$ is still an eigenstate of all B_p with eigenvalue $+1$. Moreover, it is now also an eigenstate of all A_s with eigenvalue $+1$, and thus minimizes the energy of all terms in the Hamiltonian. To obtain a PEPS representation of $|\psi\rangle$, we follow [47] and write the projector $\Pi_s = \frac{1+A_s}{2}$ as a product of four tensors, as illustrated in Fig. 24.

⁹Even though it is an important aspect of the toric code, we will not discuss here the (topological) degeneracy of such states, nor their excited states.

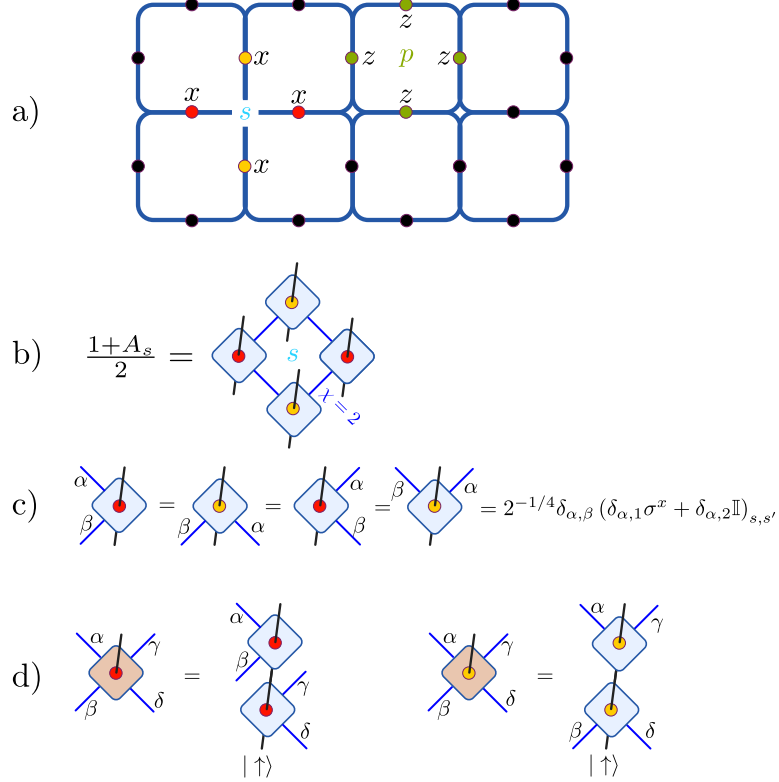


Figure 24: PEPS representation of the toric-code ground state. a) In the toric code each vertex s of the square lattice is associated with a star made with four spins (cyan dots). The associated operator A_s is the product of σ^x operators around the vertex. Each plaquette p of the square lattice is associated with four spins (green dots), as well as the operator B_p which is the product of σ^z operators around the plaquette. b) The projector $\frac{1+A_s}{2}$ can be written as an MPO using 4 tensors. c) Representation of the tensors used in the MPO. The tensors have virtual indices (blue lines) of dimension $\chi = 2$. The MPO which represents $\frac{1+A_s}{2}$ thus also has bond dimension 2. d) The local tensors of the PEPS. The form slightly depends on whether the site belongs to one sublattice or the other (yellow or red dot).

3.4 Other examples

To conclude this list of examples we mention high-dimensional generalizations of the AKLT state. The 1d AKLT construction discussed in Sec. 2.3.4 can be generalized to higher-dimensional lattices provided the value S of the spin is related to the coordination number z of the lattice by $2S = 0 \bmod z$ [72]. The idea generalizes the construction displayed in Fig. 4 and consists of placing z virtual spin- $\frac{1}{2}$ degrees of freedom on each site and forming a singlet on each bond. The physical state is obtained by projecting the z virtual spin- $\frac{1}{2}$ degrees of freedom onto the symmetric subspace of dimension $2S + 1$. For the same reason as in 1d, the resulting state can be represented exactly as a PEPS with bond dimension $\chi = 2$. Notable examples include the 2d AKLT state on the hexagonal lattice (with $S = 3/2$) and the 2d AKLT state on the square lattice (with $S = 2$).

Another family of examples is given by the so-called resonating valence bond (RVB) states, which are superpositions of valence bond (spin singlets) configurations. These states play a role in the theory of high-temperature superconductivity [73, 74] and quantum spin liquids. Some of these states can be represented as PEPS with a small bond dimension, starting from $\chi = 3$ [70]. The idea is that the virtual degrees of freedom on each bond encode whether a singlet is present on that bond or not ($\alpha = 1 \rightarrow$ no singlet, $\alpha = 2 \rightarrow$ component $|\uparrow\rangle$ of the singlet, $\alpha = 3 \rightarrow$ component $|\downarrow\rangle$ of the singlet).

3.5 Contraction of PEPS

The exact contraction of a PEPS (which is required to compute expectation values and correlation functions) is computationally hard [62, 70]. Thus, unlike for MPS, there is no known efficient algorithm (*i.e.*, polynomial) to perform the exact contraction of a PEPS in the general case. However, several *approximate* methods have been developed to contract PEPS, which allow expectation values and correlation functions to be computed with good accuracy in many cases. We can mention the boundary MPS method [8], the corner transfer matrix method [75, 76], the stochastic Monte Carlo sampling method [77], the tensor renormalization group method [78], or methods based on belief propagation [79], among others. Below we briefly describe the first method, which is the simplest.

For a finite PEPS, the contraction can be performed by grouping the tensors into rows and contracting them sequentially using MPS and MPO techniques. We will sketch the method for a square-lattice PEPS and a local operator O_i acting on a single site i , but the method can be adapted to other lattices and to more general operators. To compute the norm $\langle\psi|\psi\rangle$ and the expectation value $\langle\psi|O_i|\psi\rangle$, we need to compute the contractions of the TNs depicted in Fig. 25.

Away from the site i , the physical index can be contracted locally, giving so-called double-layer tensors

$$E_{\ell\ell',rr',uu',dd'}^{[j]} = \sum_s \overline{A_{\ell'r'u'd'}^{[j]s}} A_{\ell r u d}^{[j]s}, \quad (64)$$

whose four virtual indices have dimension χ^2 instead of χ (Fig. 25b). At the site where the operator is inserted, one uses instead $\sum_{s,s'} \overline{A^{[i]s'}} O_{s's} A^{[i]s}$. The resulting 2d network can then be contracted row by row. The first boundary row is viewed as an MPS. Its physical index is the vertical virtual index of the double-layer tensors (dimension χ^2), and its virtual indices are the horizontal virtual indices of the double-layer tensors (dimension χ^2). The second row is viewed as an MPO acting on this boundary MPS, and the contraction of the two rows gives a new boundary MPS. The same procedure is carried out from the top and from the bottom, until the two boundary MPS meet at the row where the operator O_i is inserted. Unless one performs some truncation, the bond dimension of the boundary MPS will grow exponentially with the number of rows that have been absorbed. Therefore, each MPS–MPO multiplication needs to be accompanied by a truncation step to a bond dimension χ_{env} , as explained in Sec. 2.6.5. The approximation is controlled by the boundary bond dimension χ_{env} : increasing it improves the representation of the effective environment, at the price of a larger computational cost. At the last step, one is left with the calculation of the scalar product of two MPS, which can be done efficiently (contraction from left to right, see Sec. 2.5.2). This approach can also be used to construct the environment tensors required for the optimization of PEPS, as sketched in Fig. 25c.

4 Mixed states and open systems

In an open quantum system, states are generally mixed states described by density matrices ρ rather than pure states $|\psi\rangle$. After *vectorization*, a density matrix can be treated as a vector in a doubled local Hilbert space. Most of the TN tools introduced in the previous sections can then be reused almost directly. 2d

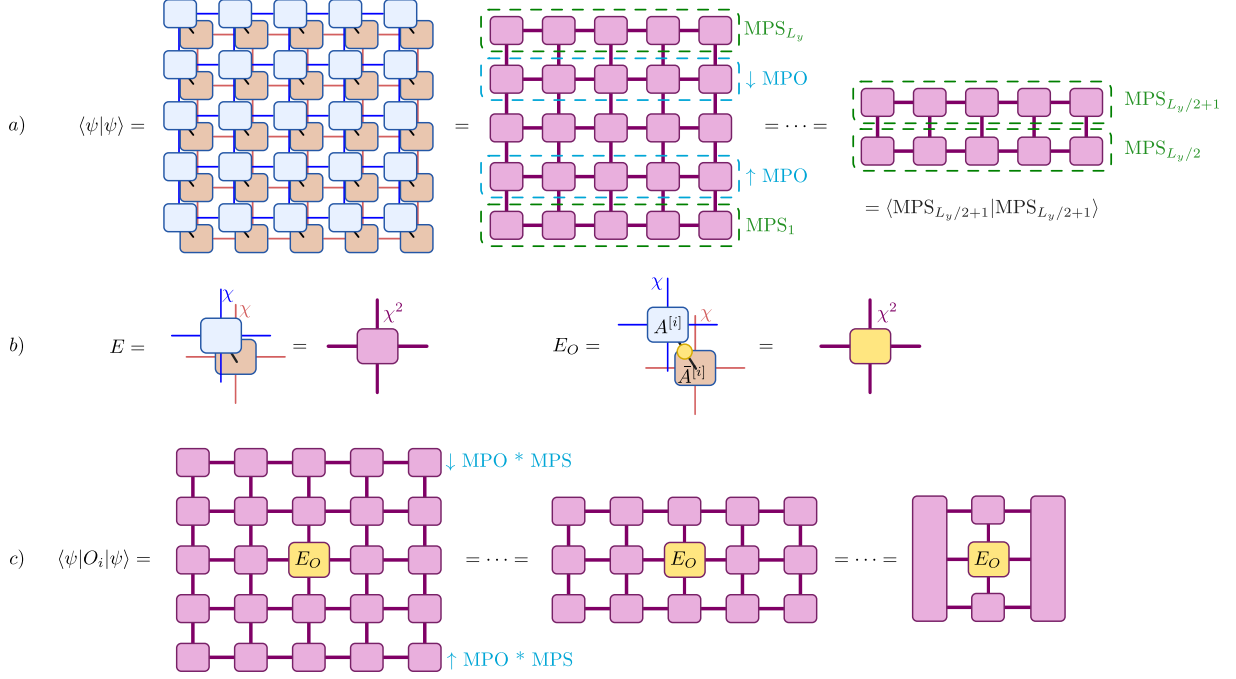


Figure 25: Contraction of a finite PEPS using double-layer tensors. The norm $\langle\psi|\psi\rangle$ and the expectation value $\langle\psi|O_i|\psi\rangle$ are represented as 2d TNs. After the physical indices are contracted, each row can be viewed as an MPO acting on a boundary MPS. The local tensor E_O differs from the double-layer tensor E only at the site where the operator O_i is inserted.

TNs have been used to describe mixed states and open quantum systems [80–82] but in these notes we will focus on MPS and MPO representations.

This section contains a few code examples based on the `TensorMixedStates` library [3], a Julia package built on `ITensors.jl` for the simulation of mixed states and open quantum systems with MPS and MPO.

4.1 Vectorization and TN for a mixed state

The vectorization of a matrix of dimension $D_1 \times D_2$ consists in reshaping it into a *vector* of dimension $D_1 D_2$. If a matrix M represents an operator $\hat{M}$ acting on a Hilbert space of dimension D it can be written as $\hat{M} = \sum_{i,j=1}^D M_{ij} |i\rangle \langle j|$, where $|i\rangle$ are basis states of the Hilbert space. The vectorization of the operator $\hat{M}$ is then given by

$$|M\rangle\rangle = \sum_{i,j} M_{ij} |i\rangle \otimes |j\rangle, \quad (65)$$

where the two copies of the basis states $|i\rangle$ and $|j\rangle$ are now viewed as basis states of an enlarged Hilbert space of dimension D^2 . This is called the Choi-Jamiolkowski isomorphism [83, 84].

Now consider the case of a density matrix ρ of a quantum system with N sites and a local Hilbert space of dimension d at each site. ρ can be decomposed in a basis of local operators as

$$\rho = \sum_{s_1, \dots, s_N} R_{s_1 \dots s_N} O^{[1]s_1} \otimes \dots \otimes O^{[N]s_N} \quad (66)$$

where $O^{[k]s_k}$ are local operators acting on site k and s_k runs from 1 to d^2 . As in Eq. (65), the coefficients $R_{s_1 \dots s_N}$ can be viewed as the components of a *pure* state in a Hilbert space of dimension d^{2N} . With this point of view, we may denote the product of local operators $O^{[1]s_1} \otimes \dots \otimes O^{[N]s_N}$ as $|s_1 \dots s_N\rangle\rangle$ and write the mixed state ρ as

$$|\rho\rangle\rangle = \sum_{s_1, \dots, s_N} R_{s_1 \dots s_N} |s_1 \dots s_N\rangle\rangle. \quad (67)$$

Here the “super-ket” notation $|\dots\rangle\rangle$ is used to distinguish the pure state $|\rho\rangle\rangle$ in the enlarged space from a usual pure state $|\psi\rangle$ in the original Hilbert space. In turn, the coefficients $R_{s_1 \dots s_N}$ can be decomposed

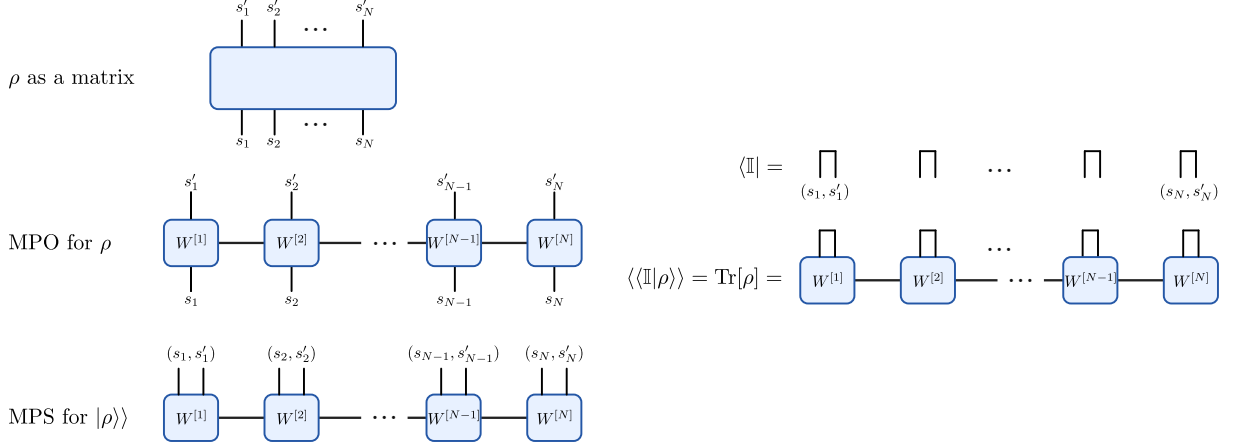


Figure 26: Left: three representations of a density matrix. The matrix ρ of an N -site system can first be viewed as a single tensor with N physical indices on the top and N physical indices on the bottom. It can then be factorized as a matrix product operator (MPO), where each local tensor has two physical indices and, except at the boundaries, two virtual indices. By vectorizing the operator, or equivalently by folding the lower physical legs upward, ρ can be represented as a super-ket $|\rho\rangle\rangle$ in MPS form, with a doubled local physical index (s_i, s'_i) on each site. Right: Graphical representation for the fully mixed state $\langle\langle\mathbb{I}|\rangle\rangle$ (identity density matrix) and for the scalar product with $|\rho\rangle\rangle$, which gives the trace of ρ .

as an MPS,

$$R_{s_1 \dots s_N} = R^{[1]s_1} \dots R^{[N]s_N}. \quad (68)$$

The vectorized density matrix then takes the (super) MPS form

$$|\rho\rangle\rangle = \sum_{s_1, \dots, s_N} R^{[1]s_1} \dots R^{[N]s_N} |s_1 \dots s_N\rangle\rangle. \quad (69)$$

The main difference with an ordinary MPS is that the local physical index now labels a local operator, or equivalently a pair of local ket/bra indices, and therefore has dimension d^2 instead of d . Note that one may also view ρ as an MPO with two physical indices per site. Folding the lower physical leg upward turns this MPO representation of ρ into the MPS representation of $|\rho\rangle\rangle$, as illustrated in Fig. 26.

Thanks to the representation of $|\rho\rangle\rangle$ as an MPS, most of the tools introduced in the previous sections for pure states can be used to manipulate mixed states.

Remark

This representation does not automatically enforce the positive-semidefinite character of the density matrix. In particular, after some truncation on the MPS representation of $|\rho\rangle\rangle$ there is no guarantee that the reconstructed ρ is positive semidefinite. In practice, in numerical simulations, one monitors the trace, Hermiticity, and the convergence of observables with the MPS bond dimension. It is also possible to use representations that automatically enforce the positivity of ρ , such as the purification representation $\rho = XX^\dagger$ with X represented as an MPO [85]. See Sec. 4.3. Such TNs, with a double-layer structure (one for X and one for $X^\dagger$), are called matrix product density operators (MPDOs). Even though it does not guarantee the positivity of ρ , the simple vectorized form is often used in practice since the purification representation can be quite expensive. As always in MPS simulations, the convergence of the results with the bond dimension should be checked.

The scalar product in the vectorized space is the Hilbert-Schmidt scalar product,

$$\langle\langle A|B\rangle\rangle = \text{Tr}(A^\dagger B). \quad (70)$$

For instance,

$$\text{Tr}(\rho) = \langle\langle I|\rho\rangle\rangle, \quad \text{Tr}(\rho^2) = \langle\langle \rho|\rho\rangle\rangle \quad \langle O \rangle = \text{Tr}(O\rho) = \langle\langle O^\dagger|\rho\rangle\rangle \quad (71)$$

(in the second equality we have used the fact that for a density matrix $\rho = \rho^\dagger$).

4.2 Operator-space entanglement entropy

The entanglement entropy of the vectorized state $|\rho\rangle\rangle$ across a spatial bipartition is called the *operator-space entanglement entropy* (OSEE) [86–89]. The OSEE quantifies the amount of correlations, both quantum and classical, between the two subsystems in the mixed state ρ . The OSEE can also be used to characterize a unitary operator (rather than a mixed state).

Just as the entanglement entropy of a pure state controls the MPS bond dimension needed to represent it, the OSEE controls the bond dimension needed to represent a density matrix in the vectorized MPS form. The OSEE is therefore an important quantity in the context of numerical simulations since it quantifies how expensive it is to represent $|\rho\rangle\rangle$ as an MPS. Roughly speaking, a bond dimension χ can describe an OSEE up to order $\sim \log \chi$. In the same way as entanglement entropy growth in pure-state dynamics may limit the use of TN methods, and MPS in particular, a fast growth of the OSEE during mixed-state or open-system dynamics may require large bond dimensions.

Two simple limits are useful to keep in mind. If $\rho = |\psi\rangle\langle\psi|$ is a pure-state density matrix, then the OSEE of $|\rho\rangle\rangle$ is twice the von Neumann entanglement entropy of $|\psi\rangle$ for the same spatial bipartition. On the other hand, if $\rho = \rho_1 \otimes \cdots \otimes \rho_N$ is a product state, then its vectorized form is also a product state, and the OSEE vanishes (a special limit is the fully mixed case, $\rho \propto \mathbb{1}$).

code example #16 is an example of code which computes the OSEE as a function of time for a dissipative spin chain. The results are shown in Fig. 30. The initial growth of the OSEE is due to the buildup of correlations in the system under the action of the Hamiltonian, while the later decrease is due to the dissipation which drives the system toward a fully mixed product state (hence with zero OSEE).

4.3 Thermal equilibrium states by purification

The idea of purification, which is not specific to TNs, is a general way of representing a mixed state as a pure state in a larger Hilbert space. This has been intensively exploited in the context of TNs and MPS in particular [22, 33, 34, 53, 90].

Consider a density matrix ρ acting on a (physical) Hilbert space $\mathcal{H}_P$, and write its spectral decomposition as

$$\rho = \sum_a p_a |a\rangle_P \langle a|_P, \quad p_a \geq 0, \quad \sum_a p_a = 1. \quad (72)$$

One then introduces an auxiliary Hilbert space $\mathcal{H}_Q$, which is a copy of $\mathcal{H}_P$, with an orthonormal basis $\{|a\rangle_Q\}$. One defines a *pure* state

$$|\Psi_\rho\rangle = \sum_a \sqrt{p_a} |a\rangle_P \otimes |a\rangle_Q \quad (73)$$

such that the original density matrix is recovered by tracing out the auxiliary space,

$$\rho = \text{Tr}_Q |\Psi_\rho\rangle \langle \Psi_\rho|. \quad (74)$$

Note that the purification is not unique: applying any unitary transformation acting only on the auxiliary space leaves the reduced density matrix on $\mathcal{H}_P$ unchanged. This can sometimes be exploited to reduce the entanglement between the auxiliary degrees of freedom.

For the thermal equilibrium state associated with a Hamiltonian H at inverse temperature β ,

$$\rho_\beta = \frac{1}{Z(\beta)} e^{-\beta H}, \quad Z(\beta) = \text{Tr} e^{-\beta H}, \quad (75)$$

the natural purification is the thermofield-double state

$$|\Psi_\beta\rangle = \frac{1}{\sqrt{Z(\beta)}} \sum_n e^{-\beta E_n/2} |n\rangle_P \otimes |n\rangle_Q, \quad (76)$$

where $|n\rangle_P$ are eigenstates of H , and $|n\rangle_Q$ are the corresponding states in the auxiliary space. By construction tracing out the auxiliary space gives the desired thermal density matrix for the physical system

$$\text{Tr}_Q |\Psi_\beta\rangle \langle \Psi_\beta| = \frac{1}{Z(\beta)} \sum_n e^{-\beta E_n} |n\rangle_P \langle n|_P = \rho_\beta. \quad (77)$$

To construct $|\Psi_\beta\rangle$, one may start from a maximally entangled purification of the infinite-temperature state ($\beta = 0$) and evolve the physical degrees of freedom in imaginary time by $\beta/2$:

$$|\Psi_\beta\rangle = \frac{1}{\sqrt{Z(\beta)}} \left(e^{-\beta H/2} \otimes \mathbb{I}_Q \right) |\Psi_0\rangle \quad (78)$$

with

$$|\Psi_0\rangle = \sum_a |a\rangle_P \otimes |a\rangle_Q. \quad (79)$$

In an MPS implementation, one convenient ordering is to alternate physical and auxiliary sites, $P_1, Q_1, P_2, Q_2, \dots, P_N, Q_N$. For a two-level system, a possible infinite-temperature purification Eq. (79) is then a product of maximally entangled states on each pair (P_i, Q_i) ,

$$|\Psi_0\rangle = \bigotimes_{i=1}^N \left(|0\rangle_{P_i} |0\rangle_{Q_i} + |1\rangle_{P_i} |1\rangle_{Q_i} \right) / \sqrt{2}. \quad (80)$$

Starting from this state, a finite-temperature purification is obtained with Eq. (78). Such imaginary-time evolution can be performed using one of the time-evolution methods described in Sec. 2.8, such as TDVP or a W method. Importantly, the imaginary-time evolution operator $e^{-\beta H/2}$ acts only on the physical sites. But due to the initial entanglement between physical and auxiliary sites, $|\Psi_\beta\rangle$ is an entangled state of the physical and auxiliary degrees of freedom. After normalization, thermal expectation values of physical observables are ordinary pure-state expectation values,

$$\langle O \rangle_\beta = \frac{\langle \Psi_\beta | O_P | \Psi_\beta \rangle}{\langle \Psi_\beta | \Psi_\beta \rangle}. \quad (81)$$

Code example #12 illustrates this construction for an open XX spin- $\frac{1}{2}$ chain. In this example, the imaginary-time evolution is performed by TDVP on an MPS representing Eq. (80).

Code example 12: Thermal equilibrium state of an XX chain by purification and TDVP

[\(source online\)](#) Finite-temperature calculation for an open XX spin- $\frac{1}{2}$ chain ($H = J \sum_i (S_i^x S_{i+1}^x + S_i^y S_{i+1}^y)$) using a purified MPS. The code prepares the infinite-temperature state as Bell pairs between physical and auxiliary sites (Eq. (80)), evolves the physical sites in imaginary time by TDVP (ITensor implementation), computes the mean energy, and compares it with the exact (free-fermion) expression: $E(\beta) = \sum_{m=1}^N \frac{\epsilon_m}{1 + \exp(\beta \epsilon_m)}$ with the fermionic mode energies $\epsilon_m = J \cos\left(\frac{\pi m}{N+1}\right)$. The script sweeps over a few inverse temperatures and prints a comparison table containing the TDVP energy, the exact free-fermion energy, the absolute error, and the maximum bond dimension of the purified MPS.

[thermal_xx_purification.jl](#) on GitHub.

4.4 Superoperators as MPOs

With vectorization, linear maps acting on density matrices become ordinary linear operators acting on the super-ket $|\rho\rangle\rangle$. With the convention of Eq. (65), one has the identity

$$|A\rho B^\dagger\rangle\rangle = (A \otimes \overline{B})|\rho\rangle\rangle, \quad (82)$$

where the overline denotes complex conjugation in the local basis used for the vectorization. In other words, acting on the left of the density matrix is equivalent to acting on the first copy of the doubled space, while acting on the right is equivalent to acting on the second copy. So, left and right multiplication of ρ become ordinary operators in the doubled Hilbert space. The idea is to express the operator $A \otimes \overline{B}$ as an MPO (acting on the doubled Hilbert space), so that $|A\rho B^\dagger\rangle\rangle$ can be computed by an MPO-MPS multiplication, as in Sec. 2.6. In the special case of a unitary gate acting on a mixed state we have

$$\rho \mapsto U\rho U^\dagger, \quad |\rho\rangle\rangle \mapsto (U \otimes \overline{U})|\rho\rangle\rangle. \quad (83)$$

Again, this can be viewed as the application of an MPO (representing $U \otimes \overline{U}$) on an MPS. More generally, a quantum channel is a completely positive map that can be written in Kraus form [91]

$$\mathcal{E}(\rho) = \sum_\mu K_\mu \rho K_\mu^\dagger, \quad \sum_\mu K_\mu^\dagger K_\mu = \mathbb{1} \quad (84)$$

for a trace-preserving channel. In vectorized form this becomes

$$|\mathcal{E}(\rho)\rangle\rangle = \sum_\mu K_\mu \otimes \overline{K_\mu} |\rho\rangle\rangle. \quad (85)$$

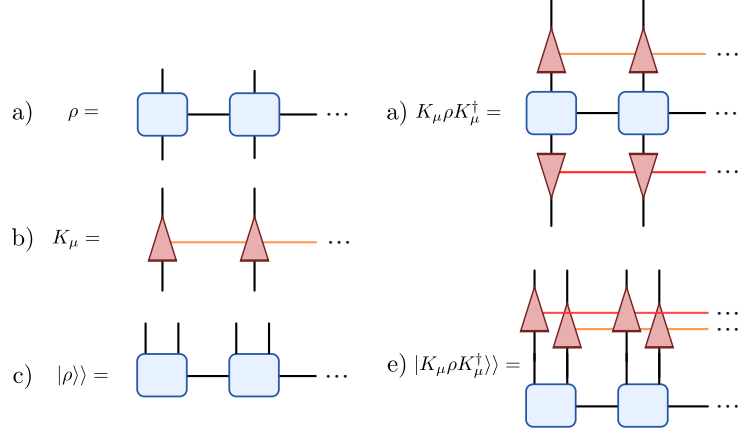


Figure 27: Diagrammatic representation of the vectorization procedure for a quantum channel in Kraus form. a) Density matrix ρ as an MPO. b) Jump/Kraus operator K_μ as an MPO. c) Vectorized density matrix $|\rho\rangle\rangle$ as an MPS in the doubled Hilbert space (see also Fig. 26). d) The result of the action of K_μ on ρ is a product of three MPOs. e) After vectorization, the action of K_μ on ρ becomes the application of an MPO on an MPS in the doubled Hilbert space.

The operator in parentheses is a superoperator and can be represented as an MPO. This is illustrated in Fig. 27.

A code example that simulates a quantum circuit containing unitary gates (as in Eq. (83)) and noise channels (as in Eq. (85)) is given in code example #13. The code provides an example of a noisy random quantum circuit simulated with MPS. The initial state is the product state $|0\rangle^{\otimes N}$, represented as a mixed state. The circuit (shown in Fig. 28) consists of layers of Haar-random two-qubit gates arranged in a brick-wall pattern, followed by a single-qubit depolarizing channel with error probability p . Because of these error gates, the output state of the circuit is a mixed state represented by a density matrix ρ . In parallel the same circuit is also simulated with $p = 0$. In that case the circuit is unitary and the output $\rho_0 = |\psi\rangle\langle\psi|$ is a pure state. The code compares the ideal output probability distribution (from ρ_0) with the noisy output probability distribution (from ρ) via the linear cross-entropy benchmarking (XEB) estimator which has been popularized by the Google quantum supremacy experiment of 2019 [92]. The XEB estimator is defined as

$$\mathcal{F}_{\text{XEB}} = 2^N \sum_{\sigma} p_{\text{noisy}}(\sigma) p_{\text{ideal}}(\sigma) - 1 \quad (86)$$

where $p_{\text{noisy}}(\sigma) = \text{Tr}[\rho|\sigma\rangle\langle\sigma|]$ and $p_{\text{ideal}}(\sigma) = \text{Tr}[\rho_0|\sigma\rangle\langle\sigma|] = |\langle\psi|\sigma\rangle|^2$ are the probabilities of the computational-basis bitstring $\sigma = (\sigma_1, \dots, \sigma_N)$ in the noisy and ideal output distributions, respectively. $\mathcal{F}_{\text{XEB}}$ is an approximation of the fidelity between the noisy and ideal output states, and it is expected to decay exponentially with the number of qubits N and the depth D of the circuit, as $\mathcal{F}_{\text{XEB}} \sim \exp(-\alpha ND)$, where α is a constant that depends on the error probability p of the noise channel.

Suppose that a quantum device is a noisy implementation of the ideal random circuit. If N is large one cannot compute *all* the probabilities $p_{\text{ideal}}(\sigma)$ of the ideal output distribution, and one cannot run enough repetitions of the device circuit to estimate all the probabilities $p_{\text{noisy}}(\sigma)$. In that case, one can estimate $\mathcal{F}_{\text{XEB}}$ by sampling a subset of bitstrings σ from the noisy distribution:

$$\mathcal{F}_{\text{XEB}} \approx 2^N \frac{1}{M} \sum_{i=1}^M p_{\text{ideal}}(\sigma_i) - 1 \quad (87)$$

where $\sigma_1, \dots, \sigma_M$ are M bitstrings sampled from the noisy distribution.¹⁰ The code example #13 performs the sampling above (variable `NSAMPLES`). It also performs an average over several realizations of the random circuit (variable `NCIRCUITS`). Figure 29 shows the XEB estimator as a function of the circuit depth D for $N = 8$ qubits, in the ideal case $p = 0$ and in the noisy case $p = 0.05$. In the ideal case, the

¹⁰Even with such sampling, the estimation of $\mathcal{F}_{\text{XEB}}$ remains computationally hard because one has to compute $p_{\text{ideal}}(\sigma_i)$ for each sampled bitstring, which requires simulating the ideal circuit. This simulation is done here with MPS. Due to the growth of the entanglement in the ideal state as a function of the circuit depth, the MPS bond dimension needed to accurately represent the ideal state grows exponentially with D . However, this is not an issue for small values of N .

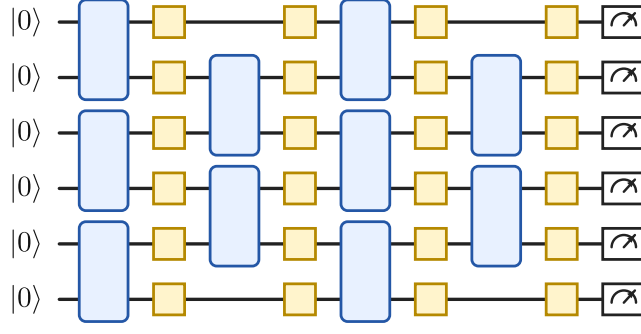


Figure 28: Brick-wall random circuit with Haar-random 2-qubit gates (blue) and single-qubit depolarizing channels (yellow), shown here for $N = 6$ qubits and depth $D = 4$. An MPS-based code that simulates this circuit is given in code example #13.

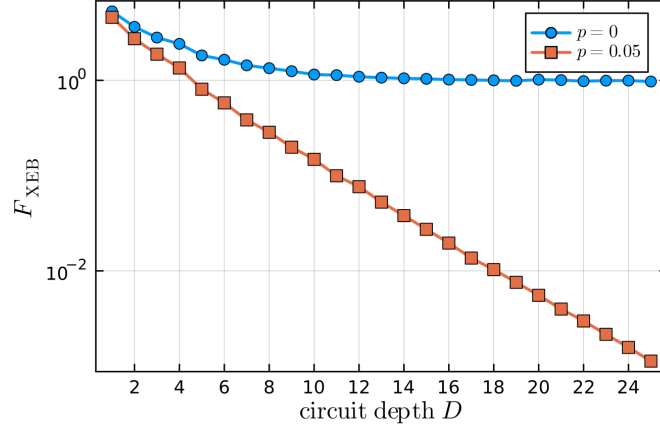


Figure 29: F_{XEB} for random brick-wall circuits with $N = 8$ qubits as a function of the circuit depth D . The two curves correspond to the ideal case, $p = 0$, and to a single-qubit depolarizing channel with error probability $p = 0.05$ applied after each unitary layer (see Fig. 28). A code for such simulation is given in code example #13.

circuit approaches a Haar-random unitary as D increases, and the XEB estimator approaches the value $F_{\text{XEB}} = 1$. Such behavior is expected for a Haar-random state, as a consequence of the Porter-Thomas distribution of the output probabilities for a Haar-random state. In the noisy case, the XEB estimator decays exponentially with D . Note that, by construction, the XEB estimator is equal to $F_{\text{XEB}} = 0$ for completely random outputs with $p_{\text{noisy}}(\sigma) = 2^{-N}$.

Code example 13: Noisy random-circuit simulation

([source online](#)) Noisy random-circuit simulation with `TensorMixedStates`, including sampling of computational-basis bitstrings and evaluation of the linear XEB estimator. Internally, the density matrix ρ is represented as a vectorized MPS, and the quantum channels (unitary gates and noise channels) as MPOs. Haar-random two-qubit gates are generated by QR decomposition of random complex matrices and arranged in a brick-wall circuit. After each unitary layer, the code applies a one-qubit depolarizing channel with error probability `PNOISE`, samples bitstrings from the noisy output state, evaluates their ideal probabilities using the corresponding $p = 0$ circuit, and averages the resulting XEB estimate over several circuit realizations. It also prints the purity, bond dimensions, OSEE of ρ , and the entanglement entropy of the ideal pure state.

[noisy_random_circuit_xeb.jl](#) on GitHub.

4.5 Lindblad dynamics in MPS form

For Markovian open-system dynamics, the density matrix often obeys a Gorini-Kossakowski-Sudarshan-Lindblad (hereafter Lindblad in short) master equation [91]

$$\frac{d\rho}{dt} = \mathcal{L}(\rho), \quad (88)$$

with

$$\mathcal{L}(\rho) = -i[H, \rho] + \sum_{\mu} \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\} \right), \quad (89)$$

where the operators L_{μ} are the jump operators describing the non-unitary part of the dynamics. In the vectorized representation this becomes an ordinary linear differential equation

$$\frac{d}{dt} |\rho\rangle\rangle = \mathbb{L} |\rho\rangle\rangle, \quad (90)$$

where $\mathbb{L}$ is the Lindbladian operator acting in the doubled Hilbert space. Using Eq. (82), the Hamiltonian part is

$$-i[H, \rho] \longrightarrow -i (H \otimes \mathbb{1} - \mathbb{1} \otimes H^T) |\rho\rangle\rangle, \quad (91)$$

and a dissipative term gives

$$L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\} \longrightarrow \left[L_{\mu} \otimes \overline{L_{\mu}} - \frac{1}{2} L_{\mu}^{\dagger} L_{\mu} \otimes \mathbb{1} - \frac{1}{2} \mathbb{1} \otimes (L_{\mu}^{\dagger} L_{\mu})^T \right] |\rho\rangle\rangle. \quad (92)$$

Like any linear operator acting on an MPS, the Hamiltonian Eq. (91), the jump operators Eq. (92) or the whole Lindbladian $\mathbb{L}$ can be represented as MPOs. To construct such MPOs, one can use the finite-state automaton picture described in Sec. 2.6.4. Consider for instance the Hamiltonian part of the Lindbladian. Assume that H contains k terms acting across a given bond. According to the finite-state automaton picture of Sec. 2.6.4, the MPO representation of H (as an operator acting on the original Hilbert space with local physical dimension d) has a bond dimension $k + 2$ across that bond. When acting on a density matrix through $[H, \cdot]$, each term gives rise to *two* terms in the Lindbladian, one acting on the left and one acting on the right of ρ . So, when acting on $|\rho\rangle\rangle$, the Hamiltonian part of the Lindbladian contains $2k$ terms acting across the same bond. The operator $[H, \cdot]$ can thus be represented as an MPO with bond dimension $2k + 2$ and local physical dimension d^2 . Now consider the jump operators L_{μ} (Eq. (89)). Each jump operator gives rise to *three* terms in the Lindbladian, one of the form $L_{\mu} \otimes \overline{L_{\mu}}$ and two of the form $L_{\mu}^{\dagger} L_{\mu} \otimes \mathbb{1}$ and $\mathbb{1} \otimes (L_{\mu}^{\dagger} L_{\mu})^T$. It is common to have dissipative models where the jump operators are local (each L_{μ} acts on a single site).¹¹ In such cases, they do not increase the bond dimension of the Lindbladian MPO compared to the Hamiltonian part (they are included in the blocks $D^{[i]}$ of the MPO of $\mathbb{L}$; see Eq. (35)).

As before, the bond dimension of these MPOs will depend on the range of the interactions. The Lindblad evolution can therefore be viewed as an MPS time-evolution problem in operator space:

$$|\rho(t)\rangle\rangle = e^{t\mathbb{L}} |\rho(0)\rangle\rangle. \quad (93)$$

The TDVP method discussed in Sec. 2.8.2 and the W^I/W^{II} MPO methods discussed in Sec. 2.8.3 can thus be used in this context. They provide different ways to approximate the dissipative evolution. The short code example in code example #14 illustrates how to inspect the MPO bond dimensions for an XX Hamiltonian, for the corresponding Hamiltonian-only Lindbladian, and for local dephasing terms.

Code example 14: The code builds the MPO representation of the Lindbladian for an XX spin- $\frac{1}{2}$ chain, with and without dephasing terms (single site dephasing and two-site dephasing in the last case). The bond dimensions of the resulting MPOs are printed.

[MPO_bond_dim.jl](#) on GitHub.

¹¹For a spin- $\frac{1}{2}$ system, typical examples of local jump operators are the so-called dephasing $L_{\mu} = \sqrt{\gamma} \sigma_i^z$ or amplitude damping or local relaxation $L_{\mu} = \sqrt{\gamma} \sigma_i^{-}$.

Code example 14: MPO bond dimensions for Lindbladians

```

1 using TensorMixedStates, .Qubits
2 using ITensorMPS: linkdims
3 n = 6
4 # MPO bond dimensions for a few examples of operators acting on pure or mixed states
5 h = sum(X(i) * X(i + 1) + Y(i) * Y(i + 1) for i in 1:(n - 1)) # XX spin chain Hamiltonian
6 psi0 = State{Pure}(System(n, Qubit()), "Up")
7 H_mpo = make_mpo(psi0, h) # psi0 needed so that make_mpo (from TensorMixedStates) knows the physical dim. of the
  ↪ tensors
8 println("XX Hamiltonian acting on a pure state: MPO bond dimensions = ", linkdims(H_mpo)) #output: 4
9
10 rho0 = State{Mixed}(System(n, Qubit()), "FullyMixed")
11 L = Evolver(-im * h) # Hamiltonian-only Lindbladian: d(rho)/dt = -i [H, rho]
12 L_mpo = make_mpo(rho0, L)
13 println("XX Hamiltonian acting on a mixed state density matrix: MPO bond dimensions = ", linkdims(L_mpo)) #output: 6
14
15 L = sum(Dissipator(1.0 * Z)(i) for i in 1:n)
16 L_mpo = make_mpo(rho0, L)
17 println("Local dephasing Lindbladian: MPO bond dimensions = ", linkdims(L_mpo)) #output: 2
18
19 L = -im*h + sum(Dissipator(1.0 * Z)(i) for i in 1:n)
20 L_mpo = make_mpo(rho0, L)
21 println("XX Hamiltonian with local dephasing: MPO bond dimensions = ", linkdims(L_mpo)) #output: 6
22
23 # Example with two-site dephasing jump operators.
24 dephase2 = 1.0 * (Z ⊗ Z)
25 dissipators = sum(Dissipator(dephase2)(i, i+1) for i in 1:n-1)
26 L = -im*h + dissipators
27 L_mpo = make_mpo(rho0, L)
28 println("XX Hamiltonian with two-site dephasing: MPO bond dimensions = ", linkdims(L_mpo)) #output: 7

```

Code examples based on the W^{II} method to solve a Lindblad problem in a dissipative spin chain are given in code examples [#15](#) and [#16](#).

Code example 15: Lindblad dynamics of a dissipative XX chain

([source online](#)) Lindblad time evolution of a dissipative XX spin- $\frac{1}{2}$ chain. The model is the boundary-driven XX chain with bulk dephasing. The boundary reservoirs are parametrized by a driving strength Γ and a magnetization bias μ (jump operators $L_{\pm,1} = \sqrt{2\Gamma}(1 \mp \mu)\sigma_i^{\pm}$ and $L_{\pm,N} = \sqrt{2\Gamma}(1 \pm \mu)\sigma_N^{\pm}$ at the left and right boundaries) while dephasing in the bulk is controlled by the rate γ (jump operators of the form $L_i = \sqrt{\gamma}\sigma_i^z$ for $i = 1, \dots, N$). The code builds the Lindbladian as an MPO (`make_mpo`), and, starting from the infinite-temperature density matrix (`rho0 = State{Mixed}(System(n, Qubit()), "FullyMixed")`), evolves the density matrix up to a finite time t , and computes the magnetization profile $\langle \sigma_i^z \rangle$. These results are compared with the exact nonequilibrium steady-state result derived in Ref. [93]. The time evolution is computed by approximating the exponential $\exp(\delta t L)$ of the Lindbladian by a product of MPOs, which is a third-order version of the W^{II} algorithm (`approx_W` and `order=3`) (see Sec. 2.8.3). With this scheme the Trotter error is of order δt^4 .

[lindblad_XX_spin_chain.jl](#) on GitHub.

The high-level interface of `TensorMixedStates` can take care of the time-step loop and of the output files. A compact example is shown in code example [#16](#).

Code example 16: Lindblad time evolution of an XX spin chain with local “decay” jump operators $L_i = \sqrt{\Gamma}\sigma_i^-$ and local “excitation” jump operators $L_i = \sqrt{\Gamma}\sigma_i^+$. The combination of these dissipators drives the system towards the fully mixed steady state $\rho_{\text{ss}} = \mathbb{1}/2^N$. The initial state is the Néel state $|\uparrow\downarrow\uparrow\downarrow \dots\rangle$. The simulation uses the high-level `SimData/runTMS` interface of `TensorMixedStates`, which allows to easily set up the time evolution and the output of multiple observables to files. The code writes the OSEE across the central bond and $\langle \sigma_{N/2}^z \rangle$ as functions of time to files, and makes the corresponding plots. These plots are displayed in Fig. 30.

[lindblad_XX_spin_chain_OSEE.jl](#) on GitHub.

Code example 16: Lindblad dynamics and OSEE in a dissipative XX chain

```

1 using TensorMixedStates, .Qubits
2 using DelimitedFiles, LaTeXStrings, Plots
3
4 N = 10 ; Γ = 0.5 ; tmax = 4.0
5 dir = "lindblad_results" # simulation name, used to create a directory with output files

```

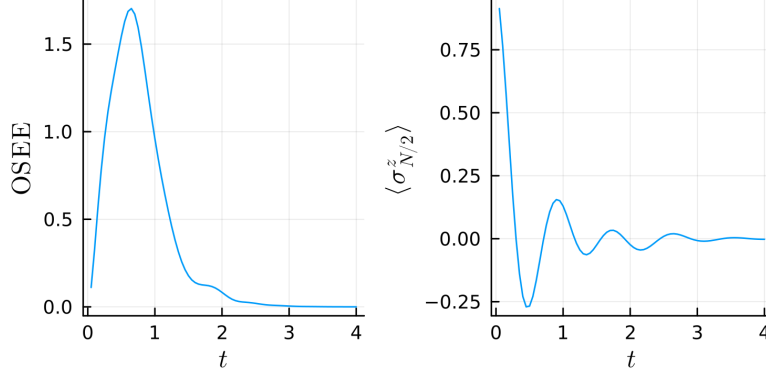


Figure 30: Left: Operator-space entanglement entropy (OSEE) across the central bond plotted as a function of time during the Lindblad evolution of the dissipative XX chain simulated in code example #16. Right: Magnetization $\langle \sigma_{N/2}^z \rangle$ at the central site. In this example the Hamiltonian is the XX spin-1/2 Hamiltonian and the jump operators in the Lindbladian are local decay and excitation operators $L_i = \sqrt{\Gamma} \sigma_i^\pm$. The dissipation drives the system towards the fully mixed state $\rho_{ss} = \mathbb{1}/2^N$.

```

6 limits = Limits(cutoff = 1e-10, maxdim = 20) # MPS truncation parameters
7 initial_state = [isodd(i) ? "Up" : "Dn" for i in 1:N] # Néel spin configuration
8 Lindbladian =
9     -im * sum(X(i) * X(i + 1) + Y(i) * Y(i + 1) for i in 1:(N - 1)) + # XX spin chain Hamiltonian
10     sum(Dissipator(sqrt(Γ) * Sm(i) for i in 1:N) + # Jump operators: lowering operators  $\sigma^-$ , rate  $\Gamma$ 
11     sum(Dissipator(sqrt(Γ) * Sp(i) for i in 1:N) # Jump operators: raising operators  $\sigma^+$ , rate  $\Gamma$ 
12 # 3 observables: center OSEE, center magnetization  $\langle \sigma_{N/2}^z \rangle$ , TraceError= $\text{Tr}[\rho(t)]-1$ 
13 obs = ["osee.dat" => EE(N ÷ 2), "zmid.dat" => Z(N ÷ 2), "TraceError.dat" => TraceError]
14 sim = SimData( # High-level simulation interface of TensorMixedStates
15     name = dir,
16     phases = [
17         CreateState(# Initial state preparation
18             type = Mixed(), system = System(N, Qubit()), state = initial_state),
19         Evolve(algo = ApproxW(order = 4, w = 2), # W{II} scheme
20             limits = limits, duration = tmax, time_step = 0.1,
21             evolver = Lindbladian, measures = obs),
22     ])
23 runTMS(sim; restart = true) # High-level simulation interface of TensorMixedStates
24 function read_data(file)
25     data = readdlm(joinpath(dir, file), '\t')
26     return data[:, 2], data[:, 3]
27 end
28
29 # Plot the results (from the output files)
30 t, osee = read_data("osee.dat")
31 _, zmid = read_data("zmid.dat")
32 p1 = plot(t, osee; label = false, xlabel = L"t", ylabel = L"\mathrm{OSEE}")
33 p2 = plot(t, zmid; label = false, xlabel = L"t", ylabel = L"\langle \sigma_{N/2}^z \rangle")
34 plot(p1, p2; layout = (1, 2), size = (500, 250), dpi = 300, plot_title = L"N = $N, \Gamma = \Gamma")
35 savefig(joinpath(dir, "lindblad_xx_OSEE.png"))

```

The steady state satisfies

$$\mathbb{L}|\rho_{ss}\rangle\rangle = 0. \quad (94)$$

One possible variational strategy is then to search for the state minimizing $\|\mathbb{L}|\rho\rangle\rangle\|^2$, equivalently the ground state of the positive operator $\mathbb{L}^\dagger \mathbb{L}$. If the state $|\rho\rangle\rangle$ is represented as an MPS, one can construct an MPO representation of $\mathbb{L}^\dagger \mathbb{L}$ and use DMRG to find its ground state. This approach is implemented in `TensorMixedStates` (SteadyState phase in the high-level interface).

Acknowledgements

I am grateful to the organizers of the 9th Les Houches School on Computational Physics – Open Quantum Systems: Marco Schiro, Tommaso Roscilde, Cécile Repellin and Markus Holzmann. I thank Jérôme Houdayer for discussions and collaboration on the `TensorMixedStates` library [3]. This work is supported by France 2030 under French National Research Agency grant No. ANR-22-PETQ-0007 (PEPR integrated project EPiQ).

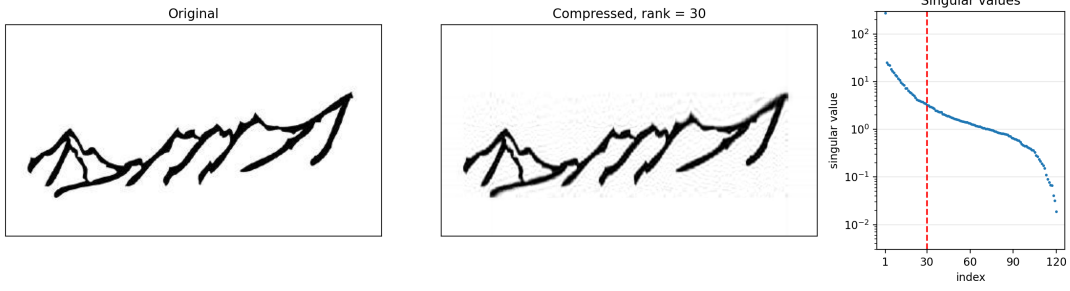


Figure 31: Example of image compression by keeping only the 30 largest singular values of a grayscale image (Les Houches School of Physics logo). The right panel shows the singular values in descending order and the vertical dotted line indicates the cutoff for keeping only the 30 largest singular values (and associated singular vectors). The image size is 394 x 221 pixels and only the first 120 singular values (out of 221) are shown in the right panel.

A Singular-value decomposition (SVD) and QR factorization

A.1 SVD and Schmidt decompositions

The singular value decomposition (SVD) is a fundamental tool in the manipulation of TN states and MPS in particular. It allows us to decompose a matrix into a product of three matrices: $M = U\Lambda V^\dagger$, where U and V are unitary matrices and Λ is a real and positive diagonal matrix containing the singular values. By truncating the small singular values in Λ , one can obtain an approximation of the original matrix M with a smaller rank. This is a key step in many TN algorithms. The faster the singular values decay, the better the approximation obtained by keeping only a few singular values.

A key property of the SVD is that it gives the optimal low-rank approximation of a matrix. If

$$M = U\Lambda V^\dagger \quad (95)$$

is the SVD of M , with singular values ordered as $\Lambda_1 \geq \Lambda_2 \geq \dots$, then the matrix

$$M_r = U_r \Lambda_r V_r^\dagger \quad (96)$$

obtained by keeping only the r largest singular values and the corresponding columns of U and V is the best approximation to M among all matrices M' of rank at most r , in the sense that it minimizes the Frobenius norm

$$\delta^2 = \text{Tr} [(M - M')^\dagger (M - M')]. \quad (97)$$

The minimal error is

$$\delta_{\min}^2 = \sum_{\alpha > r} \Lambda_\alpha^2. \quad (98)$$

This result is known as the Eckart–Young theorem [94].

An image can be considered as a matrix, and Fig. 31 shows an example of image compression by keeping only a small fraction of the largest singular values of a grayscale image. Numerically, the SVD decomposition of a matrix of size $n \times m$ takes a time of the order $\mathcal{O}(nm \cdot \min(n, m))$.

The SVD is also important for the Schmidt decomposition of a bipartite state $|\psi\rangle = \sum_{a,b} M_{ab} |a\rangle \otimes |b\rangle$ defined in the Hilbert space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$ (of dimension $d_A \times d_B$), where $|a\rangle$ and $|b\rangle$ are orthonormal bases of the two subsystems A and B , respectively. The SVD of M allows one to express $|\psi\rangle$ as

$$|\psi\rangle = \sum_i \Lambda_i |u_i\rangle \otimes |v_i\rangle \quad (99)$$

where Λ_α are the singular values (real and positive), and $|u_i\rangle = \sum_{a=1}^{d_A} U_{ai} |a\rangle$ and $|v_i\rangle = \sum_{b=1}^{d_B} \overline{V_{bi}} |b\rangle$ form orthonormal bases of the two subsystems A and B , respectively. The normalization of $|\psi\rangle$ implies that $\sum_i \Lambda_i^2 = 1$.

This decomposition is important in quantum information theory and is closely related to the entanglement properties of quantum states. Indeed, the Schmidt decomposition provides the diagonal forms

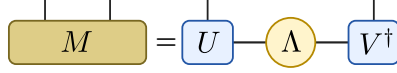


Figure 32: Graphical representation of the singular value decomposition (SVD) of a matrix M into $U\Lambda V^\dagger$. This can also be viewed as the Schmidt decomposition of a bipartite state $|\psi\rangle$ into $|\psi\rangle = \sum_\alpha \Lambda_\alpha |u_\alpha\rangle \otimes |v_\alpha\rangle$, where Λ_α are the singular values, and $|u_\alpha\rangle$ and $|v_\alpha\rangle$ are the left and right singular vectors, respectively.



Figure 33: Graphical representation of the QR decomposition. The matrix Q is unitary and the bottom part is the graphical representation for $Q^\dagger Q = 1$. The matrix R is upper triangular. Why do we choose to write $\bar{Q}$ (and not $Q^\dagger$) in the above graphical representation? The reason is that the information about which index is contracted with which index is already included in the representation: we see that the "vertical" index of the lower tensor is contracted with the same vertical index (turned down to simplify the drawing) of the upper tensor (see also Fig. 3).

of the reduced density matrices of the two subsystems:

$$\rho_A = \text{Tr}_B |\psi\rangle\langle\psi| = \sum_i \Lambda_i^2 |u_i\rangle\langle u_i| \quad (100)$$

$$\rho_B = \text{Tr}_A |\psi\rangle\langle\psi| = \sum_i \Lambda_i^2 |v_i\rangle\langle v_i|. \quad (101)$$

From the expression above we see that the singular values Λ_i are directly related to the entanglement spectrum of the state $|\psi\rangle$ with respect to the A/B partition. In particular, the von Neumann entanglement entropy of the state $|\psi\rangle$ with respect to this partition is given by

$$S_{\text{vN}} = - \sum_i \Lambda_i^2 \log \Lambda_i^2. \quad (102)$$

The graphical representation of the SVD is shown in Fig. 32. The SVD can be used to perform various operations on MPS, such as truncation (compress the state by approximating it with an MPS with lower bond dimension) or to bring the MPS into a canonical form. Concerning truncation, the Eckart–Young theorem guarantees that truncating the SVD spectrum is the optimal way to approximate a bipartite state by a state with a smaller Schmidt rank (and hence a smaller bond dimension).

A.2 QR

The QR decomposition of an $m \times n$ complex rectangular matrix A is a factorization of the form $A = QR$ where Q is an $m \times m$ unitary matrix ($Q^\dagger Q = 1$) and R is an $m \times n$ upper triangular matrix.¹² If we denote by $(a_1, \dots, a_n)$ the columns of A , the QR decomposition can be obtained by applying the Gram-Schmidt orthogonalization procedure to these columns considered as vectors. The resulting orthonormal basis $(q_1, \dots, q_n)$ forms the columns of the matrix Q , while the coefficients of the original columns of A in this new basis are the entries of the matrix R . Since the column a_i is a linear combination of the columns $q_1, \dots, q_i$, the matrix R is upper triangular.

When $m > n$ the matrix A is “tall” (more rows than columns). In such a case it is easy to see that the last $m - n$ rows of R are zero by construction, so that the last $m - n$ columns of Q do not contribute to the product QR . For this reason, one can consider instead the “thin” or “reduced” QR decomposition, where Q is an $m \times n$ matrix with orthonormal columns ($Q^\dagger Q = 1$) and R is an $n \times n$ upper triangular matrix. Such a reduced QR decomposition can be obtained from the standard QR decomposition by keeping only the first n columns of Q and the first n rows of R .

¹²If A is real then Q is orthogonal, $Q^t Q = 1$.

B Injective MPS

B.1 Map from the virtual space to the physical space

The notion of injective MPS was introduced in [95]. Consider the following MPS

$$|\psi\rangle = \sum_{s_1, \dots, s_N} A^{[1] s_1} A^{[2] s_2} \dots A^{[N] s_N} |s_1 \dots s_N\rangle \quad (103)$$

where the matrices have dimension $\chi \times \chi$ (except for the first and last ones, which are $1 \times \chi$ and $\chi \times 1$, respectively) and the local Hilbert space has dimension d . Next, consider a block of consecutive sites extending from site $n > 1$ to site $m < N$. We can define a linear map $\Gamma_{[m,n]} : \text{Mat}_\chi(\mathbb{C}) \rightarrow (\mathbb{C}^d)^{\otimes(m-n+1)}$ from the doubled virtual space (of dimension χ^2) to the physical Hilbert space of dimension $d^{(m-n+1)}$ of the block as follows:

$$\Gamma_{[m,n]} : X \mapsto \sum_{s_m, \dots, s_n} \text{Tr}(X A^{[m] s_m} A^{[m+1] s_{m+1}} \dots A^{[n] s_n}) |s_m \dots s_n\rangle. \quad (104)$$

This can also be written using a more compact notation as

$$\Gamma_{[m,n]}(X) = \sum_{\mathbf{s}} \text{Tr}(X A^{\mathbf{s}}) |\mathbf{s}\rangle \quad (105)$$

where $\mathbf{s} = (s_m, \dots, s_n)$. This map is represented graphically in Fig. 34.

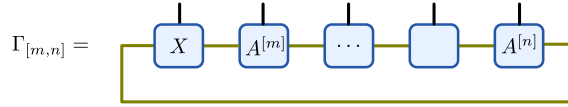


Figure 34: Graphical representation of the map $\Gamma_{[m,n]}$, which contracts a boundary matrix X with the MPS tensors on the block from site m to site n , leaving the physical indices of the block “open” (Eq. (104)).

The block is called injective if the map $\Gamma_{[m,n]}$ is injective. Injectivity implies that the space spanned by the states $\Gamma_{[m,n]}(X)$ has dimension χ^2 . Generic MPS are injective in the sense that non-injective MPS form a set of measure zero in the space of MPS. Injective MPS have several important properties, which make them particularly useful for both analytical and numerical purposes. For instance, they are the unique ground states of local gapped Hamiltonians (the so-called parent Hamiltonians) [50, 95, 96].

If the MPS is such that $A^{[k]} = A$ for all k , then the MPS is said to be injective if there is some length r such that the map $\Gamma_{[1,r]}$ is injective. In the following we will focus, for simplicity, on this case of translationally invariant MPS where all the matrices are identical, $A^{[k]} = A$.

B.2 Space of matrices spanned by products of MPS matrices

Another definition of injectivity is the following: a block of r consecutive sites is injective if the matrix space spanned by the products $A^{s_1} A^{s_2} \dots A^{s_r}$ has dimension χ^2 (and thus spans the full space of $\text{Mat}_\chi(\mathbb{C})$). This second definition is in fact equivalent to the first one, as we show below.

We define $A^{\mathbf{s}} = A^{s_1} \dots A^{s_r}$ with $\mathbf{s} = (s_1, \dots, s_r)$ and consider the block map Eq. (105) $\Gamma_r(X) = \sum_{\mathbf{s}} \text{Tr}(X A^{\mathbf{s}}) |\mathbf{s}\rangle$. We will prove that

$$\Gamma_r \text{ is injective} \iff \text{span}\{A^{\mathbf{s}}\}_{\mathbf{s}} = \text{Mat}_\chi(\mathbb{C}).$$

First, suppose that $\text{span}\{A^{\mathbf{s}}\}_{\mathbf{s}} = \text{Mat}_\chi(\mathbb{C})$. If $\Gamma_r(X) = 0$, then $\text{Tr}(X A^{\mathbf{s}}) = 0 \forall \mathbf{s}$. Since the matrices $A^{\mathbf{s}}$ span $\text{Mat}_\chi(\mathbb{C})$, it follows that $\forall Y \in \text{Mat}_\chi(\mathbb{C})$ we have $\text{Tr}(XY) = 0$. Choosing $Y = X^\dagger$ we get $X = 0$. Hence, Γ_r is injective.

Conversely, suppose that Γ_r is injective. Assume, for contradiction, that $S = \text{span}\{A^{\mathbf{s}}\}_{\mathbf{s}} \neq \text{Mat}_\chi(\mathbb{C})$. We can pick a nonzero matrix X in the orthogonal complement of S (with respect to the Hilbert–Schmidt inner product $\langle A|B \rangle = \text{Tr}(A^\dagger B)$). Being in this orthogonal complement of S means that $\text{Tr}(X A^{\mathbf{s}}) = 0 \forall \mathbf{s}$. Therefore, $\Gamma_r(X) = 0$. This contradicts the injectivity of Γ_r . Hence, $\text{span}\{A^{\mathbf{s}}\}_{\mathbf{s}} = \text{Mat}_\chi(\mathbb{C})$.

B.3 Parent Hamiltonian of an injective MPS

Consider a translation-invariant MPS on a chain of N sites, with bond dimension χ :

$$|\psi\rangle = \sum_{s_1, \dots, s_N} \text{Tr}(A^{s_1} A^{s_2} \dots A^{s_N}) |s_1 s_2 \dots s_N\rangle,$$

where each physical index s takes values in $\{1, \dots, d\}$, and each matrix A^s has dimension $\chi \times \chi$. We will construct a local Hamiltonian

$$H = \sum_n h_n$$

such that $|\psi\rangle$ is a ground state.

Take a block of r consecutive sites and consider, as in appendix B.1, the linear map

$$\Gamma_r : X \in \text{Mat}_\chi(\mathbb{C}) \mapsto \sum_{s_1, \dots, s_r} \text{Tr}(X A^{s_1} \dots A^{s_r}) |s_1, \dots, s_r\rangle.$$

Equivalently, one can write the block states

$$|\phi_{\alpha\beta}^{[r]}\rangle = \sum_{s_1, \dots, s_r} (A^{s_1} \dots A^{s_r})_{\alpha\beta} |s_1, \dots, s_r\rangle,$$

where $\alpha, \beta = 1, \dots, \chi$. These χ^2 states span a subspace

$$\text{Im}(\Gamma_r) = \text{span} \left\{ |\phi_{\alpha\beta}^{[r]}\rangle \right\}.$$

This is the set of all possible physical states that can appear on a block of r sites when the virtual boundary indices of that block are left open.

For an injective MPS, for sufficiently large r , the map Γ_r is injective. Therefore $\dim \text{Im}(\Gamma_r) = \chi^2$. Since the full block Hilbert space has dimension d^r , for large enough r one has $\chi^2 < d^r$, so $\text{Im}(\Gamma_r)$ is a proper subspace of the block Hilbert space.

We want to construct a Hamiltonian which penalizes local states that do not belong to $\text{Im}(\Gamma_r)$. For this we introduce the projector Π_r onto $\text{Im}(\Gamma_r)$ and define

$$h = \mathbb{I} - \Pi_r.$$

h is a positive-semidefinite operator acting on r consecutive sites, and

$$h|\varphi\rangle = 0$$

if and only if

$$|\varphi\rangle \in \text{Im}(\Gamma_r).$$

The parent Hamiltonian is then defined as the sum of these local projectors:

$$H = \sum_{n=1}^N h_{n, n+1, \dots, n+r-1},$$

where $h_{n, n+1, \dots, n+r-1}$ acts as h on the block $(n, \dots, n+r-1)$. For periodic boundary conditions, the site labels are understood modulo N .

Now consider the full MPS and isolate a block of r sites. The rest of the chain supplies some effective virtual boundary matrix X and the wavefunction restricted to that block is always a vector in $\text{Im}(\Gamma_r)$ and, as illustrated in Fig. 35,

$$h_{n, n+1, \dots, n+r-1} |\psi\rangle = 0$$

for every n . Hence

$$H|\psi\rangle = 0.$$

Since each term h_n is positive semidefinite,

$$H \geq 0,$$

so $|\psi\rangle$ is a ground state with ground-state energy zero. This type of Hamiltonian is called frustration-free:

$$h_n |\psi\rangle = 0 \quad \text{for all } n.$$

For an injective MPS, the corresponding parent Hamiltonian has $|\psi\rangle$ as its unique ground state on a sufficiently large periodic chain, and the spectral gap remains open in the thermodynamic limit [95, 96] (proof not given here).

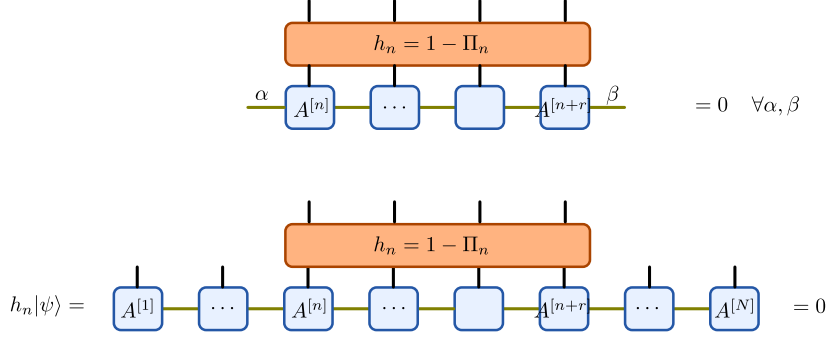


Figure 35: Parent Hamiltonian of an injective MPS. Each local term $h_n = \mathbb{I} - \Pi_n$ penalizes the orthogonal complement of the image of the block map on r consecutive sites.

C MPS representation of the AKLT state

The construction of the AKLT state given in Sec. 2.3.4 can be used to construct an explicit MPS representation. To do so, one associates the virtual spin- $\frac{1}{2}$ degrees of freedom to *bond* degrees of freedom in the MPS, as explained below. At each site j , we have two virtual spin- $\frac{1}{2}$'s, denoted by L_j and R_j . The right virtual spin of site j is paired into a singlet with the left virtual spin of site $j + 1$ (Fig. 36)

$$|\omega\rangle_{R_j, L_{j+1}} = \frac{1}{\sqrt{2}} (|\uparrow\rangle_{R_j} |\downarrow\rangle_{L_{j+1}} - |\downarrow\rangle_{R_j} |\uparrow\rangle_{L_{j+1}}).$$

At each site, the two virtual spins L_j and R_j need to be projected onto the spin-1 subspace and a natural basis for this subspace is

$$|+\rangle = |\uparrow\uparrow\rangle, \quad |0\rangle = \frac{1}{\sqrt{2}} (|\uparrow\downarrow\rangle + |\downarrow\uparrow\rangle), \quad |-\rangle = |\downarrow\downarrow\rangle.$$

The projection operator onto the $S = 1$ subspace reads

$$P = |+\rangle\langle\uparrow\uparrow| + \frac{|0\rangle}{\sqrt{2}} (\langle\uparrow\downarrow| + \langle\downarrow\uparrow|) + |-\rangle\langle\downarrow\downarrow|.$$

We now derive the MPS matrices. The basis states of the virtual spin-1/2 are denoted by $a, b \in \{\uparrow, \downarrow\}$. Thus, from the above expression for the projector we get its matrix elements

$$P_{ab}^s = \langle s | P | a, b \rangle, \quad s \in \{+, 0, -\}.$$

Thus,

$$P_{ab}^+ = \delta_{a,\uparrow} \delta_{b,\uparrow},$$

$$P_{ab}^0 = \frac{1}{\sqrt{2}} (\delta_{a,\uparrow} \delta_{b,\downarrow} + \delta_{a,\downarrow} \delta_{b,\uparrow}),$$

and

$$P_{ab}^- = \delta_{a,\downarrow} \delta_{b,\downarrow}.$$

To construct the MPS tensors we have to take into account the fact that in each bond the virtual spins $\frac{1}{2}$ are paired into a singlet. For this we introduce an MPS bond index connecting site j and $j + 1$ which takes two values (bond dimension $\chi = 2$). We choose the value of this index to be the state of the virtual spin L_{j+1} on the *right* side of the bond. If R_j is in the basis state b , then L_{j+1} is in state $c = \bar{b}$. Including the sign of the singlet wave function, the wave function of the singlet is simply $|\omega\rangle_{R_j, L_{j+1}} = \sum_{b,c=\uparrow,\downarrow} \epsilon_{bc} |b\rangle_{R_j} |c\rangle_{L_{j+1}}$, where ϵ_{bc} is the antisymmetric tensor defined by

$$\epsilon = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix},$$

The matrix of the MPS representation can then be chosen to be

$$A_{ac}^s = \sum_{b=\uparrow,\downarrow} P_{ab}^s \epsilon_{bc}. \quad (106)$$

$s \in \{+, 0, -\}$ is the physical index in the MPS, a is at the same time the MPS index of the bond $(j-1, j)$ and the state of the virtual spin L_j , and c is the MPS index of the bond $(j, j+1)$ and the state of the virtual spin L_{j+1} . The index b is the state of the virtual spin R_j , which is related to c by ϵ . This index convention is illustrated in Fig. 36.

The final step is to explicitly compute the three matrices. For $s = +$, only $P_{\uparrow\uparrow}^+ = 1$ is nonzero. Therefore

$$A_{ac}^+ = \sum_b P_{ab}^+ \epsilon_{bc} = \delta_{a,\uparrow} \epsilon_{\uparrow c},$$

so that

$$A^+ = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \sigma^+.$$

For $s = 0$, the only nonzero coefficients are

$$P_{\uparrow\downarrow}^0 = P_{\downarrow\uparrow}^0 = \frac{1}{\sqrt{2}}.$$

Hence,

$$A_{ac}^0 = \frac{1}{\sqrt{2}} (\delta_{a,\uparrow} \epsilon_{\downarrow c} + \delta_{a,\downarrow} \epsilon_{\uparrow c}),$$

which gives

$$A^0 = \frac{1}{2} \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} = -\frac{1}{2} \sigma^z.$$

Finally, for $s = -$, only $P_{\downarrow\downarrow}^- = 1$ is nonzero, and therefore

$$A_{ac}^- = \delta_{a,\downarrow} \epsilon_{\downarrow c}$$

and

$$A^- = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 0 \\ -1 & 0 \end{pmatrix} = -\frac{1}{\sqrt{2}} \sigma^-.$$

Equivalently, one can group the physical spin-1 states into the entries of a state-valued matrix

$$\mathcal{A} = \sum_{s \in \{+, 0, -\}} A^s |s\rangle = \begin{pmatrix} -\frac{1}{2} |0\rangle & \frac{1}{\sqrt{2}} |+\rangle \\ -\frac{1}{\sqrt{2}} |-\rangle & \frac{1}{2} |0\rangle \end{pmatrix}. \quad (107)$$

There is no remaining physical index on $\mathcal{A}$; each matrix entry is now a ket in the local spin-1 Hilbert space. The AKLT state on a periodic chain is obtained by taking the virtual trace of the product of N copies of this state-valued matrix, with tensor products of the local kets understood in the multiplication.

These matrices give an MPS representation of the AKLT state. Note that other normalizations and other gauges can be found in the literature. On a periodic chain the AKLT state can then be written as

$$|\Psi_{\text{AKLT}}\rangle = \sum_{s_1, \dots, s_N \in \{+, 0, -\}} \text{Tr} [A^{s_1} A^{s_2} \dots A^{s_N}] |s_1 s_2 \dots s_N\rangle.$$

For open boundary conditions, the trace is replaced by boundary vectors,

$$|\Psi_{\text{AKLT}}(\mathbf{v}_L, \mathbf{v}_R)\rangle = \sum_{s_1, \dots, s_N \in \{+, 0, -\}} \mathbf{v}_L^\dagger A^{s_1} A^{s_2} \dots A^{s_N} \mathbf{v}_R |s_1 s_2 \dots s_N\rangle.$$

These additional degrees of freedom in the choice of the boundary vectors $\mathbf{v}_L$ and $\mathbf{v}_R$ are related to the edge states of the AKLT model in the presence of open boundary conditions.

References

- [1] G. Misguich. *Introduction to matrix-product states and tensor networks – Julia code examples*. URL: <https://github.com/gregoire-misguich/Introduction-to-matrix-product-states-and-tensor-networks>.
- [2] M. Fishman, S. White, and E. Stoudenmire. *The ITensor Software Library for Tensor Network Calculations*. *SciPost Phys. Codebases* (2022), 004. DOI: [10.21468/SciPostPhysCodeb.4](https://doi.org/10.21468/SciPostPhysCodeb.4).

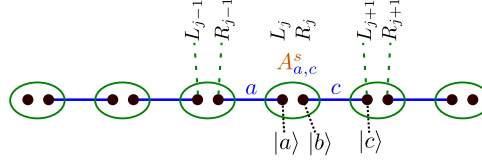


Figure 36: MPS representation of the AKLT valence-bond solid state (see also Fig. 4). The figure represents the index convention used in Eq. (106).

- [3] J. Houdayer and G. Misguich. *TensorMixedStates: A Julia Library for Simulating Pure and Mixed Quantum States Using Matrix Product States*. *SciPost Phys. Codebases* (2026), 072. DOI: [10.21468/SciPostPhysCodeb.72](https://doi.org/10.21468/SciPostPhysCodeb.72). URL: <https://github.com/jerhoud/TensorMixedStates.jl>.
- [4] P. Corboz, R. Orús, B. Bauer, and G. Vidal. *Simulation of Strongly Correlated Fermions in Two Spatial Dimensions with Fermionic Projected Entangled-Pair States*. *Phys. Rev. B* 81.16 (2010), 165104. DOI: [10.1103/PhysRevB.81.165104](https://doi.org/10.1103/PhysRevB.81.165104).
- [5] G. Vidal. *Classical Simulation of Infinite-Size Quantum Lattice Systems in One Spatial Dimension*. *Phys. Rev. Lett.* 98.7 (2007), 070201. DOI: [10.1103/PhysRevLett.98.070201](https://doi.org/10.1103/PhysRevLett.98.070201).
- [6] I. P. McCulloch. *Infinite Size Density Matrix Renormalization Group, Revisited*. 2008. DOI: [10.48550/arXiv.0804.2509](https://doi.org/10.48550/arXiv.0804.2509). arXiv: [0804.2509](https://arxiv.org/abs/0804.2509) [cond-mat].
- [7] J. Jordan, R. Orús, G. Vidal, F. Verstraete, and J. I. Cirac. *Classical Simulation of Infinite-Size Quantum Lattice Systems in Two Spatial Dimensions*. *Phys. Rev. Lett.* 101.25 (2008), 250602. DOI: [10.1103/PhysRevLett.101.250602](https://doi.org/10.1103/PhysRevLett.101.250602).
- [8] F. Verstraete and J. I. Cirac. *Renormalization Algorithms for Quantum-Many Body Systems in Two and Higher Dimensions*. 2004. arXiv: [cond-mat/0407066](https://arxiv.org/abs/cond-mat/0407066).
- [9] A. E. Feiguin, E. Rezayi, C. Nayak, and S. Das Sarma. *Density Matrix Renormalization Group Study of Incompressible Fractional Quantum Hall States*. *Phys. Rev. Lett.* 100.16 (2008), 166803. DOI: [10.1103/PhysRevLett.100.166803](https://doi.org/10.1103/PhysRevLett.100.166803).
- [10] M. P. Zaletel and R. S. K. Mong. *Exact Matrix Product States for Quantum Hall Wave Functions*. *Phys. Rev. B* 86.24 (2012), 245305. DOI: [10.1103/PhysRevB.86.245305](https://doi.org/10.1103/PhysRevB.86.245305).
- [11] M. P. Zaletel, R. S. K. Mong, and F. Pollmann. *Topological Characterization of Fractional Quantum Hall Ground States from Microscopic Hamiltonians*. *Phys. Rev. Lett.* 110.23 (2013), 236801. DOI: [10.1103/PhysRevLett.110.236801](https://doi.org/10.1103/PhysRevLett.110.236801).
- [12] J. Eisert, M. Cramer, and M. B. Plenio. *Colloquium: Area Laws for the Entanglement Entropy*. *Rev. Mod. Phys.* 82.1 (2010), 277–306. DOI: [10.1103/RevModPhys.82.277](https://doi.org/10.1103/RevModPhys.82.277).
- [13] S. Östlund and S. Rommer. *Thermodynamic Limit of Density Matrix Renormalization*. *Phys. Rev. Lett.* 75.19 (1995), 3537–3540. DOI: [10.1103/PhysRevLett.75.3537](https://doi.org/10.1103/PhysRevLett.75.3537).
- [14] I. V. Oseledets. *Tensor-Train Decomposition*. *SIAM Journal on Scientific Computing* 33 (2011).
- [15] E. Stoudenmire and S. R. White. *Studying Two-Dimensional Systems with the Density Matrix Renormalization Group*. *Annu. Rev. Condens. Matter Phys.* 3.1 (2012), 111–128. DOI: [10.1146/annurev-conmatphys-020911-125018](https://doi.org/10.1146/annurev-conmatphys-020911-125018).
- [16] T. Nishino, Y. Hieida, K. Okunishi, N. Maeshima, Y. Akutsu, and A. Gendiar. *Two-Dimensional Tensor Product Variational Formulation*. *Progress of Theoretical Physics* 105.3 (2001), 409–417. DOI: [10.1143/PTP.105.409](https://doi.org/10.1143/PTP.105.409).
- [17] G. Vidal. *Entanglement Renormalization*. *Phys. Rev. Lett.* 99.22 (2007), 220405. DOI: [10.1103/PhysRevLett.99.220405](https://doi.org/10.1103/PhysRevLett.99.220405).
- [18] G. Vidal. *Class of Quantum Many-Body States That Can Be Efficiently Simulated*. *Phys. Rev. Lett.* 101.11 (2008), 110501. DOI: [10.1103/PhysRevLett.101.110501](https://doi.org/10.1103/PhysRevLett.101.110501).
- [19] G. Evenbly and G. Vidal. *Algorithms for Entanglement Renormalization*. *Phys. Rev. B* 79.14 (2009), 144108. DOI: [10.1103/PhysRevB.79.144108](https://doi.org/10.1103/PhysRevB.79.144108).

- [20] S. R. White. *Density Matrix Formulation for Quantum Renormalization Groups*. *Phys. Rev. Lett.* 69.19 (1992), 2863–2866. DOI: [10.1103/PhysRevLett.69.2863](https://doi.org/10.1103/PhysRevLett.69.2863).
- [21] S. R. White. *Density-Matrix Algorithms for Quantum Renormalization Groups*. *Phys. Rev. B* 48.14 (1993), 10345–10356. DOI: [10.1103/PhysRevB.48.10345](https://doi.org/10.1103/PhysRevB.48.10345).
- [22] U. Schollwöck. *The Density-Matrix Renormalization Group in the Age of Matrix Product States*. *Annals of Phys.* January 2011 Special Issue 326.1 (2011), 96–192. DOI: [10.1016/j.aop.2010.09.012](https://doi.org/10.1016/j.aop.2010.09.012).
- [23] S. R. White and A. E. Feiguin. *Real-Time Evolution Using the Density Matrix Renormalization Group*. *Phys. Rev. Lett.* 93.7 (2004), 076401. DOI: [10.1103/PhysRevLett.93.076401](https://doi.org/10.1103/PhysRevLett.93.076401).
- [24] A. J. Daley, C. Kollath, U. Schollwöck, and G. Vidal. *Time-Dependent Density-Matrix Renormalization-Group Using Adaptive Effective Hilbert Spaces*. *J. Stat. Mech. Theory Exp.* 2004.04 (2004), P04005. DOI: [10.1088/1742-5468/2004/04/P04005](https://doi.org/10.1088/1742-5468/2004/04/P04005).
- [25] D. Gobert, C. Kollath, U. Schollwöck, and G. Schütz. *Real-Time Dynamics in Spin-1/2 Chains with Adaptive Time-Dependent Density Matrix Renormalization Group*. *Phys. Rev. E* 71.3 (2005), 036102. DOI: [10.1103/PhysRevE.71.036102](https://doi.org/10.1103/PhysRevE.71.036102).
- [26] G. Vidal. *Efficient Classical Simulation of Slightly Entangled Quantum Computations*. *Phys. Rev. Lett.* 91.14 (2003), 147902. DOI: [10.1103/PhysRevLett.91.147902](https://doi.org/10.1103/PhysRevLett.91.147902).
- [27] G. Vidal. *Efficient Simulation of One-Dimensional Quantum Many-Body Systems*. *Phys. Rev. Lett.* 93.4 (2004), 040502. DOI: [10.1103/PhysRevLett.93.040502](https://doi.org/10.1103/PhysRevLett.93.040502).
- [28] J. Haegeman, J. I. Cirac, T. J. Osborne, I. Pižorn, H. Verschelde, and F. Verstraete. *Time-Dependent Variational Principle for Quantum Lattices*. *Phys. Rev. Lett.* 107.7 (2011), 070601. DOI: [10.1103/PhysRevLett.107.070601](https://doi.org/10.1103/PhysRevLett.107.070601).
- [29] J. Haegeman, C. Lubich, I. Oseledets, B. Vandereycken, and F. Verstraete. *Unifying Time Evolution and Optimization with Matrix Product States*. *Phys. Rev. B* 94.16 (2016), 165116. DOI: [10.1103/PhysRevB.94.165116](https://doi.org/10.1103/PhysRevB.94.165116).
- [30] M. P. Zaletel, R. S. K. Mong, C. Karrasch, J. E. Moore, and F. Pollmann. *Time-Evolving a Matrix Product State with Long-Ranged Interactions*. *Phys. Rev. B* 91.16 (2015), 165112. DOI: [10.1103/PhysRevB.91.165112](https://doi.org/10.1103/PhysRevB.91.165112).
- [31] S. Cerezo-Roquebrún, A. Bou-Comas, J. T. Schneider, E. López, L. Tagliacozzo, and S. Carignano. *Spatio-Temporal Tensor-Network Approaches to out-of-Equilibrium Dynamics Bridging Open and Closed Systems*. 2025. DOI: [10.48550/arXiv.2502.20214](https://doi.org/10.48550/arXiv.2502.20214). arXiv: [2502.20214 \[quant-ph\]](https://arxiv.org/abs/2502.20214).
- [32] S. Carignano, G. Lami, J. D. Nardis, and L. Tagliacozzo. *Overcoming the Entanglement Barrier with Sampled Tensor Networks*. 2025. DOI: [10.48550/arXiv.2505.09714](https://doi.org/10.48550/arXiv.2505.09714). arXiv: [2505.09714 \[quant-ph\]](https://arxiv.org/abs/2505.09714).
- [33] F. Verstraete, J. J. García-Ripoll, and J. I. Cirac. *Matrix Product Density Operators: Simulation of Finite-Temperature and Dissipative Systems*. *Phys. Rev. Lett.* 93.20 (2004), 207204. DOI: [10.1103/PhysRevLett.93.207204](https://doi.org/10.1103/PhysRevLett.93.207204).
- [34] M. Zwolak and G. Vidal. *Mixed-State Dynamics in One-Dimensional Quantum Lattice Systems: A Time-Dependent Superoperator Renormalization Algorithm*. *Phys. Rev. Lett.* 93.20 (2004), 207205. DOI: [10.1103/PhysRevLett.93.207205](https://doi.org/10.1103/PhysRevLett.93.207205).
- [35] Y. Zhou, E. M. Stoudenmire, and X. Waintal. *What Limits the Simulation of Quantum Computers?:* *Phys. Rev. X* 10.4 (2020), 041038. DOI: [10.1103/PhysRevX.10.041038](https://doi.org/10.1103/PhysRevX.10.041038).
- [36] T. Ayrál, T. Louvet, Y. Zhou, C. Lambert, E. M. Stoudenmire, and X. Waintal. *Density-Matrix Renormalization Group Algorithm for Simulating Quantum Circuits with a Finite Fidelity*. *PRX Quantum* 4.2 (2023), 020304. DOI: [10.1103/PRXQuantum.4.020304](https://doi.org/10.1103/PRXQuantum.4.020304).
- [37] X. Chen, Z.-C. Gu, and X.-G. Wen. *Classification of Gapped Symmetric Phases in One-Dimensional Spin Systems*. *Phys. Rev. B* 83.3 (2011), 035107. DOI: [10.1103/PhysRevB.83.035107](https://doi.org/10.1103/PhysRevB.83.035107).
- [38] X. Chen, Z.-C. Gu, and X.-G. Wen. *Complete Classification of One-Dimensional Gapped Quantum Phases in Interacting Spin Systems*. *Phys. Rev. B* 84.23 (2011), 235128. DOI: [10.1103/PhysRevB.84.235128](https://doi.org/10.1103/PhysRevB.84.235128).
- [39] N. Schuch, D. Pérez-García, and I. Cirac. *Classifying Quantum Phases Using Matrix Product States and Projected Entangled Pair States*. *Phys. Rev. B* 84.16 (2011), 165139. DOI: [10.1103/PhysRevB.84.165139](https://doi.org/10.1103/PhysRevB.84.165139).

- [40] B. Derrida, M. R. Evans, V. Hakim, and V. Pasquier. *Exact Solution of a 1D Asymmetric Exclusion Model Using a Matrix Formulation*. *Journal of Physics A: Mathematical and General* 26.7 (1993), 1493. DOI: [10.1088/0305-4470/26/7/011](https://doi.org/10.1088/0305-4470/26/7/011).
- [41] K. Mallick. *Some Exact Results for the Exclusion Process*. *J. Stat. Mech. Theory Exp.* 2011.01 (2011), P01024. DOI: [10.1088/1742-5468/2011/01/P01024](https://doi.org/10.1088/1742-5468/2011/01/P01024).
- [42] T. Nishino. *Density Matrix Renormalization Group Method for 2D Classical Models*. *Journal of the Physical Society of Japan* 64.10 (1995), 3598–3601. DOI: [10.1143/JPSJ.64.3598](https://doi.org/10.1143/JPSJ.64.3598).
- [43] R. Orús and G. Vidal. *Infinite Time-Evolving Block Decimation Algorithm beyond Unitary Evolution*. *Phys. Rev. B* 78.15 (2008), 155117. DOI: [10.1103/PhysRevB.78.155117](https://doi.org/10.1103/PhysRevB.78.155117).
- [44] Y. Núñez Fernández, M. K. Ritter, M. Jeannin, J.-W. Li, T. Kloss, T. Louvet, S. Terasaki, O. Parcollet, J. von Delft, H. Shinaoka, et al. *Learning Tensor Networks with Tensor Cross Interpolation: New Algorithms and Libraries*. *SciPost Phys.* 18.3 (2025), 104. DOI: [10.21468/SciPostPhys.18.3.104](https://doi.org/10.21468/SciPostPhys.18.3.104).
- [45] M. K. Ritter, Y. Núñez Fernández, M. Wallerberger, J. von Delft, H. Shinaoka, and X. Waintal. *Quantics Tensor Cross Interpolation for High-Resolution Parsimonious Representations of Multivariate Functions*. *Phys. Rev. Lett.* 132.5 (2024), 056501. DOI: [10.1103/PhysRevLett.132.056501](https://doi.org/10.1103/PhysRevLett.132.056501).
- [46] X. Waintal, C.-H. Huang, and C. W. Groth. *Who Can Compete with Quantum Computers? Lecture Notes on Quantum Inspired Tensor Networks Computational Techniques*. 2026. DOI: [10.48550/arXiv.2601.03035](https://doi.org/10.48550/arXiv.2601.03035). arXiv: [2601.03035](https://arxiv.org/abs/2601.03035) [quant-ph].
- [47] R. Orús. *A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States*. *Annals of Physics* 349 (2014), 117–158. DOI: [10.1016/j.aop.2014.06.013](https://doi.org/10.1016/j.aop.2014.06.013).
- [48] J. C. Bridgeman and C. T. Chubb. *Hand-Waving and Interpretive Dance: An Introductory Course on Tensor Networks*. *J. Phys. A Math. Theor.* 50.22 (2017), 223001. DOI: [10.1088/1751-8121/aa6dc3](https://doi.org/10.1088/1751-8121/aa6dc3).
- [49] P. Silvi, F. Tschirsich, M. Gerster, J. Jünemann, D. Jaschke, M. Rizzi, and S. Montangero. *The Tensor Networks Anthology: Simulation Techniques for Many-Body Quantum Lattice Systems*. *SciPost Phys. Lect.* (2019), 008. DOI: [10.21468/SciPostPhysLectNotes.8](https://doi.org/10.21468/SciPostPhysLectNotes.8).
- [50] J. I. Cirac, D. Pérez-García, N. Schuch, and F. Verstraete. *Matrix Product States and Projected Entangled Pair States: Concepts, Symmetries, Theorems*. *Rev. Mod. Phys.* 93.4 (2021), 045003. DOI: [10.1103/RevModPhys.93.045003](https://doi.org/10.1103/RevModPhys.93.045003).
- [51] M. C. Bañuls. *Tensor Network Algorithms: A Route Map*. *Annu. Rev. Condens. Matter Phys.* 14 (2023), 173–191. DOI: [10.1146/annurev-conmatphys-040721-022705](https://doi.org/10.1146/annurev-conmatphys-040721-022705).
- [52] B. V.-D. Cuiper, W. Wiesiolek, and F. Verstraete. *Les Houches Lecture Notes on Tensor Networks*. <https://arxiv.org/abs/2512.24390v2>. 2025.
- [53] S. Paeckel, T. Köhler, A. Swoboda, S. R. Manmana, U. Schollwöck, and C. Hubig. *Time-Evolution Methods for Matrix-Product States*. *Annals of Physics* 411 (2019), 167998. DOI: [10.1016/j.aop.2019.167998](https://doi.org/10.1016/j.aop.2019.167998).
- [54] J. von Delft. *Lecture Course on Tensor Networks, Ludwig Maximilian University Munich (2025)*. URL: <https://www.theorie.physik.uni-muenchen.de/lsvondelft/teams-groups/teaching/tensornetworks1>.
- [55] J. Gray. *quimb: a python library for quantum information and many-body calculations*. *Journal of Open Source Software* 3.29 (2018), 819. DOI: [10.21105/joss.00819](https://doi.org/10.21105/joss.00819).
- [56] J. Hauschild and F. Pollmann. *Efficient Numerical Simulations with Tensor Networks: Tensor Network Python (TeNPy)*. *SciPost Phys. Lect.* (2018), 005. DOI: [10.21468/SciPostPhysLectNotes.5](https://doi.org/10.21468/SciPostPhysLectNotes.5).
- [57] *TenPy: Tensor Network Python Library*. URL: <https://tenpy.readthedocs.io>.
- [58] *tensornetwork.org*. URL: <https://tensornetwork.org/>.
- [59] G. Evenbly. *tensor.net*. URL: <https://tensor.net/>.
- [60] I. Affleck, T. Kennedy, E. H. Lieb, and H. Tasaki. *Rigorous Results on Valence-Bond Ground States in Antiferromagnets*. *Phys. Rev. Lett.* 59.7 (1987), 799–802. DOI: [10.1103/PhysRevLett.59.799](https://doi.org/10.1103/PhysRevLett.59.799).
- [61] L. Tagliacozzo, G. Evenbly, and G. Vidal. *Simulation of Two-Dimensional Quantum Systems Using a Tree Tensor Network That Exploits the Entropic Area Law*. *Phys. Rev. B* 80.23 (2009), 235127. DOI: [10.1103/PhysRevB.80.235127](https://doi.org/10.1103/PhysRevB.80.235127).

- [62] N. Schuch, M. M. Wolf, F. Verstraete, and J. I. Cirac. *Computational Complexity of Projected Entangled Pair States*. *Phys. Rev. Lett.* 98.14 (2007), 140506. DOI: [10.1103/PhysRevLett.98.140506](https://doi.org/10.1103/PhysRevLett.98.140506).
- [63] G. M. Crosswhite and D. Bacon. *Finite Automata for Caching in Matrix Product Algorithms*. *Phys. Rev. A* 78.1 (2008), 012356. DOI: [10.1103/PhysRevA.78.012356](https://doi.org/10.1103/PhysRevA.78.012356).
- [64] E. M. Stoudenmire and S. R. White. *Minimally Entangled Typical Thermal State Algorithms*. *New Journal of Physics* 12.5 (2010), 055026. DOI: [10.1088/1367-2630/12/5/055026](https://doi.org/10.1088/1367-2630/12/5/055026).
- [65] P. Calabrese and J. Cardy. *Entanglement Entropy and Quantum Field Theory*. *J. Stat. Mech. Theory Exp.* 2004.06 (2004), P06002. DOI: [10.1088/1742-5468/2004/06/P06002](https://doi.org/10.1088/1742-5468/2004/06/P06002).
- [66] T. Antal, Z. Rácz, A. Rákos, and G. Schütz. *Transport in the XX Chain at Zero Temperature: Emergence of Flat Magnetization Profiles*. *Phys. Rev. E* 59.5 (1999), 4912–4918. DOI: [10.1103/PhysRevE.59.4912](https://doi.org/10.1103/PhysRevE.59.4912).
- [67] K. Bidzhiev and G. Misguich. *Out-of-equilibrium dynamics in a quantum impurity model: Numerics for particle transport and entanglement entropy*. *Phys. Rev. B* 96.19 (2017), 195117. DOI: [10.1103/PhysRevB.96.195117](https://doi.org/10.1103/PhysRevB.96.195117).
- [68] D. S. Rokhsar and S. A. Kivelson. *Superconductivity and the Quantum Hard-Core Dimer Gas*. *Phys. Rev. Lett.* 61.20 (1988), 2376–2379. DOI: [10.1103/PhysRevLett.61.2376](https://doi.org/10.1103/PhysRevLett.61.2376).
- [69] J.-M. Stéphan, S. Furukawa, G. Misguich, and V. Pasquier. *Shannon and Entanglement Entropies of One- and Two-Dimensional Critical Wave Functions*. *Phys. Rev. B* 80.18 (2009), 184421. DOI: [10.1103/PhysRevB.80.184421](https://doi.org/10.1103/PhysRevB.80.184421).
- [70] F. Verstraete, M. M. Wolf, D. Perez-Garcia, and J. I. Cirac. *Criticality, the Area Law, and the Computational Power of Projected Entangled Pair States*. *Phys. Rev. Lett.* 96.22 (2006), 220601. DOI: [10.1103/PhysRevLett.96.220601](https://doi.org/10.1103/PhysRevLett.96.220601).
- [71] A. Y. Kitaev. *Fault-Tolerant Quantum Computation by Anyons*. *Annals of Physics* 303.1 (2003), 2–30. DOI: [10.1016/S0003-4916\(02\)00018-0](https://doi.org/10.1016/S0003-4916(02)00018-0).
- [72] I. Affleck, T. Kennedy, E. H. Lieb, and H. Tasaki. *Valence Bond Ground States in Isotropic Quantum Antiferromagnets*. *Communications in Mathematical Physics* 115.3 (1988), 477–528. DOI: [10.1007/BF01218021](https://doi.org/10.1007/BF01218021).
- [73] P. W. Anderson. *Resonating Valence Bonds: A New Kind of Insulator?: Materials Research Bulletin* 8.2 (1973), 153–160. DOI: [10.1016/0025-5408\(73\)90167-0](https://doi.org/10.1016/0025-5408(73)90167-0). (Visited on 03/24/2014).
- [74] P. W. Anderson. *The Resonating Valence Bond State in La₂CuO₄ and Superconductivity*. *Science* 235.4793 (1987), 1196–1198. DOI: [10.1126/science.235.4793.1196](https://doi.org/10.1126/science.235.4793.1196).
- [75] T. Nishino and K. Okunishi. *Corner Transfer Matrix Renormalization Group Method*. *Journal of the Physical Society of Japan* 65.4 (1996), 891–894. DOI: [10.1143/JPSJ.65.891](https://doi.org/10.1143/JPSJ.65.891).
- [76] R. Orús and G. Vidal. *Simulation of Two-Dimensional Quantum Systems on an Infinite Lattice Revisited: Corner Transfer Matrix for Tensor Contraction*. *Phys. Rev. B* 80.9 (2009), 094403. DOI: [10.1103/PhysRevB.80.094403](https://doi.org/10.1103/PhysRevB.80.094403).
- [77] L. Wang, I. Pižorn, and F. Verstraete. *Monte Carlo Simulation with Tensor Network States*. *Phys. Rev. B* 83.13 (2011), 134421. DOI: [10.1103/PhysRevB.83.134421](https://doi.org/10.1103/PhysRevB.83.134421).
- [78] Z. Y. Xie, J. Chen, M. P. Qin, J. W. Zhu, L. P. Yang, and T. Xiang. *Coarse-Graining Renormalization by Higher-Order Singular Value Decomposition*. *Phys. Rev. B* 86.4 (2012), 045139. DOI: [10.1103/PhysRevB.86.045139](https://doi.org/10.1103/PhysRevB.86.045139).
- [79] R. Alkabetz and I. Arad. *Tensor Networks Contraction and the Belief Propagation Algorithm*. *Phys. Rev. Res.* 3.2 (2021), 023073. DOI: [10.1103/PhysRevResearch.3.023073](https://doi.org/10.1103/PhysRevResearch.3.023073).
- [80] A. Kshetrimayum, H. Weimer, and R. Orús. *A Simple Tensor Network Algorithm for Two-Dimensional Steady States*. *Nature Communications* 8.1 (2017), 1291. DOI: [10.1038/s41467-017-01511-6](https://doi.org/10.1038/s41467-017-01511-6).
- [81] C. Mc Keever and M. H. Szymańska. *Stable iPEPO Tensor-Network Algorithm for Dynamics of Two-Dimensional Open Quantum Lattice Models*. *Phys. Rev. X* 11.2 (2021), 021035. DOI: [10.1103/PhysRevX.11.021035](https://doi.org/10.1103/PhysRevX.11.021035).
- [82] H. Weimer, A. Kshetrimayum, and R. Orús. *Simulation Methods for Open Quantum Many-Body Systems*. *Rev. Mod. Phys.* 93.1 (2021), 015008. DOI: [10.1103/RevModPhys.93.015008](https://doi.org/10.1103/RevModPhys.93.015008).

- [83] M.-D. Choi. *Completely Positive Linear Maps on Complex Matrices*. *Linear Algebra and Its Applications* 10.3 (1975), 285–290. DOI: [10.1016/0024-3795\(75\)90075-0](https://doi.org/10.1016/0024-3795(75)90075-0).
- [84] A. Jamiołkowski. *Linear Transformations Which Preserve Trace and Positive Semidefiniteness of Operators*. *Reports on Mathematical Physics* 3.4 (1972), 275–278. DOI: [10.1016/0034-4877\(72\)90011-0](https://doi.org/10.1016/0034-4877(72)90011-0).
- [85] A. H. Werner, D. Jaschke, P. Silvi, M. Kliesch, T. Calarco, J. Eisert, and S. Montangero. *Positive Tensor Network Approach for Simulating Open Quantum Many-Body Systems*. *Phys. Rev. Lett.* 116.23 (2016), 237201. DOI: [10.1103/PhysRevLett.116.237201](https://doi.org/10.1103/PhysRevLett.116.237201).
- [86] P. Zanardi. *Entanglement of Quantum Evolutions*. *Phys. Rev. A* 63.4 (2001), 040304. DOI: [10.1103/PhysRevA.63.040304](https://doi.org/10.1103/PhysRevA.63.040304).
- [87] T. Prosen and I. Pižorn. *Operator Space Entanglement Entropy in a Transverse Ising Chain*. *Phys. Rev. A* 76.3 (2007), 032316. DOI: [10.1103/PhysRevA.76.032316](https://doi.org/10.1103/PhysRevA.76.032316).
- [88] I. Pižorn and T. Prosen. *Operator Space Entanglement Entropy in XY Spin Chains*. *Phys. Rev. B* 79.18 (2009), 184416. DOI: [10.1103/PhysRevB.79.184416](https://doi.org/10.1103/PhysRevB.79.184416).
- [89] J. Dubail. *Entanglement Scaling of Operators: A Conformal Field Theory Approach, with a Glimpse of Simulability of Long-Time Dynamics in $1 + 1$ d*. *J. Phys. A Math. Theor.* 50.23 (2017), 234001. DOI: [10.1088/1751-8121/aa6f38](https://doi.org/10.1088/1751-8121/aa6f38).
- [90] A. E. Feiguin and S. R. White. *Finite-Temperature Density Matrix Renormalization Using an Enlarged Hilbert Space*. *Phys. Rev. B* 72.22 (2005), 220401. DOI: [10.1103/PhysRevB.72.220401](https://doi.org/10.1103/PhysRevB.72.220401).
- [91] H.-P. Breuer and F. Petruccione. *The Theory of Open Quantum Systems*. *The Theory of Open Quantum Systems*. Oxford University Press, 2007. ISBN: 978-0-19-921390-0. DOI: [10.1093/acprof:oso/9780199213900.002.14006](https://doi.org/10.1093/acprof:oso/9780199213900.002.14006).
- [92] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. S. L. Brandao, D. A. Buell, et al. *Quantum Supremacy Using a Programmable Superconducting Processor*. *Nature* 574.7779 (2019), 505–510. DOI: [10.1038/s41586-019-1666-5](https://doi.org/10.1038/s41586-019-1666-5).
- [93] M. Žnidarič. *Exact Solution for a Diffusive Nonequilibrium Steady State of an Open Quantum Chain*. *J. Stat. Mech. Theory Exp.* 2010.05 (2010), L05002. DOI: [10.1088/1742-5468/2010/05/L05002](https://doi.org/10.1088/1742-5468/2010/05/L05002).
- [94] C. Eckart and G. Young. *The Approximation of One Matrix by Another of Lower Rank*. *Psychometrika* 1.3 (1936), 211–218. DOI: [10.1007/BF02288367](https://doi.org/10.1007/BF02288367).
- [95] D. Perez-Garcia, F. Verstraete, M. M. Wolf, and J. I. Cirac. *Matrix Product State Representations*. *Quantum Info. Comput.* 7.5 (2007), 401–430.
- [96] M. Fannes, B. Nachtergaele, and R. F. Werner. *Finitely Correlated States on Quantum Spin Chains*. *Comm.Math. Phys.* 144.3 (1992), 443–490. DOI: [10.1007/BF02099178](https://doi.org/10.1007/BF02099178).